



Protégéo

SOMMAIRE

Présentation du projet.....	5
1. Contexte.....	6
2. Présentation de Protégéo	6
3. Recherches et analyse de l'existant	7
4. Cahier des charges fonctionnel	8
5. Charte graphique	8
5.1. Une identité visuelle inspirant confiance	8
5.2. Choix des couleurs	9
5.3. Typographie.....	9
5.4. Principes d'ergonomie	10
Développement.....	11
Partie 1 - Développer la partie Front-End d'une application web ou web mobile sécurisée	12
1. Installer et configurer son environnement de travail.....	12
1.1. Présentation de l'environnement.....	12
1.2. Outils utilisés.....	12
1.3. Architecture du projet.....	13
1.4. Gestion du code source	13
1.5. Configuration des environnements	13
2. Maquetter des interfaces utilisateur.....	13
2.1. Conception des maquettes	13
2.2. Organisation des interfaces	13
2.3. Présentation des principales maquettes	14

2.4.	Ergonomie des interfaces	15
3.	Réaliser des interfaces statiques	15
3.1.	Intégration des maquettes	15
3.2.	Création de composants réutilisables	15
3.3.	Réalisation des formulaires.....	16
3.4.	Construction du tableau de bord.....	16
3.5.	Profil de la personne protégée.....	16
3.6.	Mise en forme et organisation du CSS	17
4.	Développer les interfaces dynamiques	18
4.1.	Mise en place du routage.....	18
4.2.	Authentification	18
4.3.	Communication avec l'API	19
4.4.	Gestion des données.....	19
4.5.	Gestion des formulaires	20
4.6.	Mise à jour des données.....	20
4.7.	Réutilisation des composants	20
4.8.	Gestion des états.....	20

Partie 2 - Développer la partie Back-End d'une application web ou web mobile sécurisée

1.	Mettre en place une base de données relationnelle.....	22
1.1.	Analyse des besoins.....	22
1.2.	Modélisation de la base de données	22
1.3.	Création de la base de données MySQL	23
1.4.	Les principales tables	23
1.5.	Relations entre les données	24
2.	Développer des composants d'accès aux SQL et NoSQL.....	25

2.1.	Création des entités Symfony	25
2.2.	Repositories et Doctrine ORM	25
2.3.	Services métier	26
2.4.	Opérations CRUD	26
2.5.	Sécurisation de l'accès aux données.....	27
3.	Développer des composants métier côté serveur	27
3.1.	Organisation de la logique métier	28
3.2.	Développement d'un service métier.....	28
3.3.	Contrôle de l'accès aux données.....	29
3.4.	Traitement d'une requête par le contrôleur	30
3.5.	Validation des données et gestion des erreurs	31
3.6.	Test des composants métier	32
4.	Documenter le déploiement d'une application dynamique web et web mobile.	33
4.1.	Architecture de déploiement	33
4.2.	Préparation de l'hébergement.....	34
4.3.	Déploiement et configuration du Back-end Symfony	35
4.4.	Déploiement du Front-end React.....	36
4.5.	Sécurisation de l'application	37
4.6.	Mise à jour et maintenance.....	39
4.7.	État actuel du déploiement	41
	Annexes techniques	42

Présentation du projet

1. Contexte

Les mesures de protection juridique (tutelle, curatelle, sauvegarde de justice, etc.) sont mises en place afin d'accompagner les personnes majeures qui n'ont pas la capacité de gérer seules leurs intérêts personnels ou patrimoniaux. Les personnes désignées pour exercer ces mesures, qu'il s'agisse de tuteurs familiaux, de curateurs ou de mandataires judiciaires, doivent assurer un suivi administratif et financier rigoureux.

Chaque année, elles sont notamment tenues de réaliser un compte de gestion retraçant l'ensemble des ressources, des dépenses et de la situation bancaire de la personne protégée. Lors de l'ouverture d'une mesure, un inventaire du patrimoine ainsi qu'un budget prévisionnel doivent également être établis. Ces démarches reposent principalement sur des formulaires administratifs papier fournis par le ministère de la Justice.

Dans la pratique, ces documents nécessitent de rassembler de nombreuses informations provenant de différentes sources : relevés bancaires, justificatifs de dépenses, documents administratifs ou encore informations relatives au patrimoine. Cette gestion peut rapidement devenir complexe, en particulier pour les tuteurs et curateurs familiaux qui ne disposent pas d'outils spécialisés.

Ce constat est à l'origine du projet **Protégéo**. En tant que curatrice familiale, j'ai été confrontée à ces démarches administratives et aux difficultés qu'elles représentent au quotidien. Cette expérience m'a permis d'identifier un besoin réel de centralisation des informations et de simplification des procédures.

Le projet Protégéo a donc pour objectif de proposer une application web permettant de regrouper au sein d'une interface unique les informations administratives, patrimoniales et financières d'une personne protégée. L'application vise à faciliter le suivi des dossiers, à préparer les documents obligatoires et à accompagner les différents acteurs de la mesure de protection grâce à une interface claire, intuitive et sécurisée. Cette orientation est également décrite dans le cahier des charges du projet.

2. Présentation de Protégéo

Protégéo est une application web conçue pour accompagner les tuteurs familiaux, les curateurs, les mandataires judiciaires à la protection des majeurs ainsi que les personnes protégées dans la gestion de leurs démarches administratives et patrimoniales.

L'application s'adresse principalement aux tuteurs familiaux, curateurs, mandataires judiciaires à la protection des majeurs ainsi qu'aux subrogés tuteurs ou curateurs. À terme, un accès spécifique pourra également être proposé aux personnes protégées et à certains membres de leur famille, avec des droits adaptés à leur rôle.

Protégéo est composé de deux espaces complémentaires :

- **Une partie publique**, accessible à tous, qui présente les différentes mesures de protection juridique, les aides disponibles, des modèles de documents, les principales démarches administratives ainsi que des ressources utiles.

- **Une partie privée**, accessible après authentification, qui constitue le cœur de l'application. Elle permet aux utilisateurs autorisés de gérer un ou plusieurs dossiers de personnes protégées, de suivre les informations administratives, les comptes bancaires, les ressources, les dépenses, ainsi que les différents documents nécessaires au suivi de la mesure.

Protégéo est organisé autour de plusieurs modules permettant d'accompagner les utilisateurs tout au long de la gestion d'une mesure de protection juridique :

- Authentification ;
- Tableau de bord ;
- Dossiers ;
- Personne protégée ;
- Comptes ;
- Transactions.

Le développement de Protégéo repose sur une architecture moderne séparant le front-end et le back-end. L'interface utilisateur est développée avec **React**, tandis que la logique métier est implémentée au sein d'une **API Symfony** communiquant avec une base de données **MySQL**. Cette architecture facilite l'évolution de l'application et permet de distinguer clairement la présentation des données de leur traitement.

Enfin, Protégéo est conçu comme un projet évolutif. Les fonctionnalités actuellement développées constituent une première version de l'application. D'autres modules, tels que la génération automatique de documents, les notifications d'échéances ou encore un espace documentaire sécurisé, pourront être intégrés lors de futures versions.

3. Recherches et analyse de l'existant

Avant d'entamer la conception de Protégéo, une phase de recherche et d'analyse a été menée afin de mieux comprendre les besoins liés aux mesures de protection juridique et d'identifier les outils et documents existants.

La première étape a consisté à étudier les documents officiels utilisés dans le cadre du suivi d'une mesure de protection. Les formulaires **Inventaire et budget prévisionnel** (annexe A) ainsi que **Compte de gestion annuel** (Annexe B), fournis par le ministère de la Justice, ont servi de référence pour identifier les informations à collecter et les obligations auxquelles doivent répondre les tuteurs, curateurs et mandataires judiciaires. Ces documents ont notamment permis de définir les principales catégories de données qui seront gérées par l'application, telles que les informations relatives à la personne protégée, les comptes bancaires, les ressources, les dépenses, le patrimoine ou encore les actes de gestion.

En complément, plusieurs sites institutionnels ont été étudiés afin d'analyser leurs parcours utilisateurs, leur organisation et leurs choix ergonomiques. Les sites de la **CAF**, d'**Ameli** et de **Service-Public.fr** ont constitué des sources d'inspiration pour la présentation des informations administratives et la simplicité des démarches. Le **Système de Design de l'État (DSFR)** a également été consulté afin de s'appuyer sur des recommandations reconnues en matière d'accessibilité, de cohérence graphique et d'expérience utilisateur. Ces références figurent dans la charte graphique du projet.

L'ensemble de ces recherches et documents préparatoires a permis de définir une architecture fonctionnelle cohérente, de disposer d'une vision globale du projet avant le début du développement, et ont conduit à la réalisation de plusieurs livrables de conception présentés dans les chapitres suivants.

4. Cahier des charges fonctionnel

Avant le développement de l'application, un cahier des charges a été rédigé afin de formaliser les besoins fonctionnels et techniques du projet. Ce document a servi de fil conducteur tout au long de la conception et du développement de Protégéo. Il a permis de définir les objectifs de l'application, les utilisateurs visés, les fonctionnalités attendues ainsi que les contraintes à prendre en compte.

Le cahier des charges a également permis de planifier les différentes phases du projet et de prioriser les fonctionnalités développées au fil des versions.

Ce document a été régulièrement mis à jour au cours du projet afin de prendre en compte les nouveaux besoins identifiés lors de la conception et du développement.

Le cahier des charges complet est présenté dans les documents complémentaires.

5. Charte graphique

La charte graphique a été élaborée afin de garantir une identité visuelle cohérente sur l'ensemble de l'application. Elle définit les couleurs, les typographies, le logo ainsi que les principaux principes d'ergonomie qui ont servi de référence tout au long de la conception des maquettes et du développement de Protégéo.

5.1. Une identité visuelle inspirant confiance

Afin de créer une identité visuelle cohérente et rassurante, une charte graphique a été réalisée. Les utilisateurs manipulent quotidiennement des informations personnelles, administratives et financières sensibles. Il était donc essentiel de concevoir une interface inspirant confiance, sérénité et professionnalisme.

Le logo représente une silhouette humaine entourée de deux formes protectrices. Il symbolise l'accompagnement, la bienveillance et la protection apportés à la personne faisant l'objet d'une mesure de protection juridique. Le slogan retenu, « Maîtrisez le budget, facilitez la tutelle », rappelle l'objectif principal de l'application : simplifier les démarches administratives et financières des tuteurs et curateurs.



5.2. Choix des couleurs

Le choix des couleurs ne repose pas uniquement sur un aspect esthétique. Chaque couleur possède une signification fonctionnelle afin d'aider l'utilisateur à identifier rapidement les différentes actions de l'application.

Bleu profond (#003366)

Le bleu profond constitue la couleur principale de Protégéo. Traditionnellement associé à la confiance, à la stabilité et au sérieux, il est utilisé pour les éléments structurants de l'interface tels que la navigation principale, les titres et les boutons d'action principaux. Cette couleur renforce l'image institutionnelle de l'application.

Bleu clair (#0078A8)

Le bleu clair apporte davantage de légèreté et de proximité. Il est utilisé pour les liens, les icônes, certains éléments graphiques ainsi que les blocs d'information. Il permet également de distinguer les informations importantes sans créer un contraste trop agressif.

Vert (#4CAF50)

Le vert est réservé aux actions positives : validation d'un formulaire, confirmation d'une opération ou indicateur de réussite. Cette convention facilite la compréhension de l'interface et permet à l'utilisateur d'identifier rapidement les actions réussies.

Rouge (#D32F2F)

Le rouge est utilisé exclusivement pour signaler une erreur, un dépassement de budget ou une information nécessitant une attention particulière. Son utilisation est volontairement limitée afin de préserver son impact visuel.

Blanc et gris

Une large place est laissée au blanc afin d'aérer les interfaces et de limiter la charge cognitive. Le gris foncé est utilisé pour les textes afin d'assurer une bonne lisibilité tout en offrant un contraste moins agressif que le noir pur.

5.3. Typographie

La charte graphique s'appuie sur la police **Marianne®**, utilisée par les services de l'État français, pour les titres et les éléments d'interface. Cette police renforce le caractère institutionnel de Protégéo et crée une cohérence avec les sites publics consultés par les utilisateurs.

La police **Spectral** est utilisée pour certains éléments de mise en valeur. Des polices de substitution (Arial et Georgia) sont également prévues afin de garantir la compatibilité entre les différents navigateurs et systèmes d'exploitation.

Typographie

Marianne®

Spectral

Titres

Textes

5.4. Principes d'ergonomie

Au-delà de l'identité visuelle, la charte graphique définit plusieurs règles de conception destinées à améliorer l'expérience utilisateur.

L'interface a été pensée pour être rassurante, claire et facile à prendre en main. Les formulaires suivent une progression logique afin d'accompagner l'utilisateur étape par étape. Les informations importantes sont regroupées dans des blocs distincts, tandis que les textes réglementaires sont présentés de manière synthétique afin de limiter la surcharge d'informations. Les espaces blancs, les contrastes et la hiérarchisation visuelle permettent de rendre la lecture plus fluide et de réduire le stress lié à des démarches administratives souvent complexes.

Ces choix graphiques ont ensuite été appliqués lors de la réalisation des maquettes puis des interfaces de l'application, garantissant ainsi une cohérence visuelle entre la conception et le développement.

La charte graphique complète est présentée dans les documents complémentaires.

Développement

Partie 1 - Développer la partie Front-End d'une application web ou web mobile sécurisée

1. Installer et configurer son environnement de travail

1.1. Présentation de l'environnement

Avant de débuter le développement de Protégéo, un environnement de travail complet a été mis en place afin de disposer de tous les outils nécessaires au développement, aux tests et au suivi du projet. L'objectif était de séparer clairement la partie front-end de la partie back-end tout en facilitant le développement local.

1.2. Outils utilisés

Le développement a été réalisé sous Windows avec les outils suivants :

Outils	Utilisation
Visual Studio Code	Développement du front-end et du back-end
Git	Suivi des modifications du code
GitHub	Hébergement des dépôts et sauvegarde du projet
Laragon	Environnement de développement local
Composer	Gestion des dépendances Symfony
Symfony CLI	Exécution de l'API Symfony
Node.js et npm	Installation des dépendances et lancement du front-end
MySQL	Stockage des données
Postman	Tests des routes de l'API REST

1.3. Architecture du projet

Afin de faciliter la maintenance du projet, l'application a été séparée en deux parties distinctes :

- une API REST développée avec Symfony ;
- une interface utilisateur développée avec React et Vite.

Cette architecture permet de séparer l'interface utilisateur de la logique métier. Elle facilite la maintenance, les évolutions futures et le déploiement indépendant du front-end et du back-end.

1.4. Gestion du code source

Le code source est hébergé sur GitHub et sauvegardé régulièrement au cours du développement. Git a été utilisé pour conserver un historique des évolutions du projet et faciliter les sauvegardes des différentes fonctionnalités.

1.5. Configuration des environnements

Les informations sensibles, telles que les identifiants de connexion à la base de données ou les clés utilisées par l'application, sont stockées dans des fichiers de configuration (.env.local) qui ne sont pas versionnés. Cette approche permet de protéger les données sensibles tout en facilitant le déploiement de l'application sur différents environnements.

2. Maquetter des interfaces utilisateur

2.1. Conception des maquettes

Après la rédaction du cahier des charges et la définition de la charte graphique, les interfaces de Protégéo ont été conçues avec Figma. Les maquettes ont permis de définir l'organisation des différentes pages, la navigation entre les écrans ainsi que la disposition des composants avant le début du développement.

Compte tenu de la vision déjà établie du projet, les interfaces ont été réalisées directement sous forme de maquettes haute fidélité, sans passer par une étape de wireframes. Cette approche a permis d'intégrer dès la phase de conception les couleurs, les typographies et les composants définis dans la charte graphique.

2.2. Organisation des interfaces

L'application a été organisée autour de deux espaces distincts : une partie publique destinée à l'information des utilisateurs et une partie privée réservée aux personnes authentifiées.

L'espace privé repose sur un tableau de bord centralisant l'ensemble des fonctionnalités. Les différentes sections (profil, personne protégée, comptes bancaires, transactions ou compte de gestion)

sont accessibles depuis une navigation latérale commune à toutes les pages. Cette organisation garantit une navigation cohérente et facilite l'accès aux informations.

Les maquettes ont également permis d'harmoniser les différents composants de l'application, tels que les formulaires, les tableaux, les cartes d'information, les boutons et les zones de navigation. Cette homogénéité facilite la prise en main de l'application et limite les changements d'interface entre les différents modules.

Les principales maquettes de Protégéo sont présentées dans les figures 6 à 10 des annexes techniques.

2.3. Présentation des principales maquettes

La maquette de la page d'accueil est présentée dans la figure 6 des annexes techniques.

La page d'accueil constitue le point d'entrée de l'application. Elle présente Protégéo et oriente les utilisateurs vers les principales rubriques de l'espace public. Des cartes d'information mettent en avant les fonctionnalités essentielles, tandis que des contenus pratiques, comme les informations relatives aux versements de la CAF, permettent d'apporter rapidement des réponses aux utilisateurs avant leur authentification. La navigation a été conçue pour être simple et intuitive afin de faciliter l'accès aux différentes ressources disponibles.

La maquette du tableau de bord est présentée dans la figure 7 des annexes techniques.

Le tableau de bord constitue la page principale de l'espace privé. Il offre une vue d'ensemble des dossiers suivis grâce à des indicateurs, des échéances, des justificatifs à compléter et des actions rapides. Cette interface permet à l'utilisateur d'identifier rapidement les informations importantes dès sa connexion.

L'organisation du profil de la personne protégée est visible dans la figure 8 des annexes techniques.

Cette interface centralise l'ensemble des informations administratives relatives à la personne protégée. Les informations sont réparties dans plusieurs onglets thématiques (identité, jugement, famille, référents et contacts utiles), ce qui facilite leur consultation sans surcharger l'interface.

Les interfaces du compte de gestion et des comptes bancaires sont présentées dans les figures 9 et 10 des annexes techniques.

Cette interface reprend l'organisation du compte de gestion annuel afin de faciliter sa préparation. Les ressources, les dépenses, les comptes bancaires et les indicateurs financiers sont regroupés dans une seule interface pour offrir une vision globale de la situation financière de la personne protégée.

Cette interface permet de renseigner les informations relatives aux comptes bancaires utilisés dans le cadre du compte de gestion. Les différents comptes sont présentés de manière claire, avec leurs soldes, leurs mouvements et les principales informations bancaires nécessaires au suivi patrimonial.

2.4. Ergonomie des interfaces

Lors de la conception des maquettes, une attention particulière a été portée à l'ergonomie de l'application. Les interfaces utilisent une structure commune reposant sur une navigation latérale, des titres clairement identifiés et des composants réutilisables afin de limiter la charge cognitive de l'utilisateur.

Les formulaires sont organisés par thématiques afin de faciliter la saisie des informations. Les couleurs définies dans la charte graphique sont utilisées de manière cohérente pour distinguer les actions principales, les informations importantes et les différents états de l'application.

Les maquettes ont principalement été conçues pour un affichage sur ordinateur. Les utilisateurs de Protégéo réalisent régulièrement des démarches administratives nécessitant la consultation simultanée de nombreux documents et informations, ce qui rend ce format particulièrement adapté. Certaines pages publiques restent néanmoins consultables sur smartphone grâce à une interface responsive.

3. Réaliser des interfaces statiques

Après la réalisation des maquettes, leur intégration a été effectuée en développant les interfaces statiques de l'application. Cette étape a consisté à reproduire fidèlement les écrans conçus sous Figma à l'aide de HTML, CSS et React, avant d'y intégrer les fonctionnalités dynamiques.

L'objectif était de construire une base d'interfaces cohérente, réutilisable et conforme à la charte graphique définie lors de la phase de conception. Une attention particulière a été portée à l'organisation des composants, à la lisibilité des informations et à la cohérence visuelle entre les différentes pages.

3.1. Intégration des maquettes

La page d'accueil a été développée en React à partir des maquettes réalisées sous Figma. Elle est composée de plusieurs sections indépendantes, telles que le bandeau principal, les cartes de navigation et le bloc d'informations relatif aux versements de la CAF. Cette organisation permet de maintenir un code clair et de réutiliser facilement chaque composant dans d'autres pages.

3.2. Création de composants réutilisables

Bouton

*Le composant **Button**, capable de produire différents types d'éléments selon les propriétés reçues, est présenté dans la figure 12 des annexes techniques.*

Afin d'éviter la duplication de code, plusieurs composants réutilisables ont été développés. Le composant **Button** centralise l'affichage des boutons de l'application. Selon les propriétés transmises, il peut afficher un bouton classique, un lien externe ou une navigation interne avec React Router.

Cartes

Le composant réutilisable utilisé pour générer les cartes de redirection est présenté dans la figure 13 des annexes techniques.

Les cartes de navigation de la partie publique reposent sur un composant unique. Celui-ci reçoit son contenu (titre, description, image et lien) via des propriétés, ce qui permet de générer plusieurs cartes sans réécrire leur structure HTML.

Sections

La structure générique des sections de type « hero » est présentée dans la figure 14 des annexes techniques.

Les différentes sections de la partie publique utilisent également un composant générique permettant d'afficher un titre, une description, un badge et un bouton. Cette approche garantit une présentation homogène sur l'ensemble de l'application.

3.3. Réalisation des formulaires

Page d'inscription

L'interface et une partie du code du formulaire d'inscription sont présentées dans les figures 15 et 16 des annexes techniques.

Les formulaires ont été développés sous forme de composants afin de garantir une présentation homogène et de faciliter leur maintenance. Chaque champ est généré à partir d'une configuration décrivant son type, son libellé, son caractère obligatoire et son texte d'aide.

Connexion

La page de connexion et les principaux composants qui la constituent sont présentés dans les figures 17 à 18 des annexes techniques.

3.4. Construction du tableau de bord

Le tableau de bord constitue l'interface principale de l'espace privé. Il rassemble les informations essentielles relatives aux dossiers suivis et permet à l'utilisateur d'accéder rapidement aux différentes fonctionnalités de l'application. Son développement a reposé sur plusieurs composants réutilisables afin de garantir une interface homogène.

Les interfaces développées du tableau de bord et du profil de la personne protégée sont présentées dans les figures 19 et 20 des annexes techniques.

3.5. Profil de la personne protégée

La composition du composant principal du profil est illustrée par la figure 21 des annexes techniques.

L'écran de consultation de la personne protégée est composé de plusieurs composants réutilisables permettant d'afficher les informations personnelles, les différents groupes de champs ainsi que les actions de modification. Cette organisation facilite la maintenance du code et permet de conserver une présentation homogène dans l'ensemble de l'application.

3.6. Mise en forme et organisation du CSS

La mise en forme des interfaces a été réalisée en CSS afin de reproduire les maquettes conçues sous Figma et d'appliquer la charte graphique de Protégéo.

Les styles ont été organisés selon plusieurs niveaux. Les variables globales regroupent une partie des couleurs, des typographies, des tailles de texte et des espacements utilisés dans l'application. Les styles de base définissent ensuite les règles communes, tandis que les fichiers dédiés aux composants et aux layouts gèrent leur apparence spécifique.

Le CSS est principalement structuré autour des composants plutôt qu'autour des pages. Les boutons, cartes, formulaires, sections, tableaux et éléments de navigation disposent ainsi de leurs propres fichiers de style. Cette organisation correspond à l'architecture React du projet et permet de réutiliser un même composant dans plusieurs interfaces sans dupliquer ses règles visuelles.

Les mises en page reposent principalement sur Flexbox et CSS Grid. Flexbox est utilisé pour l'alignement des éléments, la navigation, les boutons et les groupes de champs. CSS Grid est utilisé lorsque l'interface nécessite une organisation en plusieurs colonnes, notamment pour certaines cartes et zones d'information.

Une partie des valeurs récurrentes a été centralisée dans des variables CSS. Cette organisation est progressivement appliquée à l'ensemble des fichiers afin d'améliorer la cohérence et de faciliter les futures modifications.

Enfin, des règles responsives ont également été ajoutées à l'aide de media queries. Elles permettent d'adapter la largeur des contenus, l'organisation des cartes, les espacements et la navigation selon la taille de l'écran. La navigation desktop est remplacée par un menu mobile et certains éléments sont réorganisés pour préserver la lisibilité. Les interfaces du tableau de bord restent principalement optimisées pour une utilisation sur ordinateur, en raison de la quantité d'informations administratives et financières affichées simultanément. Certaines adaptations pourront être complétées dans une future version pour améliorer son utilisation sur tablette et mobile.

Les principales valeurs de la charte graphique sont centralisées dans des variables CSS. Elles regroupent notamment les typographies, les tailles de texte, les couleurs, les graisses, les ombrages, les espaces et les bordures arrondies, utilisées dans l'application. Cette organisation facilite la cohérence visuelle et permet de modifier une valeur globale sans intervenir dans chaque fichier de style.

Un fichier de base définit les règles communes à l'ensemble de l'application, telles que la suppression des marges par défaut, le modèle de boîte, les typographies, les couleurs principales et le comportement des images. Ces règles constituent la base visuelle sur laquelle reposent ensuite les composants et les pages.

Les styles sont principalement regroupés par composant. Le composant de section possède ainsi ses propres règles pour gérer son alignement, ses espacements, sa largeur, sa typographie et ses variantes visuelles. Cette séparation facilite la réutilisation du composant dans plusieurs pages tout en conservant une présentation homogène.

Les variantes, comme celles utilisées sur la page d'accueil ou dans le tableau de bord, complètent la classe principale sans dupliquer toute la structure du composant.

Les adaptations responsives sont définies à l'aide de media queries. Pour les écrans plus étroits, la navigation principale est masquée au profit d'un bouton burger et d'un menu mobile. Les dimensions et les espacements sont également ajustés afin de conserver une interface lisible.

Les variables globales et un exemple d'adaptation responsive sont présentés dans les figures 22 et 23 des annexes techniques.

4. Développer les interfaces dynamiques

Après la réalisation des interfaces statiques, les fonctionnalités dynamiques ont été développées avec React. Cette étape a permis de gérer la navigation entre les pages, l'authentification des utilisateurs, la récupération des données depuis l'API Symfony ainsi que leur affichage et leur mise à jour.

Afin de conserver une architecture claire, les responsabilités ont été séparées entre les composants d'interface, les hooks personnalisés, les services chargés de communiquer avec l'API et les fonctions utilitaires. Cette organisation facilite la maintenance du projet et limite la duplication de code.

4.1. Mise en place du routage

La navigation de Protégéo est gérée avec React Router. Les routes ont été organisées selon les différents espaces de l'application : pages publiques, authentification, guides et tableau de bord. Les pages privées sont protégées afin de vérifier que l'utilisateur est connecté avant de lui donner accès aux fonctionnalités du tableau de bord.

Cette organisation permet de centraliser la navigation, de séparer clairement les différentes catégories de pages et de faciliter l'ajout de nouvelles routes au cours du développement.

Le point d'entrée du routage et l'organisation des routes publiques et protégées sont présentés dans la figure 24 des annexes techniques.

4.2. Authentification

L'authentification permet de sécuriser l'accès à l'espace privé de Protégéo. Le formulaire de connexion récupère les identifiants saisis, vérifie leur validité puis les transmet à l'API Symfony.

Lorsque l'authentification réussit, l'API retourne un jeton JWT, conservé dans le navigateur et ajouté aux requêtes nécessitant une authentification. L'utilisateur est ensuite redirigé vers le tableau de bord. Les erreurs de saisie, les erreurs retournées par l'API et l'état de chargement sont traités séparément afin d'informer clairement l'utilisateur.

Le composant de connexion, le hook chargé de gérer son état et le service d'authentification sont présentés dans les figures 25 à 27 des annexes techniques.

4.3. Communication avec l'API

Après avoir mis en place le routage et l'authentification, l'étape suivante a consisté à connecter l'application React à l'API Symfony afin de récupérer et de modifier les données.

Les échanges entre l'interface React et l'API Symfony sont regroupés dans des services dédiés. Une fonction commune construit les requêtes, ajoute les en-têtes nécessaires et traite les réponses retournées par l'API.

Les opérations propres à chaque fonctionnalité sont ensuite réparties dans des services spécialisés. Pour les contacts, des services distincts permettent de récupérer, d'ajouter ou de modifier une donnée. Cette séparation garde les fonctions courtes et facilite leur maintenance.

La fonction générique utilisée pour communiquer avec l'API est présentée dans la figure 28 des annexes techniques.

Des exemples de services dédiés à la lecture, à la création et à la modification des contacts sont présentés dans la figure 29 des annexes techniques.

4.4. Gestion des données

Une fois les échanges avec l'API mis en place, il a été nécessaire d'organiser la récupération des données au sein de hooks personnalisés.

Les données récupérées depuis l'API sont gérées par des hooks personnalisés. Ceux-ci regroupent les appels aux services, les données reçues, l'état de chargement et les éventuelles erreurs.

Cette organisation évite de placer toute la logique de récupération directement dans les composants d'affichage. Les composants reçoivent ainsi des informations déjà préparées et peuvent se concentrer sur leur présentation.

Le hook chargé de récupérer la personne protégée est présenté dans la figure 30 des annexes techniques.

La gestion générique des formulaires modifiables est présentée dans la figure 31 des annexes techniques.

4.5. Gestion des formulaires

Les différents formulaires de l'application partageant une logique commune, leur gestion a été factorisée afin d'éviter la duplication de code.

Plusieurs formulaires de l'application partagent les mêmes besoins : initialiser les valeurs, passer en mode édition, modifier un champ, annuler les changements et enregistrer les nouvelles données.

Pour éviter de répéter cette logique, elle a été regroupée dans des hooks personnalisés. Un hook générique gère les formulaires modifiables, tandis que des hooks plus spécifiques prennent en charge les particularités de certaines fonctionnalités, comme l'ajout des contacts familiaux.

La composition de la section consacrée aux contacts familiaux ainsi que les composants réutilisables associés sont présentés dans les figures 32 à 33 des annexes techniques.

4.6. Mise à jour des données

La modification des informations repose sur des hooks et des services spécialisés. Lorsqu'un formulaire est validé, les nouvelles données sont transmises à l'API avec une requête adaptée, puis l'interface est actualisée.

Les états de chargement et d'erreur permettent d'empêcher les soumissions multiples et d'indiquer à l'utilisateur si l'enregistrement a réussi ou échoué.

4.7. Réutilisation des composants

L'interface a été construite à partir de composants réutilisables afin de conserver une présentation homogène et de limiter la duplication du code.

Les composants principaux assemblent les données, les hooks et les composants d'interface. Les composants plus simples, comme les listes ou les cartes, reçoivent les informations à afficher sous forme de propriétés. Cette organisation permet d'utiliser une même structure pour plusieurs types de contacts ou plusieurs sections de l'application.

4.8. Gestion des états

Les interfaces dynamiques reposent principalement sur les hooks `useState` et `useEffect`.

`useState` conserve les valeurs des formulaires, les données récupérées, l'ouverture des fenêtres modales, les chargements et les erreurs. `useEffect` déclenche certaines actions lorsqu'une donnée

évolue, par exemple le chargement d'une personne protégée lorsque l'utilisateur sélectionne un autre dossier.

La séparation des états de chargement, d'erreur et de contenu permet d'adapter l'affichage à chaque situation et de rendre l'interface réactive aux actions de l'utilisateur.

Partie 2 - Développer la partie Back-End d'une application web ou web mobile sécurisée

1. Mettre en place une base de données relationnelle

1.1. Analyse des besoins

Avant de concevoir la base de données, une phase d'analyse a été réalisée afin d'identifier les informations que l'application devait gérer. Cette analyse s'est appuyée sur plusieurs éléments complémentaires.

Les maquettes fonctionnelles ont permis de définir les différents écrans de l'application et les informations attendues dans chaque interface. Les formulaires administratifs officiels, notamment ceux utilisés dans le cadre des mesures de protection juridique, ont également servi de référence afin d'identifier les données réellement demandées par les organismes et les juridictions.

Enfin, une réflexion fonctionnelle a été menée en se plaçant du point de vue d'un curateur. Étant moi-même curatrice familiale, j'ai pu m'appuyer sur mon expérience personnelle afin d'identifier les difficultés rencontrées au quotidien et les fonctionnalités susceptibles de faciliter la gestion d'une mesure de protection.

Cette démarche a permis d'identifier les principales entités de l'application, telles que les utilisateurs, les dossiers, les personnes protégées, les mesures de protection, les contacts, les comptes bancaires, les opérations financières et les comptes de gestion. Les relations entre ces données ont ensuite servi de base à la conception du modèle conceptuel de données (MCD), puis du modèle logique de données (MLD).

Le cahier des charges et les formulaires administratifs officiels sont présents en annexe.

1.2. Modélisation de la base de données

À partir de l'analyse des besoins, un modèle de données a été élaboré afin de structurer les informations nécessaires au fonctionnement de Protégéo. Cette étape avait pour objectif de définir les différentes entités de l'application, leurs attributs ainsi que les relations qui les unissent.

La conception a débuté par la réalisation d'un **Modèle Conceptuel de Données (MCD)**. Ce modèle permet de représenter les données de manière fonctionnelle, indépendamment du système de gestion de base de données. Il met en évidence les principales entités de l'application, telles que les dossiers, les utilisateurs, les personnes protégées, les mesures de protection, les contacts, les comptes bancaires, les transactions et les comptes de gestion, ainsi que leurs relations.

Le MCD a ensuite été transformé en **Modèle Logique de Données (MLD)**. Cette étape consiste à adapter le modèle conceptuel vers une structure relationnelle exploitable par une base de données

SQL. Les clés primaires, les clés étrangères, les types de données et les différentes contraintes ont été définis afin de garantir l'intégrité des informations.

Cette modélisation constitue le socle de la base de données de Protégéo. Elle facilite le développement des entités Symfony, la génération des migrations Doctrine ainsi que la mise en place des relations entre les différentes tables de l'application.

Le Modèle Conceptuel de Données et le Modèle Logique de Données de Protégéo sont présentés dans les figures 34 et 35 des annexes techniques.

1.3. Création de la base de données MySQL

Une fois le modèle logique de données validé, la base de données relationnelle a été mise en place sous MySQL. Ce système de gestion de base de données a été retenu pour sa fiabilité, ses performances et sa parfaite compatibilité avec le framework Symfony.

La structure de la base a été implémentée à partir des entités du projet. Chaque entité Symfony représente une table de la base de données et définit ses différents attributs ainsi que les relations avec les autres entités. Les associations entre les tables (OneToOne, OneToMany et ManyToOne) ont été configurées directement dans les classes afin de garantir la cohérence des données.

Les migrations Doctrine ont ensuite été générées afin de créer automatiquement la structure de la base de données. Ces scripts de migration permettent de suivre l'historique des changements de la base de données et de la garder toujours alignée avec le code de l'application.

Enfin, la base de données a été vérifiée à l'aide de phpMyAdmin afin de contrôler la création des tables, des clés étrangères et des relations définies lors de la phase de conception.

La traduction du modèle en entités Symfony et migrations Doctrine est illustrée par les figures 36 et 37 des annexes techniques.

1.4. Les principales tables

La base de données de Protégéo est composée de plusieurs tables permettant de gérer les différents aspects d'une mesure de protection juridique. Chaque table correspond à une entité métier et possède un rôle précis dans le fonctionnement de l'application.

Les tables suivantes constituent le cœur de la base de données de Protégéo. Elles regroupent les principales informations nécessaires au fonctionnement de l'application.

Table	Description
users	Stocke les informations des utilisateurs de l'application ainsi que leurs données de connexion.

dossiers	Représente un dossier de mesure de protection et sert de point central entre les différentes entités.
dossier_users	Associe un ou plusieurs utilisateurs à un dossier en fonction de leur rôle (curateur, tuteur, subrogé...).
protected_person	Contient les informations personnelles de la personne protégée.
measure_protection	Enregistre les caractéristiques de la mesure de protection (type, dates, tribunal, durée...).
contacts	Regroupe les membres de la famille, les personnes de confiance ainsi que les organismes et professionnels liés au dossier.
bank_accounts	Réfère les comptes bancaires et produits financiers de la personne protégée.
transactions	Enregistre les ressources et les dépenses utilisées pour le compte de gestion.
banking_transactions	Gère les virements entre les différents comptes bancaires.
management_account	Permet de suivre le compte de gestion annuel et son état d'avancement.

1.5. Relations entre les données

L'ensemble des tables de la base de données est relié par des clés primaires et des clés étrangères afin d'assurer la cohérence des informations. Cette organisation permet de limiter les redondances, de garantir l'intégrité des données et de faciliter les échanges entre les différentes fonctionnalités de l'application.

Le dossier constitue l'élément central de la base de données. Il regroupe l'ensemble des informations relatives à une mesure de protection et sert de point de liaison entre les utilisateurs, la personne protégée, la mesure de protection, les contacts, les comptes bancaires, les transactions, les inventaires et le compte de gestion.

Les relations ont été définies en fonction des besoins fonctionnels de l'application. Par exemple, un dossier peut être associé à plusieurs utilisateurs ayant des rôles différents, contenir plusieurs contacts et plusieurs comptes bancaires, tandis qu'une personne protégée et une mesure de protection sont directement rattachées à un seul dossier. Cette organisation garantit une structure claire, évolutive et adaptée aux besoins de Protégéo.

Cette organisation relationnelle facilite également le développement de l'application Symfony, les associations entre les entités étant directement exploitées par Doctrine lors des opérations de lecture, de création et de mise à jour des données.

La structure obtenue dans la base de données et les principales relations sont présentées dans les figures 38 et 39 des annexes techniques.

2. Développer des composants d'accès aux SQL et NoSQL

2.1. Création des entités Symfony

À partir du modèle logique de données, les différentes tables de la base ont été représentées sous forme d'entités Symfony. Chaque entité correspond à un objet métier de l'application et décrit les propriétés qui seront enregistrées dans la base de données.

Les attributs Doctrine permettent de définir le nom des colonnes, leur type, leur longueur, leur caractère obligatoire ou facultatif ainsi que les contraintes associées. Les relations entre les entités sont également configurées directement dans les classes à l'aide d'associations telles que OneToOne, OneToMany et ManyToOne.

Cette organisation permet de manipuler les données sous forme d'objets PHP plutôt que d'écrire directement des requêtes SQL pour chaque opération. Doctrine assure ensuite la correspondance entre les entités de l'application et les tables de la base MySQL.

Les entités ont été créées progressivement en fonction des fonctionnalités développées. Elles représentent notamment les utilisateurs, les dossiers, les personnes protégées, les mesures de protection, les contacts, les comptes bancaires, les transactions et les comptes de gestion.

Un exemple d'entité Symfony utilisant les attributs Doctrine est présenté dans la figure 36 des annexes techniques.

2.2. Repositories et Doctrine ORM

Une fois les entités créées, Symfony génère automatiquement un repository pour chacune d'entre elles. Les repositories constituent la couche d'accès aux données et permettent de récupérer les informations enregistrées dans la base de données.

Doctrine ORM assure la communication entre les objets PHP de l'application et la base de données MySQL. Il permet de manipuler les entités Symfony sans avoir à écrire directement des requêtes SQL pour les opérations les plus courantes. Les repositories sont utilisés pour récupérer les données, tandis que l'EntityManager permet d'enregistrer, modifier ou supprimer les entités. Les migrations Doctrine permettent quant à elles de faire évoluer la structure de la base de données en fonction des modifications apportées aux entités.

Les méthodes fournies par Doctrine, telles que find(), findOneBy() ou findBy(), répondent aux besoins les plus courants. Lorsque des recherches plus spécifiques sont nécessaires, des méthodes personnalisées sont développées afin de filtrer les données selon les besoins de l'application.

Dans Protégéo, plusieurs repositories ont été enrichis avec des requêtes adaptées au fonctionnement du projet. Par exemple, les informations peuvent être récupérées en fonction du dossier concerné afin de limiter l'accès aux données autorisées. Cette organisation centralise les requêtes d'accès aux données dans une seule couche de l'application et facilite leur maintenance.

Les repositories sont ensuite utilisés par les services métier, qui exploitent les données récupérées pour appliquer les différentes règles de gestion avant de retourner les résultats aux contrôleurs.

Un exemple de méthode personnalisée dans un repository Doctrine est présenté dans la figure 40 des annexes techniques.

2.3. Services métier

Afin de respecter une architecture claire et facilement maintenable, la logique métier de l'application a été regroupée dans des services Symfony. Les contrôleurs ont ainsi pour rôle de recevoir les requêtes HTTP, de vérifier les données reçues et de déléguer les traitements aux services correspondants.

Chaque service est dédié à une fonctionnalité précise de l'application. Cette organisation permet de centraliser les traitements, d'éviter la duplication de code et de faciliter les évolutions futures du projet. Les services sont notamment responsables de la création, de la modification, de la suppression et de la récupération des données, tout en appliquant les règles de gestion propres à Protégéo.

Plusieurs services ont été développés au cours du projet, notamment pour la gestion des dossiers, des personnes protégées, des mesures de protection, des contacts, des comptes bancaires, des transactions financières ou encore des comptes de gestion. Ils utilisent les repositories Doctrine pour accéder aux données, puis renvoient les résultats aux contrôleurs qui construisent les réponses destinées au Front-end.

Cette séparation des responsabilités améliore la lisibilité du code et facilite sa maintenance. Lorsqu'une règle métier doit être modifiée, le changement est réalisé directement dans le service concerné, sans impacter les autres couches de l'application.

Un exemple de service métier Symfony est présenté dans la figure 41 des annexes techniques.

2.4. Opérations CRUD

Les services développés dans Protégéo permettent de réaliser les principales opérations de gestion des données, communément appelées CRUD (Create, Read, Update, Delete). Ces opérations sont utilisées dans l'ensemble de l'application afin de créer, consulter, modifier ou supprimer les informations enregistrées dans la base de données.

Les opérations de lecture permettent de récupérer les données nécessaires à l'affichage des différentes interfaces, comme les dossiers, les personnes protégées, les contacts ou les comptes bancaires. Les opérations de création sont utilisées lors de l'ajout de nouvelles informations, tandis

que les opérations de modification permettent de mettre à jour les données existantes. Enfin, certaines fonctionnalités autorisent la suppression d'un enregistrement lorsqu'il n'est plus nécessaire.

Chaque opération est réalisée par un service métier, qui utilise les repositories Doctrine pour accéder à la base de données. Les contrôleurs appellent ensuite ces services afin de répondre aux requêtes du Front-end tout en garantissant le respect des règles de gestion définies dans l'application.

Cette organisation permet de proposer une architecture claire, dans laquelle chaque couche possède une responsabilité précise : les contrôleurs gèrent les requêtes HTTP, les services appliquent la logique métier et les repositories assurent les échanges avec la base de données.

Les principales opérations CRUD regroupées dans ce service sont également illustrées par la figure 41 des annexes techniques.

2.5. Sécurisation de l'accès aux données

La sécurité constitue un élément essentiel de Protégéo, l'application manipulant des données personnelles, administratives et financières particulièrement sensibles. Plusieurs mécanismes ont été mis en place afin de protéger les informations stockées en base de données et de limiter les accès aux seuls utilisateurs autorisés.

L'API Symfony est sécurisée par une authentification basée sur des jetons JWT (JSON Web Token). Après la connexion, chaque requête transmise par le Front-end contient un jeton permettant d'identifier l'utilisateur et de vérifier ses droits d'accès.

Au-delà de l'authentification, les services métier vérifient systématiquement que les données demandées appartiennent bien au dossier concerné. Cette vérification permet d'empêcher qu'un utilisateur puisse accéder ou modifier les informations d'un autre dossier en manipulant simplement un identifiant dans l'URL.

Les informations sensibles de l'application sont également protégées à plusieurs niveaux. Les mots de passe des utilisateurs sont stockés sous forme de hachage, conformément aux mécanismes de sécurité fournis par Symfony. Les paramètres de configuration, tels que les identifiants de connexion à la base de données ou les clés de l'application, sont conservés dans des fichiers d'environnement non versionnés afin de ne pas être diffusés avec le code source.

Cette approche permet de renforcer la confidentialité des données tout en garantissant un accès sécurisé aux différentes fonctionnalités de l'application.

La vérification des droits d'accès avant la consultation d'un compte bancaire est présentée dans la figure 42 des annexes techniques.

3. Développer des composants métier côté serveur

Le développement du Back-end de Protégéo ne repose pas uniquement sur l'accès aux données. L'application doit également appliquer des règles de gestion liées aux dossiers, aux utilisateurs, aux mesures de protection, aux contacts, aux comptes bancaires et aux transactions.

Afin d'organiser cette logique métier, l'API Symfony a été structurée en plusieurs couches. Les contrôleurs reçoivent les requêtes HTTP envoyées par le Front-end, les services exécutent les traitements métier, tandis que les repositories assurent la communication avec la base de données.

Cette séparation permet de limiter la quantité de logique présente dans les contrôleurs et de centraliser les traitements réutilisables dans des services dédiés. Elle améliore la lisibilité du code, facilite sa maintenance et évite la duplication de règles identiques dans plusieurs routes.

Les composants métier développés dans Protégéo assurent notamment la vérification des droits d'accès, la récupération des ressources associées à un dossier, la validation des données reçues ainsi que la création, la modification et la suppression des informations enregistrées.

3.1. Organisation de la logique métier

L'architecture du Back-end de Protégéo repose sur une séparation claire des responsabilités entre les contrôleurs, les services et les repositories.

Les contrôleurs constituent le point d'entrée de l'API. Ils reçoivent les requêtes envoyées par l'application React, récupèrent les paramètres transmis, vérifient l'authentification de l'utilisateur et appellent les services correspondant à l'action demandée.

Les services regroupent la logique métier de l'application. Ils permettent notamment de vérifier les droits d'accès à un dossier, de rechercher une ressource précise, de préparer une nouvelle entité ou d'appliquer les modifications demandées.

Les repositories centralisent les requêtes vers la base de données. Ils utilisent Doctrine afin de récupérer uniquement les informations nécessaires et de respecter les relations définies entre les différentes entités.

Cette organisation évite de placer l'ensemble des traitements dans les contrôleurs. Chaque classe conserve ainsi un rôle précis :

- le contrôleur gère la requête et la réponse HTTP ;
- le service applique les règles métier ;
- le repository interroge la base de données ;
- l'entité représente les données enregistrées.

L'organisation générale de la logique métier consacrée aux contacts est illustrée par la figure 43 des annexes techniques.

3.2. Développement d'un service métier

Les services ont pour rôle de centraliser les traitements propres au fonctionnement de l'application. Ils sont appelés par les contrôleurs et coordonnent les différentes opérations nécessaires avant l'enregistrement ou la récupération des données.

Dans Protégéo, la gestion des mouvements bancaires constitue un exemple de composant métier côté serveur. Un mouvement bancaire correspond à un transfert réalisé entre deux comptes appartenant à une même personne protégée. Contrairement à une ressource ou à une dépense, ce mouvement ne modifie pas le patrimoine global de la personne : il déplace une somme d'un compte vers un autre.

Le service `BankingTransactionService` regroupe les opérations permettant de consulter, créer, modifier et supprimer ces mouvements. Il contrôle également les comptes bancaires concernés avant de transmettre les modifications à Doctrine.

Pour retrouver un mouvement précis, le service ne se limite pas à son identifiant. Il utilise également l'identifiant du dossier afin de vérifier que le mouvement demandé est bien associé à l'un des comptes de la personne protégée concernée. Cette règle empêche qu'un utilisateur puisse consulter ou modifier une opération appartenant à un autre dossier en changeant simplement l'identifiant contenu dans l'URL.

Lors de la création d'un mouvement, les données reçues sont contrôlées avant l'enregistrement. Le service vérifie notamment le montant, la date de l'opération ainsi que les comptes source et destinataire. Les deux comptes doivent exister et être accessibles depuis le dossier sélectionné.

Une fois ces vérifications effectuées, le service crée l'entité, lui associe les comptes bancaires correspondants, puis demande à l'`EntityManager` de l'enregistrer dans la base de données. La logique est ainsi centralisée dans une classe réutilisable, tandis que le contrôleur reste principalement chargé de recevoir la requête et de construire la réponse HTTP.

Les méthodes de consultation des mouvements bancaires sont présentées dans la figure 44 des annexes techniques.

La création d'un mouvement entre deux comptes est illustrée par la figure 45 des annexes techniques.

3.3. Contrôle de l'accès aux données

Protégéo manipule des données personnelles, administratives et financières sensibles. Il est donc nécessaire de vérifier qu'un utilisateur authentifié ne peut consulter que les dossiers auxquels il est réellement associé.

Le contrôle d'accès est effectué dès le traitement de la requête par le contrôleur. L'utilisateur connecté est d'abord récupéré avec la méthode `getUser()`. Si aucun utilisateur n'est authentifié, l'API renvoie une réponse avec le code HTTP 401.

Le dossier demandé est ensuite recherché à partir de son identifiant et de l'utilisateur connecté. La méthode `findOneByIdAndUser()` vérifie directement dans la requête que le dossier appartient bien aux dossiers accessibles par cet utilisateur.

Si aucun dossier correspondant n'est trouvé, le contrôleur renvoie une réponse d'erreur. Le message reste volontairement commun aux cas « dossier introuvable » et « accès refusé », afin de ne pas révéler l'existence d'un dossier à un utilisateur non autorisé.

Une seconde vérification est appliquée lors de la récupération du compte bancaire. Le service `BankAccountService` utilise la méthode `getByIdAndDossierId()`, qui recherche le compte à partir de son identifiant et de celui du dossier.

Cette double vérification permet de sécuriser l'accès à la ressource. Même si un utilisateur modifie manuellement l'identifiant du compte bancaire dans l'URL, l'API ne retournera aucune donnée si ce compte n'appartient pas au dossier autorisé.

Le contrôle d'accès repose ainsi sur plusieurs niveaux : authentification de l'utilisateur, vérification de son accès au dossier et contrôle de l'appartenance de la ressource à ce dossier. Cette organisation limite les accès non autorisés et renforce la confidentialité des données enregistrées dans Protégéo.

Les différentes étapes du contrôle d'accès sont présentées dans les figures 46 et 47 des annexes techniques.

3.4. Traitement d'une requête par le contrôleur

Le contrôleur constitue le point d'entrée de l'API. Il reçoit les requêtes HTTP envoyées par le Front-end React, récupère les paramètres transmis, appelle les services nécessaires puis construit la réponse retournée au client.

Dans Protégéo, chaque méthode de contrôleur correspond à une action précise, comme consulter une ressource, créer une nouvelle donnée, la modifier ou la supprimer. Les routes Symfony permettent d'associer une URL et une méthode HTTP à chacune de ces actions.

Lorsqu'une requête est reçue, le contrôleur commence par récupérer les informations nécessaires à son traitement. Il peut s'agir de paramètres présents dans l'URL, de données envoyées dans le corps de la requête ou de l'utilisateur authentifié.

Après les vérifications d'accès, le contrôleur transmet les informations au service métier correspondant. Le service exécute alors les traitements nécessaires, comme rechercher une ressource, valider des données ou enregistrer une nouvelle entité.

Le contrôleur récupère ensuite le résultat du service, le formate si nécessaire et le retourne au Front-end sous la forme d'une réponse JSON accompagnée d'un code HTTP adapté.

Cette organisation permet de conserver des contrôleurs lisibles. Ils assurent principalement la coordination entre la requête reçue, les services métier et la réponse HTTP, tandis que les traitements plus complexes restent centralisés dans les services.

Exemple avec la consultation d'un compte bancaire

La méthode `show()` du `BankAccountController` illustre ce fonctionnement.

Elle reçoit l'identifiant du dossier et celui du compte bancaire depuis l'URL. Elle récupère ensuite l'utilisateur connecté, recherche le dossier accessible par cet utilisateur puis appelle le `BankAccountService` afin de récupérer le compte demandé.

Si le compte bancaire n'existe pas dans le dossier, le contrôleur renvoie une réponse d'erreur. Dans le cas contraire, les données sont formatées puis retournées au Front-end avec un code HTTP 200.

Le contrôleur ne réalise donc pas directement la requête vers la base de données. Il délègue cette responsabilité au service, qui appelle lui-même le repository correspondant.

Cette séparation permet de limiter la logique présente dans les contrôleurs et de rendre le traitement des requêtes plus facile à comprendre. Chaque couche possède une responsabilité précise, ce qui facilite la maintenance et l'évolution de l'API.

Un exemple de traitement d'une requête de création par un contrôleur ainsi que le parcours général d'une requête sont présentés dans les figures 48 et 49 des annexes techniques.

3.5. Validation des données et gestion des erreurs

Les données envoyées par le Front-end ne sont jamais enregistrées directement dans la base de données. Elles sont d'abord contrôlées afin de vérifier leur présence, leur format et leur cohérence avec les règles de gestion de l'application.

Dans Protégéo, une première validation est effectuée dans le contrôleur. Celui-ci vérifie que le contenu de la requête peut être converti en tableau, puis contrôle la présence des informations nécessaires à l'opération demandée.

La création d'un mouvement bancaire nécessite notamment l'identifiant d'un compte source et celui d'un compte destinataire. Ces identifiants sont récupérés dans les données JSON puis validés afin de vérifier qu'ils correspondent bien à des nombres entiers.

Lorsqu'une donnée obligatoire est absente ou incorrecte, le traitement est immédiatement interrompu. Le contrôleur renvoie alors une réponse JSON contenant un message explicite ainsi qu'un code HTTP 400, correspondant à une requête invalide.

Après cette première étape, les comptes bancaires sont recherchés à partir de leur identifiant et de celui du dossier. Cette vérification permet de confirmer leur existence et leur appartenance au dossier consulté.

Le service métier applique ensuite des contrôles complémentaires. Il vérifie notamment les données obligatoires du mouvement, le format de la date d'opération et la cohérence entre les comptes source et destinataire.

Cette validation répartie entre le contrôleur et le service permet de distinguer deux responsabilités :

- le contrôleur vérifie la structure générale de la requête et les paramètres nécessaires ;
- le service applique les règles métier propres au mouvement bancaire.

En cas d'erreur, une exception ou une réponse HTTP adaptée est retournée au Front-end. Cette organisation évite l'enregistrement de données incorrectes et fournit à l'utilisateur des informations compréhensibles sur l'origine du problème.

Les réponses de l'API utilisent des codes HTTP adaptés au résultat du traitement :

- 200 lorsque la requête est exécutée avec succès ;
- 201 lorsqu'une ressource est créée ;
- 400 lorsque les données envoyées sont absentes ou invalides ;
- 401 lorsque l'utilisateur n'est pas authentifié ;
- 404 lorsqu'un dossier, un compte ou une autre ressource n'est pas trouvé.

Ces codes permettent au Front-end d'identifier rapidement le résultat d'une requête et d'afficher un message adapté à l'utilisateur.

La validation est ainsi effectuée à plusieurs étapes du traitement. Le contrôleur vérifie la structure de la requête, tandis que le service applique les règles propres au métier. Cette organisation protège la cohérence de la base de données et facilite la gestion des erreurs par l'interface React.

Des exemples de validation et de gestion des erreurs sont présentés dans les figures 51 à 53 des annexes techniques.

3.6. Test des composants métier

Après le développement des contrôleurs et des services métier, les routes de l'API ont été testées avec Postman. Cet outil permet d'envoyer directement des requêtes HTTP au Back-end, sans passer par l'interface React.

Ces tests permettent de vérifier le parcours complet d'une requête : authentification de l'utilisateur, contrôle de l'accès au dossier, validation des données reçues, exécution du service métier et construction de la réponse JSON.

Pour accéder aux routes protégées, l'utilisateur doit d'abord s'authentifier. L'API retourne alors un jeton JWT, qui est ajouté dans l'en-tête Authorization des requêtes suivantes.

Plusieurs scénarios ont été vérifiés :

- consultation des données associées à un dossier ;
- création d'une nouvelle ressource avec des données valides ;
- refus d'une requête lorsque l'utilisateur n'est pas authentifié ;

- rejet de données manquantes ou incorrectes ;
- recherche d'une ressource inexistante ou appartenant à un autre dossier.

Pour chaque requête, la réponse JSON et le code HTTP retourné ont été contrôlés. Un test réussi doit produire le résultat attendu sans créer de données incohérentes dans la base de données.

Ces tests fonctionnels ont permis de vérifier la communication entre les contrôleurs, les services, les repositories et la base de données avant l'intégration complète des fonctionnalités dans le Front-end.

Le développement des composants métier a permis de centraliser les règles de gestion de Protégéo dans des services dédiés. Les contrôleurs coordonnent les requêtes, les services valident et traitent les données, tandis que les repositories assurent leur récupération dans la base de données.

Les contrôles d'authentification, d'accès aux dossiers et d'appartenance des ressources renforcent la confidentialité des informations manipulées. Les tests fonctionnels réalisés avec Postman permettent enfin de vérifier le comportement de l'API dans les principaux scénarios de réussite et d'erreur.

Cette organisation fournit une base structurée et évolutive pour poursuivre le développement des fonctionnalités métier de l'application.

Les principaux scénarios testés avec Postman, notamment l'authentification, la création d'un mouvement bancaire, le rejet de données invalides et le refus d'une requête non authentifiée, sont présentés dans les figures 50 à 53 des annexes techniques.

4. Documenter le déploiement d'une application dynamique web et web mobile

La procédure présentée dans ce chapitre décrit les différentes étapes réellement réalisées pour mettre en production Protégéo : préparation de l'hébergement, déploiement des dépôts Git, configuration de l'API Symfony, création de la base de données, compilation du Front-end React et sécurisation des échanges avec HTTPS.

La version actuellement déployée reste évolutive. Certaines fonctionnalités du tableau de bord sont encore en cours de développement, mais l'architecture de production est opérationnelle et permet de publier progressivement les nouvelles versions de l'application.

4.1. Architecture de déploiement

L'architecture de Protégéo repose sur plusieurs éléments distincts qui communiquent entre eux.

Le Front-end est développé avec React et Vite. Il contient l'interface affichée dans le navigateur et communique avec le Back-end grâce à des requêtes HTTP.

Le Back-end est développé avec Symfony. Il expose une API REST chargée de l'authentification, des règles métier, du contrôle des autorisations et des échanges avec la base de données.

La base de données MySQL stocke les utilisateurs, les dossiers, les personnes protégées, les contacts, les comptes bancaires et les différentes informations nécessaires au fonctionnement de l'application.

En production, l'organisation prévue est la suivante :

- le domaine principal *protegeo.fr* héberge la partie publique et l'application React ;
- un sous-domaine dédié, par exemple *api.protegeo.fr*, héberge l'API Symfony ;
- la base de données MySQL est installée sur l'hébergement O2Switch ;
- les échanges entre le Front-end et l'API sont réalisés en HTTPS.

Cette séparation permet de déployer et de mettre à jour le Front-end et le Back-end indépendamment. Elle facilite également la maintenance, car chaque partie possède sa propre configuration et son propre dépôt de code.

Cette séparation est également appliquée au stockage du code source. Le Front-end et le Back-end sont enregistrés dans deux dépôts GitHub distincts :

- *protegeo-showcase*, qui contient l'application React ;
- *protegeo-api*, qui contient l'API Symfony.

Chaque dépôt possède ainsi son propre historique, ses dépendances et sa procédure de déploiement. Une modification de l'interface peut être publiée sans modifier directement le dépôt de l'API, et inversement.

Les fichiers contenant les valeurs propres à l'environnement local ou de production ne sont pas enregistrés dans les dépôts. Le Front-end conserve seulement un fichier *.env.example* servant de modèle. Dans le dépôt Symfony, les fichiers de configuration généraux comme *.env*, *.env.dev* et *.env.test* sont présents, mais le fichier sensible *.env.local* est absent.

4.2. Préparation de l'hébergement

L'hébergement O2Switch a été retenu afin de disposer d'un environnement compatible avec PHP, Symfony, Node.js et MySQL. L'interface cPanel permet de gérer les domaines, les sous-domaines, les fichiers, les bases de données, les certificats SSL et l'accès au terminal du serveur.

Le domaine *protegeo.fr* a été rattaché à l'hébergement afin d'accueillir l'interface React. Un sous-domaine *api.protegeo.fr* a ensuite été configuré pour l'API Symfony.

L'espace de production a été organisé dans un dossier principal *protegeo*, lui-même séparé en deux répertoires :

- protegeo-front, destiné au Front-end React ;
- protegeo-api, contenant le projet Symfony.

Cette séparation reprend l'architecture utilisée pendant le développement et permet de mettre à jour les deux parties indépendamment.

Le sous-domaine `api.protegeo.fr` a été configuré pour pointer directement vers le dossier public du projet Symfony. Cette configuration est essentielle, car seul ce répertoire doit être accessible depuis le navigateur. Les fichiers internes de l'application, comme les contrôleurs, les services, les variables d'environnement ou les dépendances, restent ainsi en dehors de la racine publique.

Une base de données MySQL dédiée à Protégéo ainsi qu'un utilisateur possédant les privilèges nécessaires ont également été créés depuis cPanel.

La configuration du sous-domaine de l'API et la création de la base de données de production sont présentées dans les figures 54 et 55 des annexes techniques.

4.3. Déploiement et configuration du Back-end Symfony

Le projet Symfony a été déployé depuis son dépôt GitHub à l'aide du terminal proposé par O2Switch. Le dépôt a été cloné dans le répertoire *protegeo-api*, ce qui permet de conserver son historique Git et de faciliter les futures mises à jour.

Une vérification de l'environnement du serveur a ensuite été réalisée. La version de PHP, la version de Composer ainsi que la présence des extensions nécessaires à Symfony et à MySQL ont été contrôlées depuis le terminal.

Le déploiement du projet Symfony depuis son dépôt GitHub est présenté dans la figure 56 des annexes techniques.

Le serveur utilise PHP 8.4 et dispose des extensions nécessaires au fonctionnement de Symfony et à la connexion avec MySQL. La présence de `intl`, `mbstring`, `openssl` et `pdo_mysql` a été vérifiée dans le terminal avant l'installation des dépendances.

Ces extensions assurent notamment la gestion correcte des caractères, certaines fonctions de sécurité ainsi que la communication entre PHP et la base de données MySQL.

Les dépendances PHP définies dans le fichier `composer.lock` ont ensuite été installées avec Composer. Cette opération a généré le dossier `vendor`, optimisé l'autoload de l'application et exécuté les scripts Symfony nécessaires, notamment le nettoyage du cache et l'installation des ressources publiques.

Les paramètres propres à la production ont été renseignés dans un fichier `.env.local`. Ce fichier contient notamment la connexion à la base de données et les paramètres sensibles de l'application. Il n'est pas versionné dans Git et ses permissions ont été limitées afin de réduire les possibilités de lecture par un autre utilisateur du serveur.

Une fois la connexion à MySQL configurée, les migrations Doctrine ont été exécutées. Elles ont créé dans la base de production l'ensemble des tables, colonnes, relations et contraintes définies par les entités Symfony.

Enfin, l'API a été testée depuis le sous-domaine *api.protegeo.fr*. L'appel d'une route protégée sans jeton JWT renvoie une erreur HTTP 401, ce qui confirme à la fois l'accessibilité de l'API et le fonctionnement de sa protection par authentification.

L'exécution des migrations Doctrine et la vérification de l'API déployée sont présentées dans les figures 57 et 58 des annexes techniques.

Les tests réalisés avec Postman confirment que l'API Symfony est opérationnelle sur le serveur de production. L'authentification retourne correctement un jeton JWT et les routes protégées deviennent accessibles lorsque ce jeton est transmis. La base de données de production étant encore vide au moment du test, la récupération des dossiers retourne un tableau vide, ce qui correspond au résultat attendu.

L'authentification de production ainsi que l'accès à une route protégée avec un jeton JWT valide sont présentés dans les figures 59 et 60 des annexes techniques.

4.4. Déploiement du Front-end React

Le Front-end de Protégéo est développé avec React et Vite. Contrairement à l'environnement local, dans lequel l'application est exécutée avec un serveur de développement, la version destinée à la production doit être compilée avant d'être publiée.

Le projet React a été installé dans le répertoire *protegeo-front* de l'hébergement O2Switch. Cette séparation permet de conserver les sources du Front-end indépendamment du projet Symfony placé dans le répertoire *protegeo-api*.

Avant la compilation, l'adresse utilisée pour communiquer avec le Back-end doit correspondre à l'environnement de production. L'application React doit donc envoyer ses requêtes vers l'API accessible depuis :

`https://api.protegeo.fr/api`

Cette adresse est récupérée depuis une variable d'environnement utilisée par Vite. Les services React peuvent ainsi employer une adresse différente en développement local et en production, sans modifier directement leur code.

La compilation du projet a ensuite été exécutée depuis le terminal O2Switch avec la commande :

`npm run build`

Cette commande demande à Vite d'analyser les composants React, les feuilles de style et les ressources utilisées par l'application. Elle génère une version optimisée pour la production dans le dossier *dist*.

La compilation du Front-end avec Vite est présentée dans la figure 61 des annexes techniques.

Le dossier généré contient notamment :

- le fichier *index.html*, qui constitue le point d'entrée de l'application ;
- les fichiers JavaScript compilés ;
- les feuilles de style ;
- les images et autres ressources statiques.

Ces fichiers ont ensuite été placés dans le répertoire servi par le domaine *protegeo.fr*. Le navigateur ne reçoit donc pas directement les fichiers sources React : il charge la version compilée et optimisée produite par Vite.

Une configuration du serveur permet également de rediriger les différentes adresses de l'application vers le fichier *index.html*. Cette étape est nécessaire pour que les routes gérées par React Router restent accessibles lorsque l'utilisateur actualise une page ou saisit directement son adresse.

Après le transfert des fichiers compilés, le domaine *protegeo.fr* a été ouvert dans le navigateur afin de vérifier le résultat. La page d'accueil de Protégéo s'affiche correctement depuis l'hébergement O2Switch, ce qui confirme le fonctionnement du déploiement du Front-end.

Le résultat du déploiement du Front-end sur le domaine protegeo.fr est présenté dans la figure 62 des annexes techniques.

Le Front-end et le Back-end étant placés sur deux adresses différentes, leur communication repose sur des requêtes HTTPS entre *protegeo.fr* et *api.protegeo.fr*.

Lorsqu'un utilisateur remplit le formulaire de connexion, React envoie ses identifiants vers la route d'authentification de l'API. Après validation, le jeton JWT retourné est utilisé pour accéder aux routes protégées.

Les tests réalisés précédemment avec Postman ont permis de vérifier que l'API de production accepte correctement une authentification et retourne les données lorsqu'un jeton valide est transmis. La dernière étape consiste donc à vérifier le même parcours directement depuis l'interface React.

Le déploiement du Front-end repose ainsi sur trois étapes principales : la configuration de l'adresse de l'API, la compilation du projet avec Vite et la publication du contenu du dossier *dist*. L'application React est désormais accessible depuis le domaine principal et peut être mise à jour indépendamment de l'API Symfony.

4.5. Sécurisation de l'application

Protégéo manipule des informations personnelles, administratives et financières. La sécurisation de l'environnement de production constitue donc une étape essentielle du déploiement.

Plusieurs protections ont été mises en place au niveau de l'hébergement, de l'API Symfony et de l'application React.

Les échanges entre le navigateur, le Front-end et l'API sont réalisés avec le protocole HTTPS. Un certificat SSL Let's Encrypt a été installé pour le sous-domaine `api.protegeo.fr`. Le domaine principal `protegeo.fr` est également accessible avec une connexion sécurisée.

Le protocole HTTPS chiffre les données échangées entre le navigateur et le serveur. Il protège notamment les identifiants transmis lors de la connexion, le jeton JWT retourné par l'API et les informations envoyées depuis le tableau de bord.

L'API Symfony est hébergée dans un répertoire distinct et le sous-domaine `api.protegeo.fr` pointe uniquement vers le dossier public. Les autres répertoires du projet, comme `src`, `config`, `vendor` ou `migrations`, ne sont donc pas directement accessibles depuis Internet.

Les paramètres sensibles de production sont enregistrés dans le fichier `.env.local`. Ce fichier contient notamment la connexion à la base de données ainsi que les informations propres à l'environnement de production.

Le fichier `.env.local` n'est pas enregistré dans le dépôt GitHub. Ses permissions ont également été limitées avec la commande `chmod 600`, afin que seul le propriétaire du fichier puisse le lire et le modifier.

L'authentification des utilisateurs repose sur un jeton JWT. Lorsqu'un utilisateur se connecte avec des identifiants valides, l'API lui retourne un jeton signé. Ce jeton doit ensuite être transmis dans l'en-tête `Authorization` pour accéder aux routes protégées.

Une requête envoyée sans jeton vers une route protégée est refusée avec un code HTTP 401. L'API contrôle également que l'utilisateur possède l'autorisation d'accéder au dossier et aux ressources demandées.

Les mots de passe des utilisateurs ne sont pas enregistrés en clair dans la base de données. Symfony utilise un mécanisme de hachage avant leur enregistrement. Lors de la connexion, le mot de passe transmis est comparé à la valeur hachée stockée.

Ces différentes protections permettent de limiter l'exposition des fichiers internes, de sécuriser les échanges avec le serveur et de restreindre l'accès aux données selon l'utilisateur authentifié.

Comme présenté précédemment, le fichier `.env.local` est ignoré par Git et ses permissions sont limitées au propriétaire du compte d'hébergement.

Le test d'une route protégée sans jeton, présenté précédemment, retourne un code HTTP 401. Cette réponse confirme que les données de l'API ne sont pas accessibles sans authentification.

Le domaine principal *protegeo.fr* est lui aussi servi en HTTPS, comme le montre l'accès au Front-end présenté dans la section précédente.

Authentification et protection des routes

L'authentification JWT permet à l'API de rester sans session serveur traditionnelle. Après la connexion, le Front-end conserve le jeton retourné et l'ajoute aux requêtes nécessitant une authentification.

L'API vérifie la validité du jeton avant d'exécuter le contrôleur demandé. Si le jeton est absent, incorrect ou expiré, l'accès à la route est refusé.

Cette protection est complétée par des vérifications métier. Un utilisateur authentifié ne peut accéder qu'aux dossiers auxquels il est associé. Les comptes bancaires, contacts et autres ressources sont ensuite recherchés avec l'identifiant du dossier afin d'éviter l'accès à des données appartenant à un autre utilisateur.

Protection des paramètres de production

Les valeurs propres à l'environnement de production ne sont pas écrites directement dans le code source.

La connexion MySQL, les paramètres JWT et les secrets de l'application sont définis dans des fichiers de configuration locaux non versionnés.

Cette séparation permet d'utiliser des paramètres différents entre l'environnement local et le serveur de production. Elle évite également de publier accidentellement les mots de passe ou les clés privées dans le dépôt GitHub.

Les captures ajoutées au dossier ne montrent pas le contenu de ces fichiers. Les mots de passe, les jetons et les informations complètes de connexion ont été masqués afin de ne pas exposer de données sensibles.

La sécurisation de Protégéo repose ainsi sur plusieurs niveaux complémentaires : HTTPS pour les échanges, limitation de la racine publique de Symfony, protection des fichiers de configuration, hachage des mots de passe, authentification JWT et contrôle de l'accès aux dossiers.

Ces mesures réduisent les risques d'exposition des informations sensibles et permettent de sécuriser l'utilisation de l'application dans son environnement de production.

Le refus d'une requête non authentifiée est illustré par la figure 58, tandis que les figures 59 et 60 montrent le fonctionnement de l'authentification JWT sur l'API de production.

4.6. Mise à jour et maintenance

Partie 2 – Développer la partie Back-end d'une application web et web mobile sécurisée

Une application déployée doit pouvoir être mise à jour afin d'ajouter de nouvelles fonctionnalités, de corriger des erreurs et d'améliorer sa sécurité. Les projets React et Symfony étant enregistrés dans deux dépôts GitHub distincts, ils peuvent être mis à jour indépendamment.

Avant chaque publication, les modifications sont d'abord testées dans l'environnement local, puis enregistrées dans Git avec un commit. Elles sont ensuite envoyées vers le dépôt distant GitHub.

Sur le serveur O2Switch, le terminal permet de se placer dans le répertoire du projet concerné et de récupérer les dernières modifications avec la commande :

git pull

Pour le Back-end Symfony, une mise à jour peut également nécessiter :

- l'installation ou la mise à jour des dépendances avec Composer ;
- l'exécution des migrations Doctrine lorsqu'une entité ou la structure de la base de données a été modifiée ;
- le nettoyage du cache de production ;
- la vérification du fonctionnement des routes de l'API.

Pour le Front-end React, les nouvelles sources doivent être recompilées avec la commande :

npm run build

Le contenu généré dans le dossier *dist* remplace ensuite la version précédemment publiée.

Après chaque mise à jour, plusieurs vérifications sont effectuées : affichage du site, fonctionnement de la navigation, connexion à l'API, authentification de l'utilisateur et accès aux principales fonctionnalités.

Cette procédure permet de publier progressivement les évolutions de Protégéo sans devoir recommencer l'ensemble du déploiement.

```
git pull
composer install
php bin/console doctrine:migrations:migrate
php bin/console cache:clear --env=prod
```

```
git pull  
npm install  
npm run build
```

4.7. État actuel du déploiement

Au moment de la rédaction du dossier, l'infrastructure de production de Protégéo est opérationnelle.

Le domaine principal *protegeo.fr* héberge le Front-end React, tandis que le sous-domaine *api.protegeo.fr* permet d'accéder à l'API Symfony. La base de données MySQL de production a été créée et les migrations Doctrine ont été exécutées. Les échanges sont protégés par HTTPS et l'authentification JWT a été testée avec succès.

La version actuellement publiée reste cependant une version en cours de développement. La partie publique est accessible, mais certaines fonctionnalités du tableau de bord doivent encore être terminées ou corrigées.

Les travaux restants prioritaires concernent notamment :

- la correction du parcours d'inscription dans l'interface React ;
- l'achèvement des différentes pages de la Dashboard ;
- les tests des fonctionnalités déjà développées après leur mise en production.

D'autres évolutions sont également envisagées, comme l'affichage temporaire de certaines informations bancaires sensibles après déchiffrement et l'ajout de fichiers ou d'images. Ces fonctionnalités constituent des améliorations complémentaires et ne bloquent pas l'utilisation des principales fonctions de l'application.

L'architecture mise en place permet désormais de publier les prochaines corrections et fonctionnalités progressivement, grâce aux dépôts GitHub et aux procédures de mise à jour du Front-end et du Back-end.

Annexes techniques

A. Environnement et outils

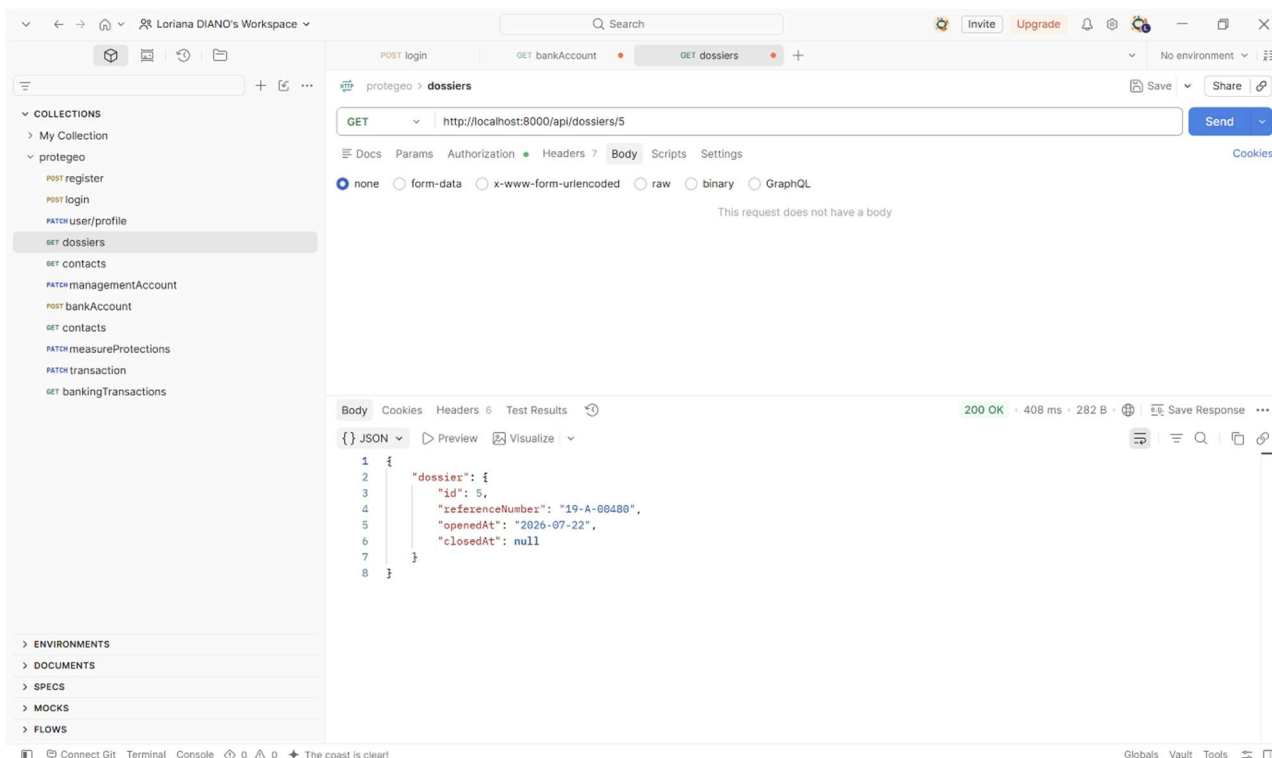


Figure 1 – Postman

B. Maquettes de l'application

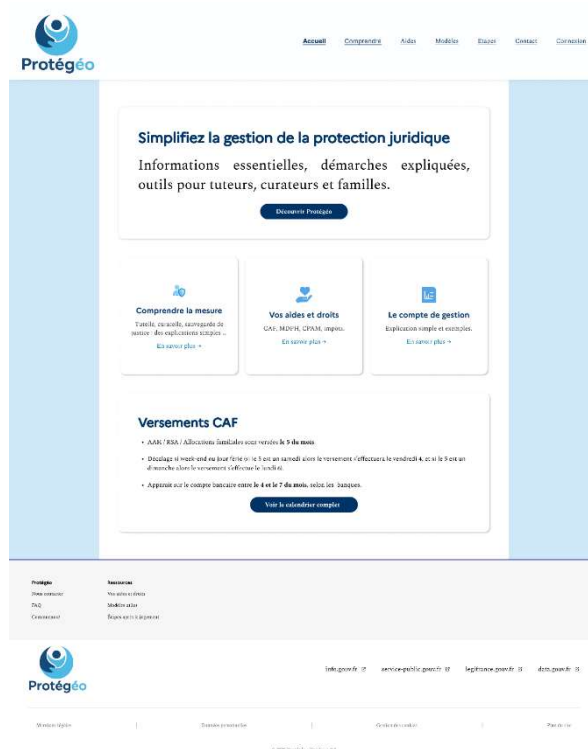


Figure 2 - Maquette de la page d'accueil

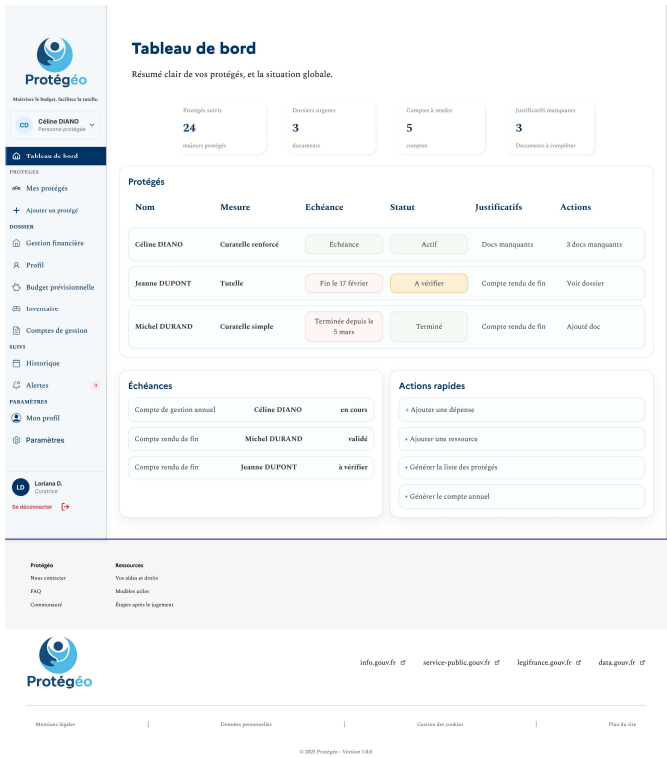


Figure 3 - Tableau de bord

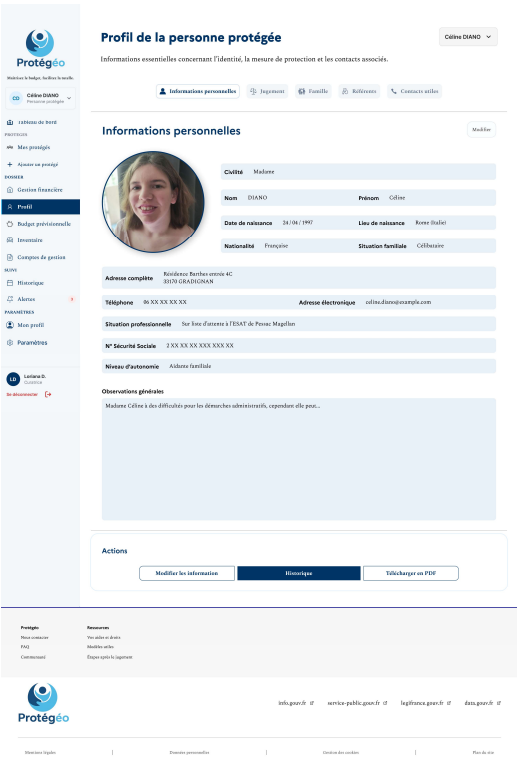


Figure 4 - Profil de la personne protégée

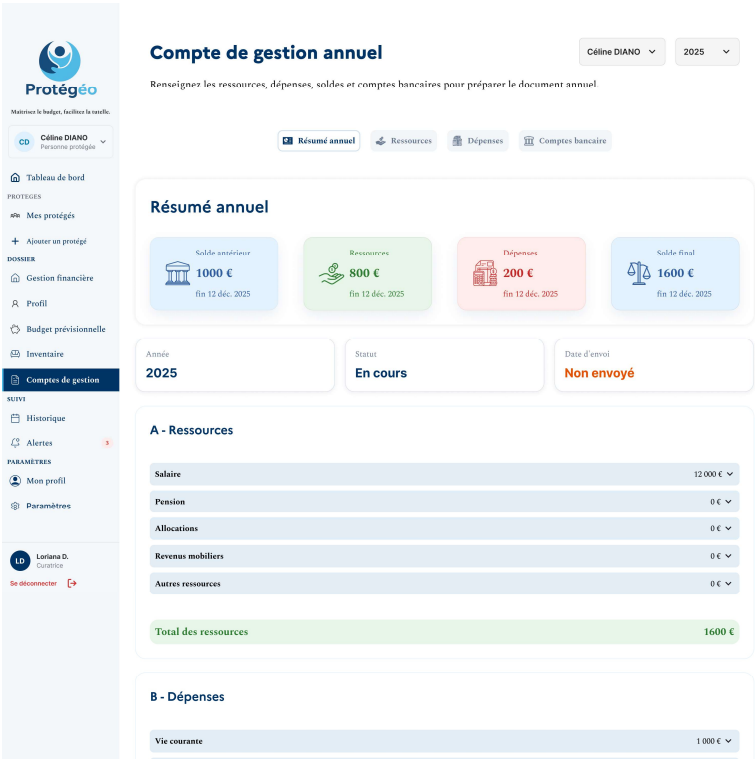


Figure 5 - Compte de gestion

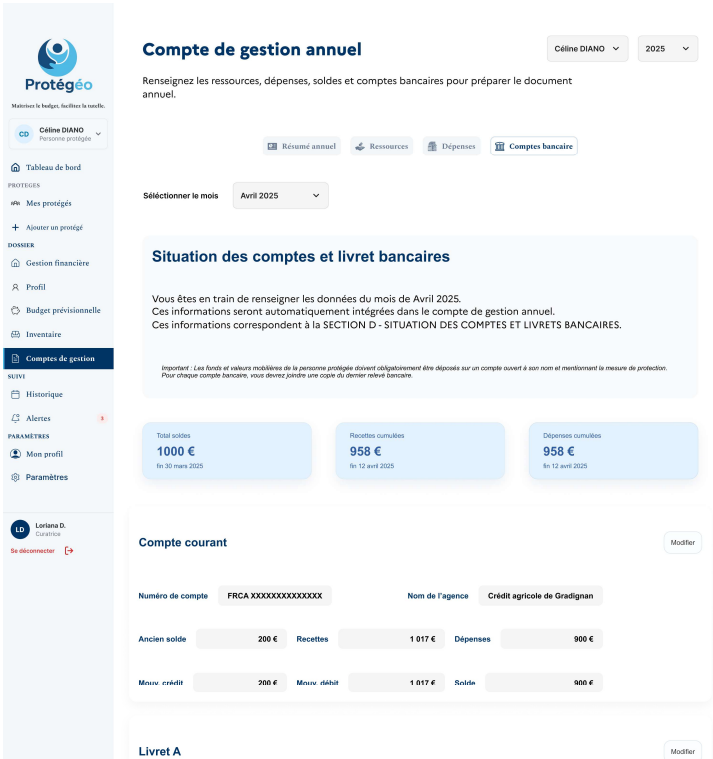


Figure 6 - Gestion des comptes bancaires

C. Développement des interfaces Front-end



Figure 7 - Page d'accueil développée

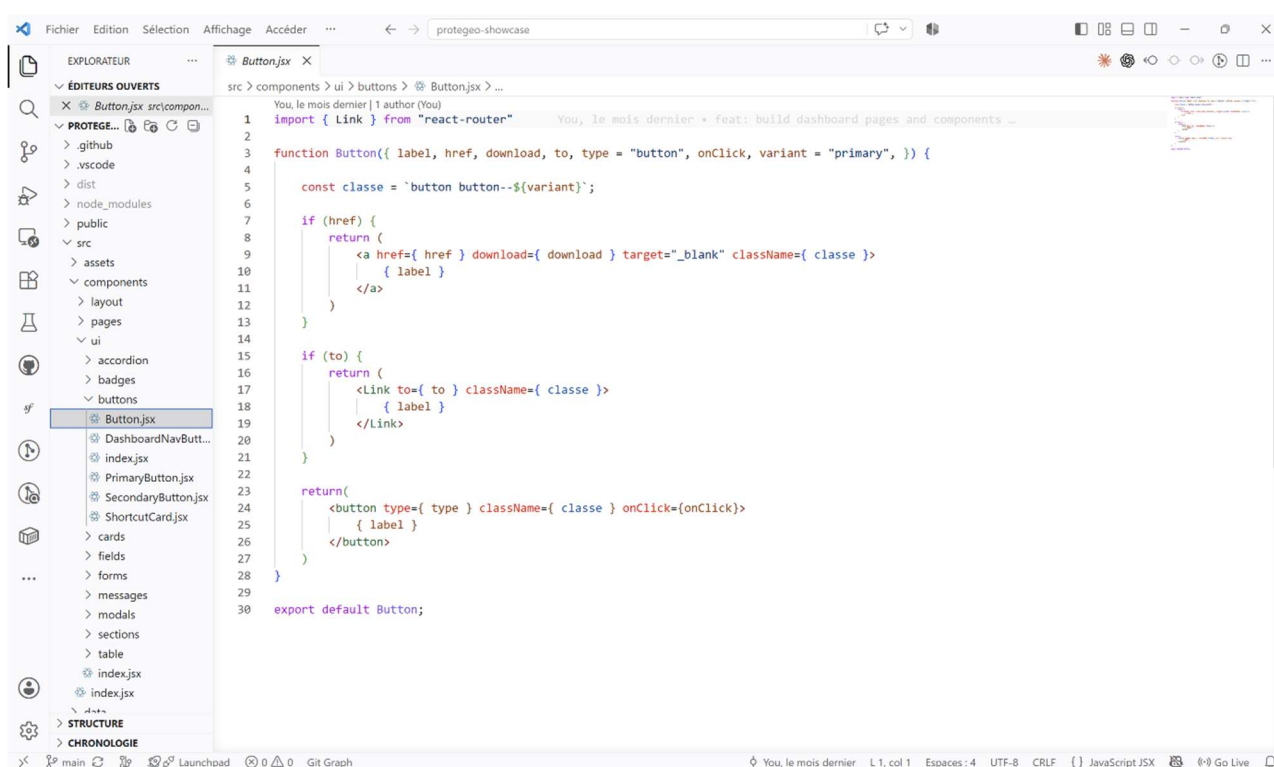


Figure 8 - Bouton

Annexes techniques

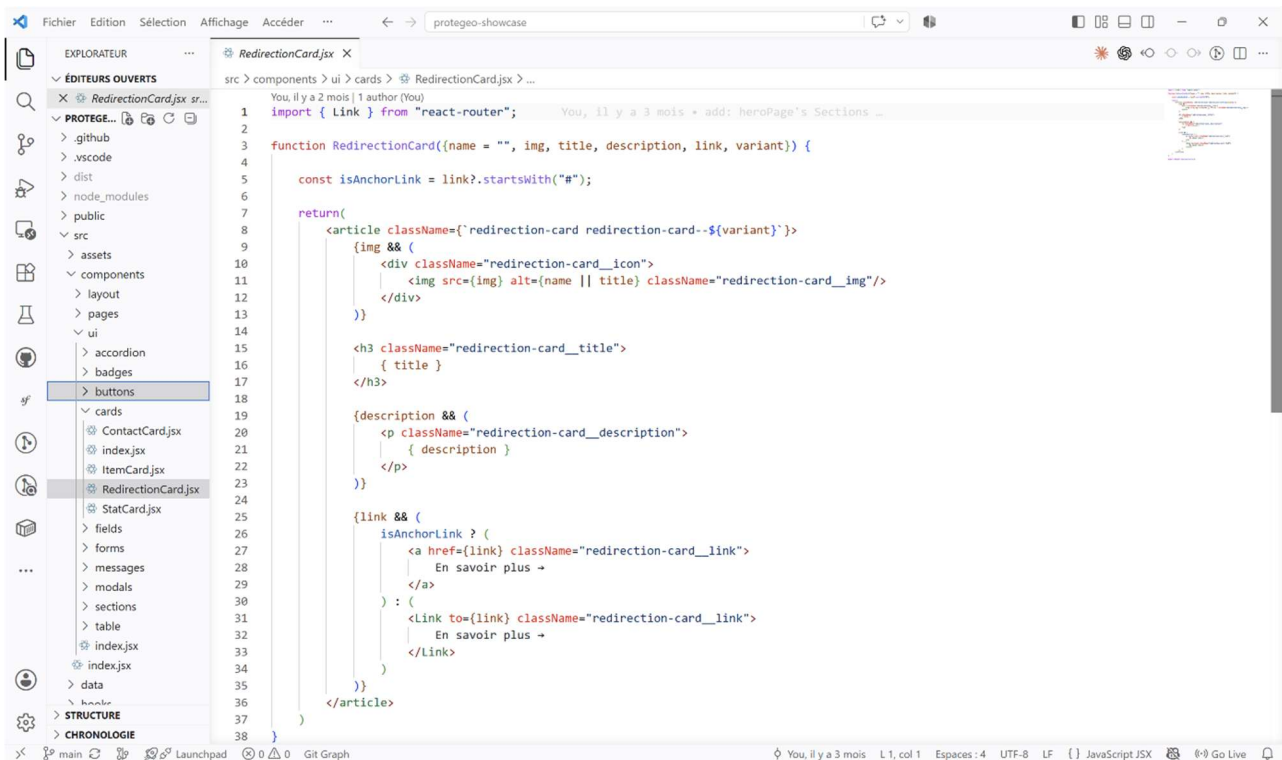


Figure 9 - Cartes de redirection

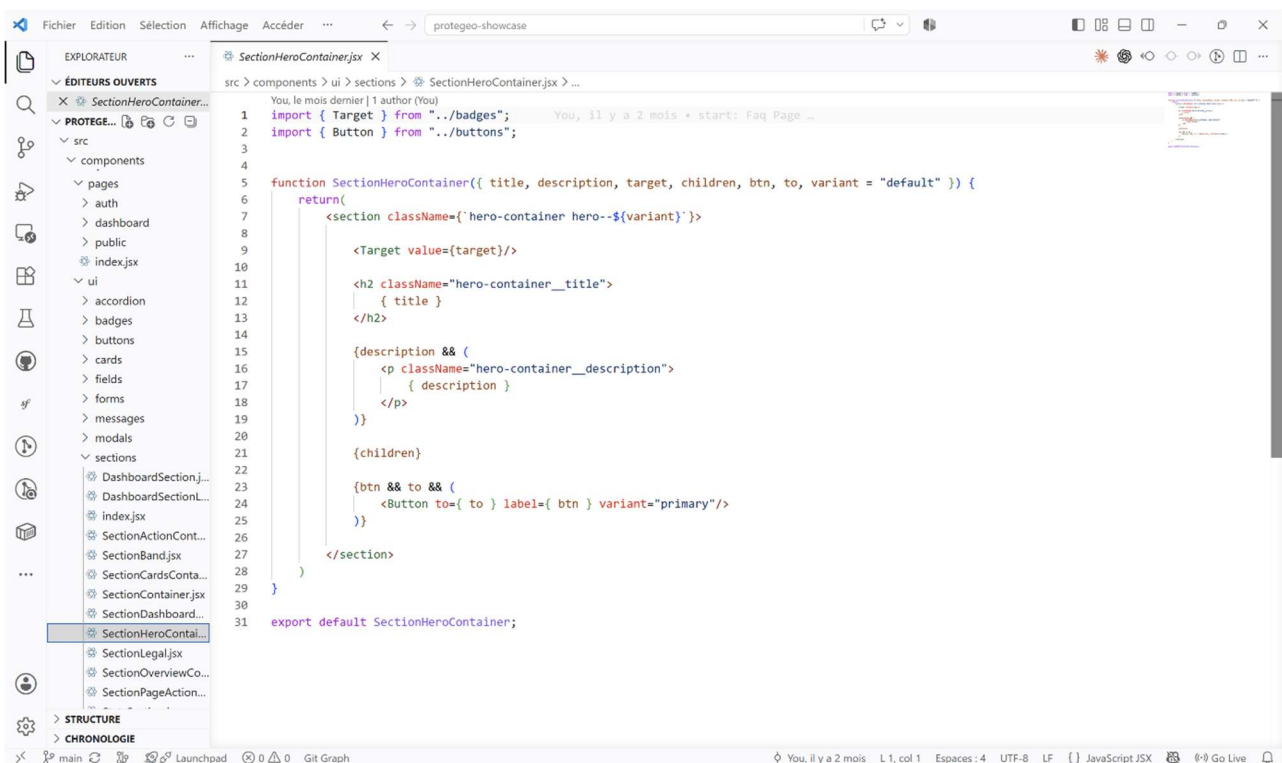


Figure 10 - Composant réutilisable pour les sections hero

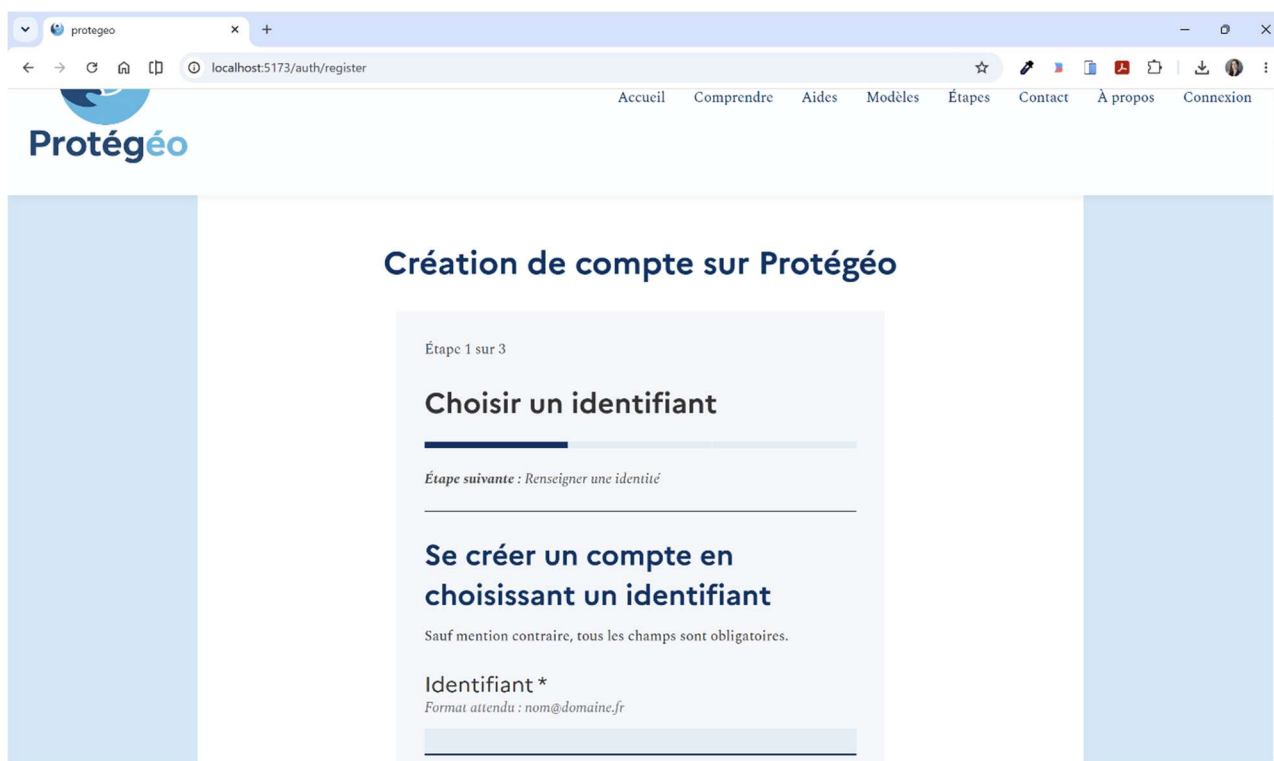


Figure 11 - Page d'inscription

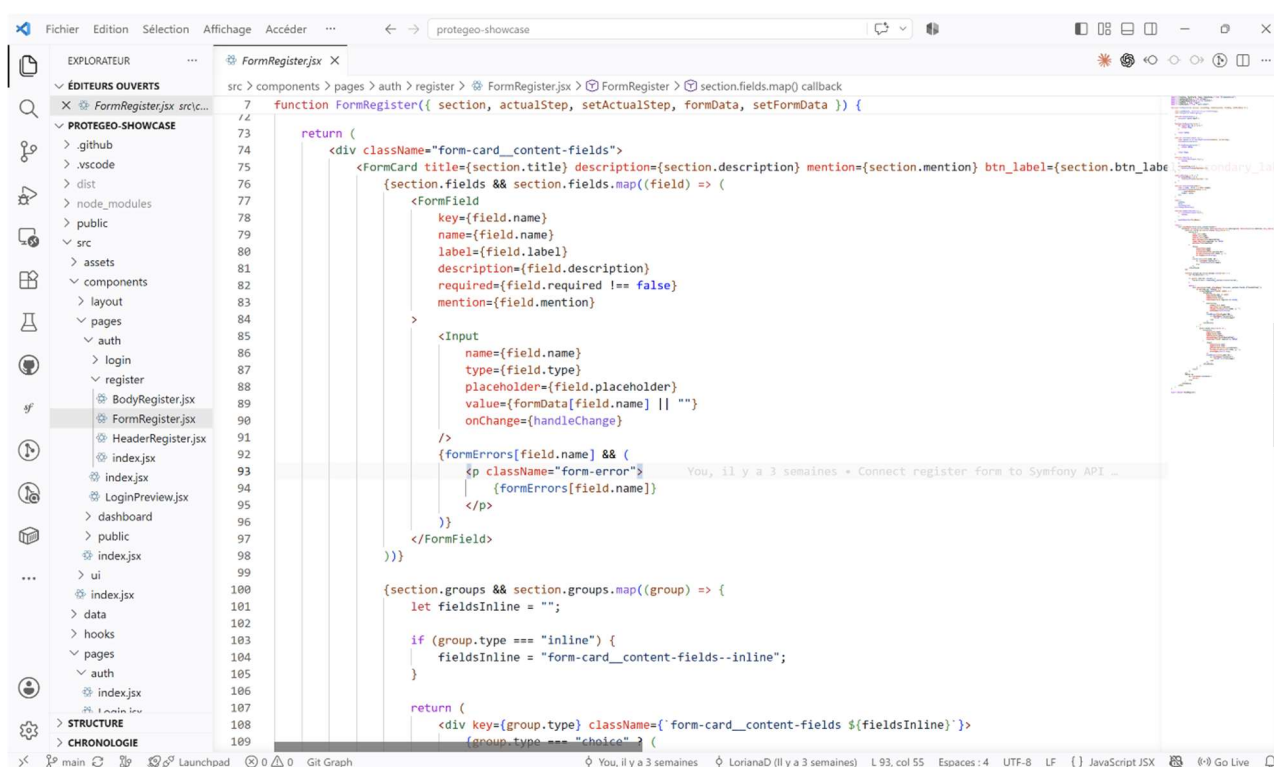


Figure 12 - Code du formulaire d'inscription

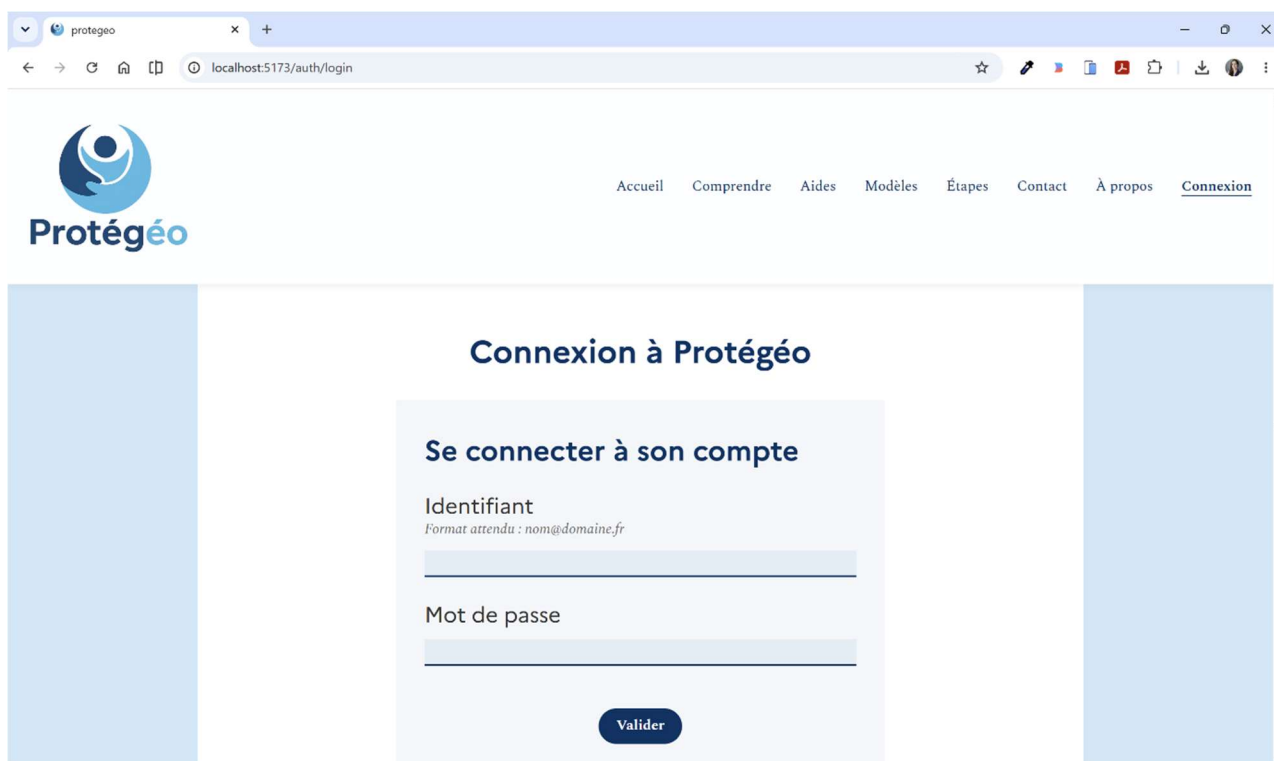


Figure 13 - Page de connexion

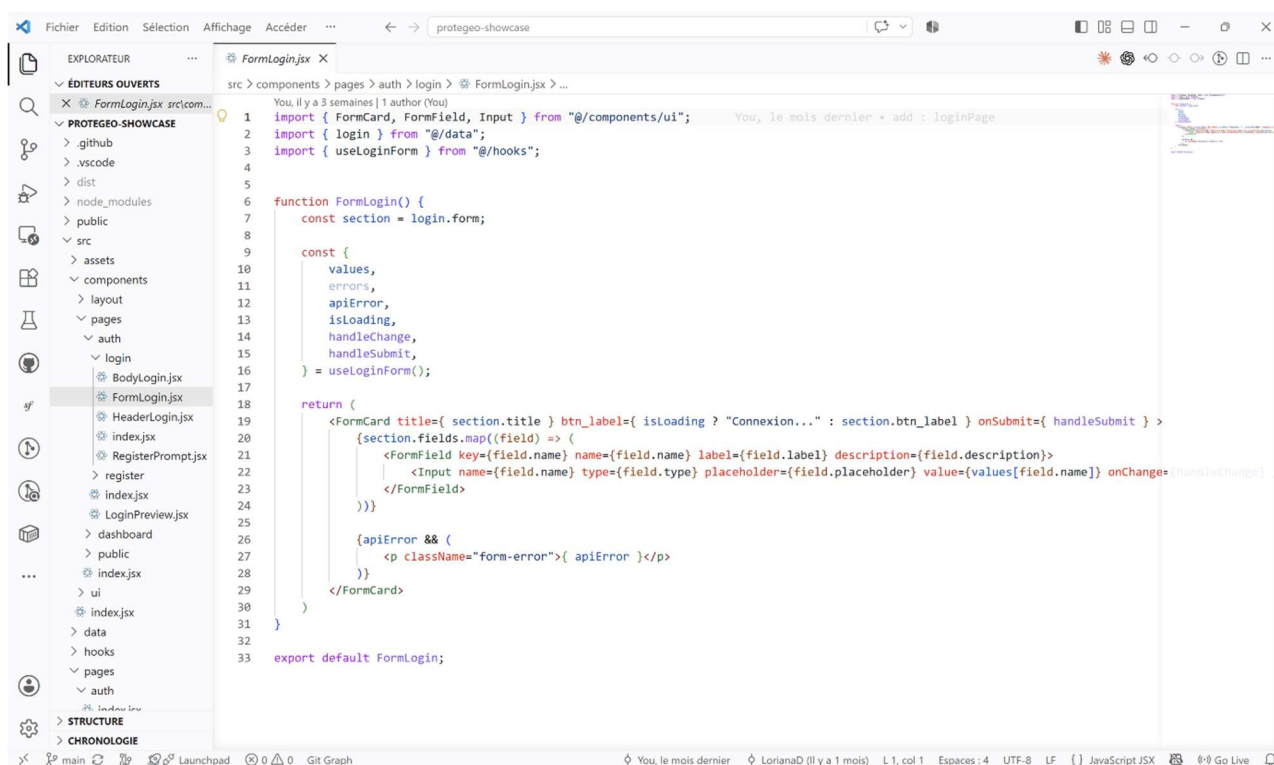


Figure 14 - Composant du formulaire de connexion

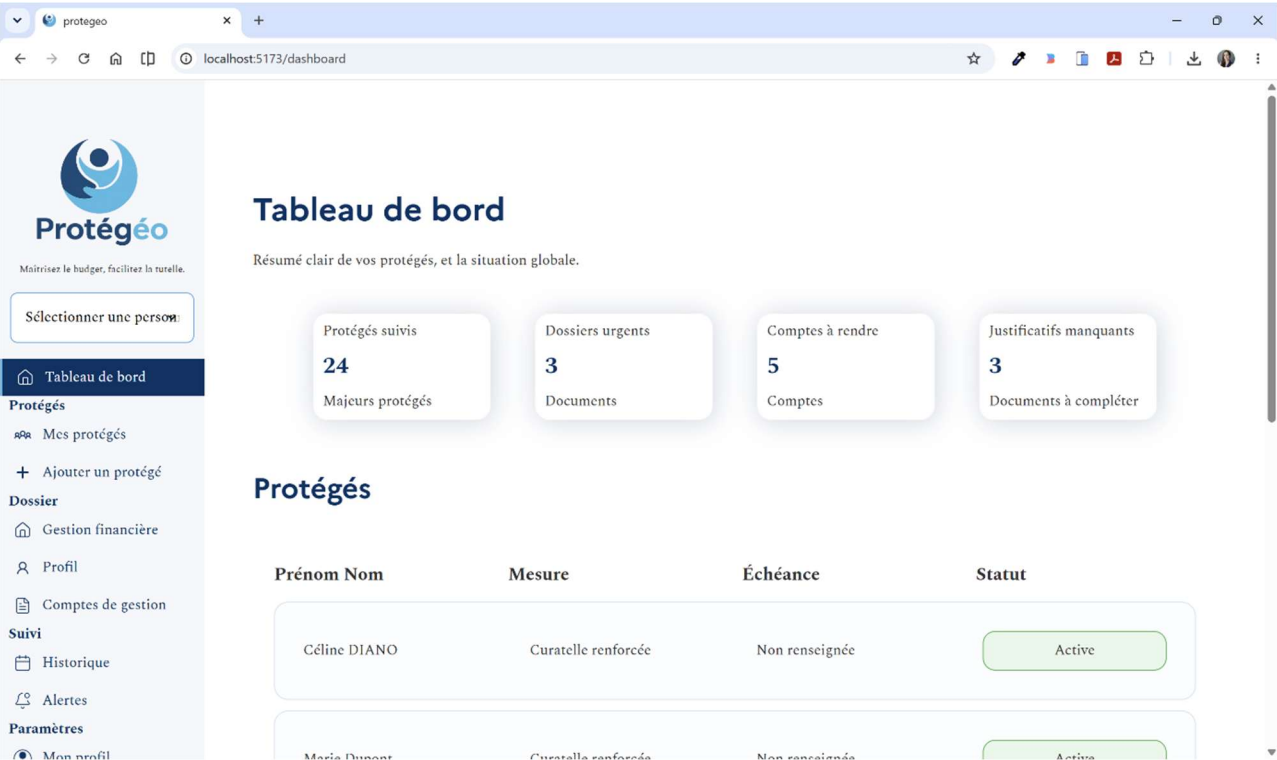


Figure 15 - Tableau de bord

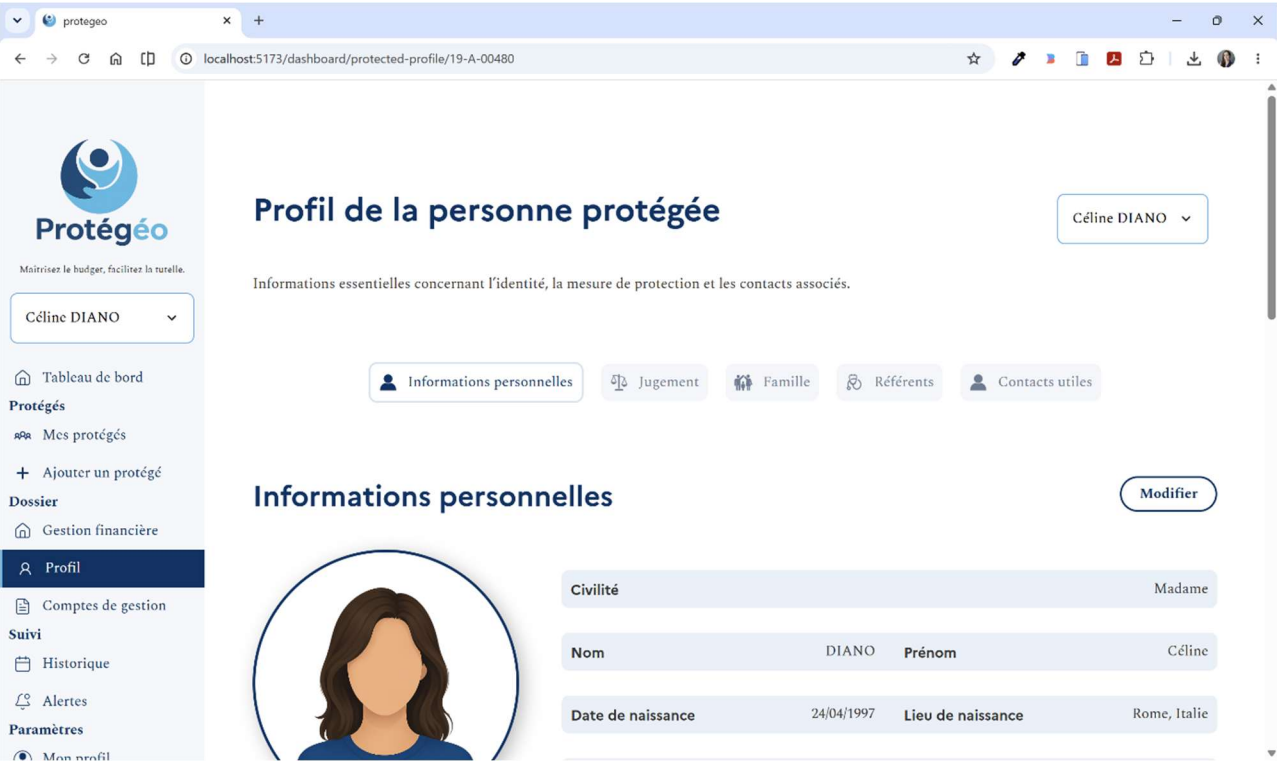


Figure 16 - Page du profil de la personne protégée

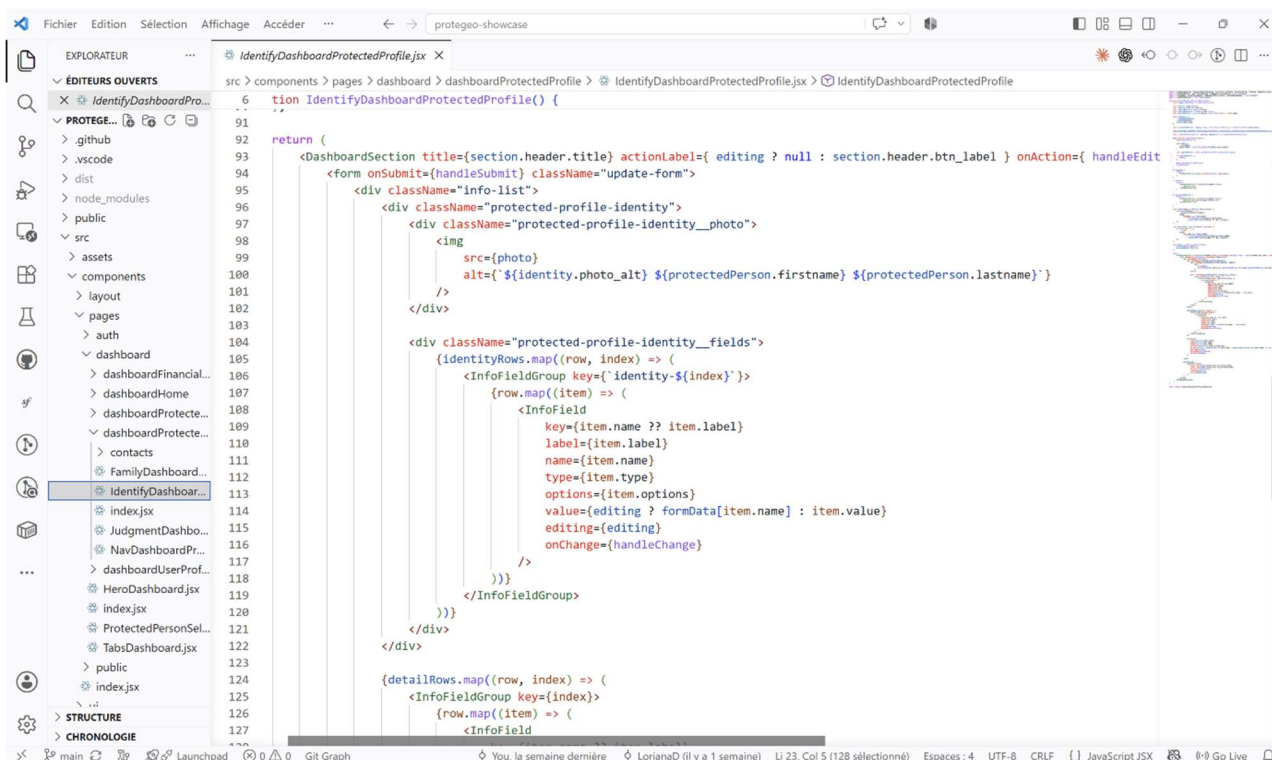


Figure 17 - Composant principal du profil de la personne protégée

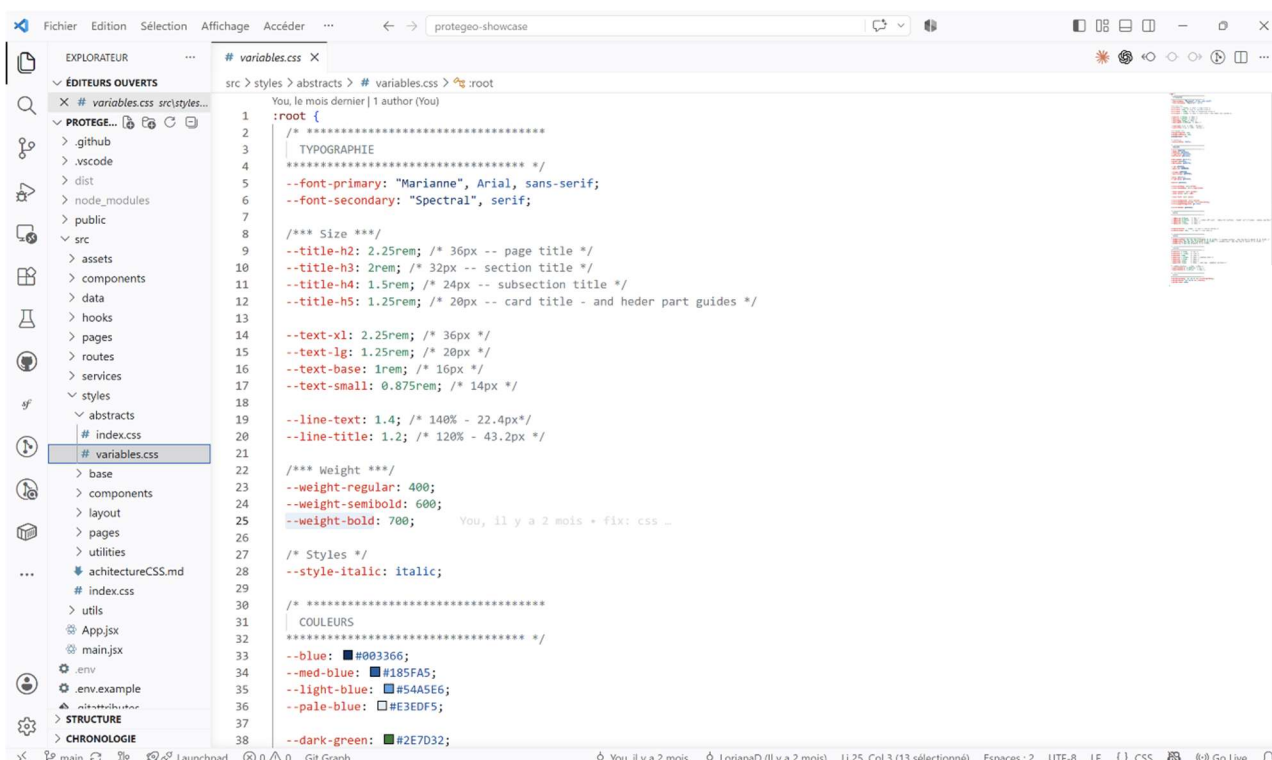


Figure 18 - Variables globales de la charte graphique

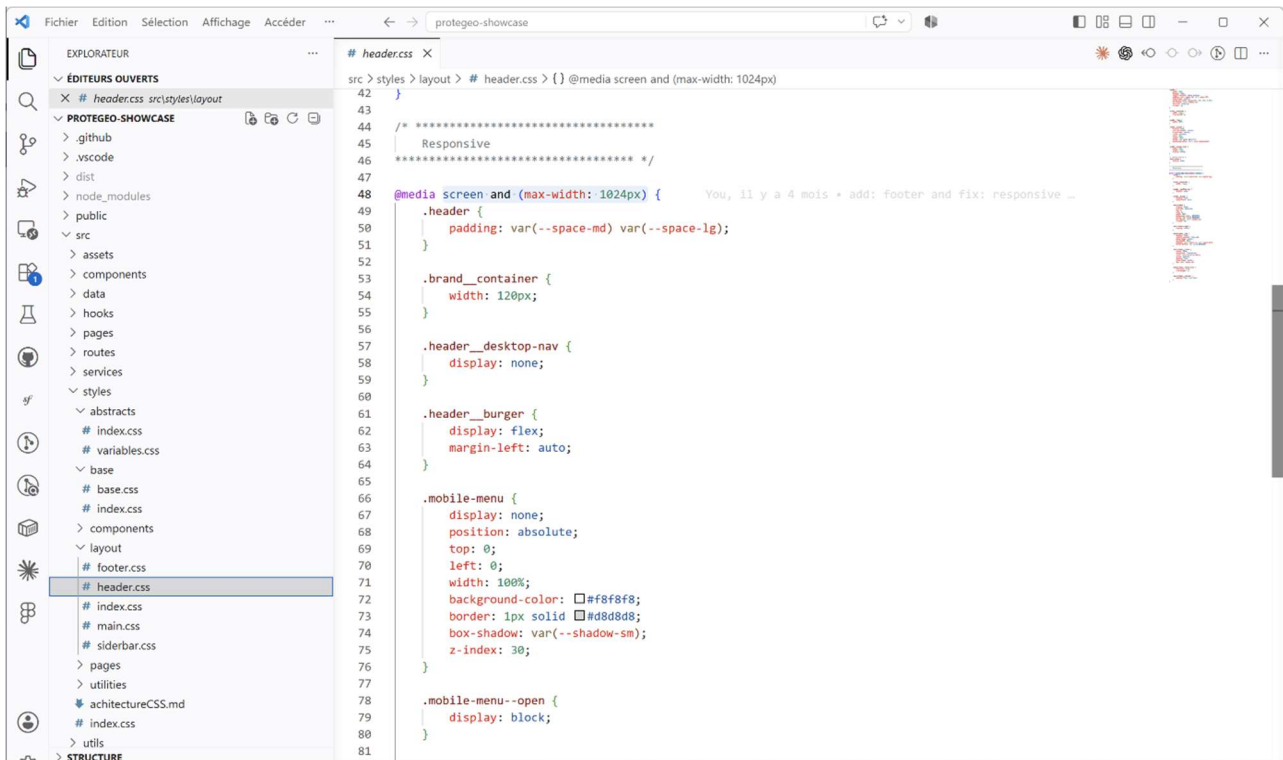


Figure 19 - Adaptation responsive de la navigation

D. Dynamisation et gestion des données Front-end

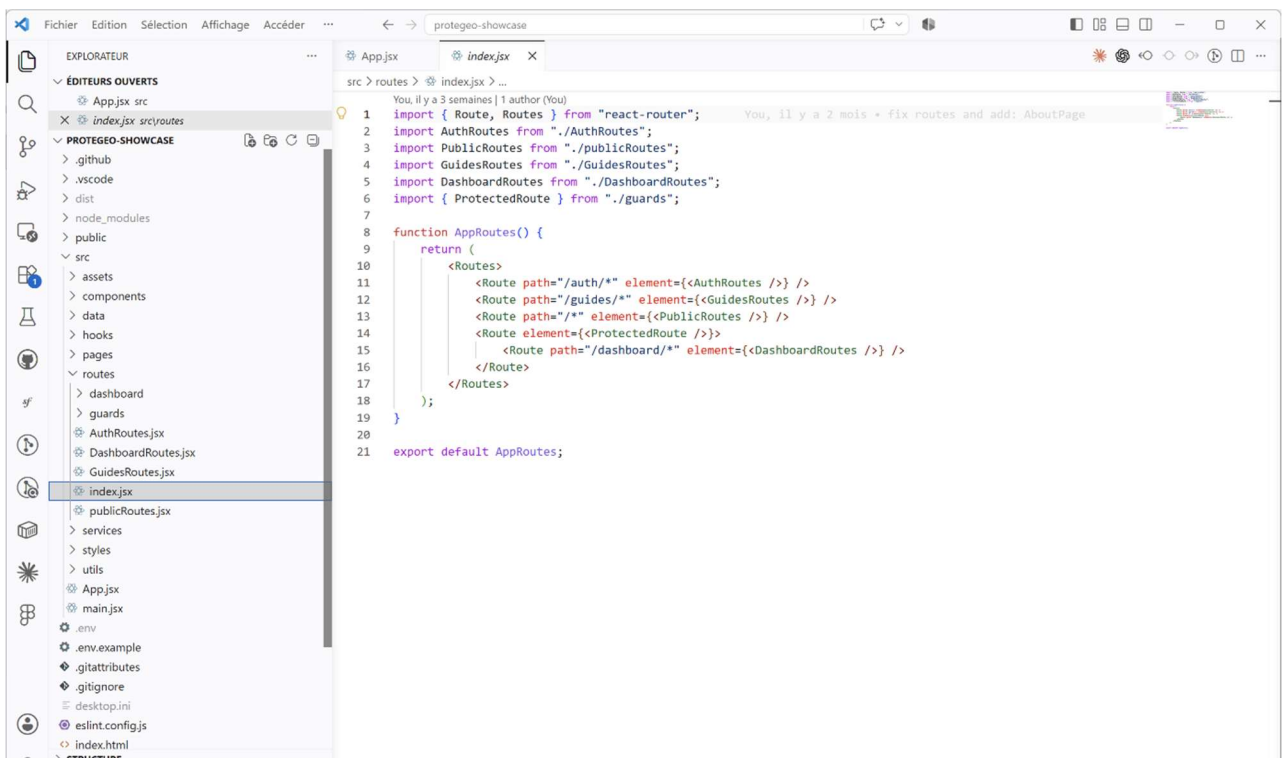


Figure 20 - Organisation des routes publiques et protégées
Les routes sont regroupées par espace et les pages du tableau de bord sont placées derrière une route protégée.

Annexes techniques

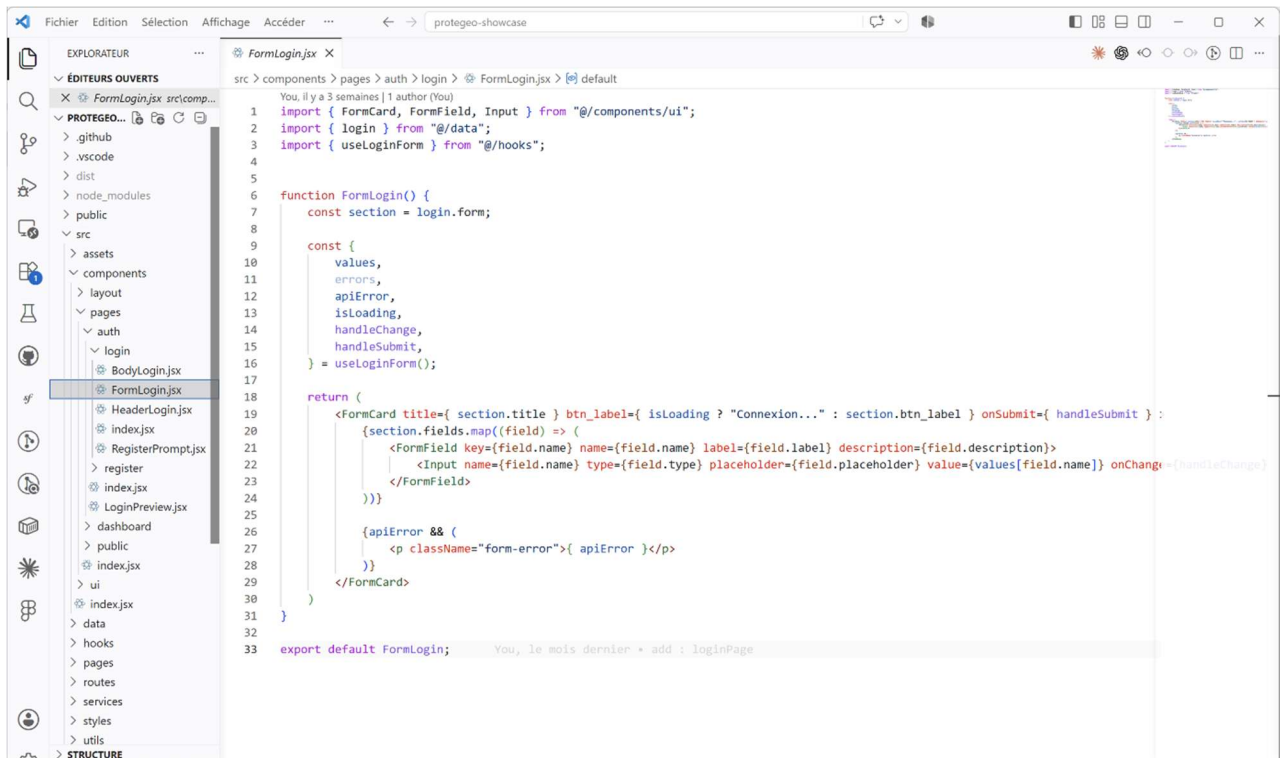


Figure 21 - Composant du formulaire de connexion
Le composant affiche les champs et utilise un hook personnalisé pour gérer le fonctionnement du formulaire.

```
import { login } from "@services";
import { validateLoginForm } from "@utils";
import { useState } from "react";
import { useNavigate } from "react-router";

function useLoginForm() {
  const navigate = useNavigate();

  const [values, setValues] = useState({
    email: "",
    password: "",
  });

  const [errors, setErrors] = useState({});
  const [apiError, setApiError] = useState("");
  const [isLoading, setIsLoading] = useState(false);

  function handleChange(event) {
    const { name, value } = event.target;

    setValues((currentValues) => ({
      ...currentValues,
      [name]: value,
    }));
  }

  async function handleSubmit(event) {
    event.preventDefault();

    console.log('Le formulaire est soumis !');
    console.log(values);

    setApiError("");

    const validation = validateLoginForm(values);
    setErrors(validation.errors);

    if (!validation.isValid) {
      return;
    }

    try {
      setIsLoading(true);
      await login(values.email, values.password);
      navigate("/dashboard");
    } catch (error) {
      setApiError(error.message);
    } finally {
      setIsLoading(false);
    }
  }

  return {
    values,
    errors,
    apiError,
    isLoading,
    handleChange,
    handleSubmit,
  };
}

export { useLoginForm };
```

Figure 22 - Gestion du formulaire de connexion avec un hook personnalisé
Le hook centralise les valeurs, la validation, les erreurs, le chargement et la redirection.

Annexes techniques

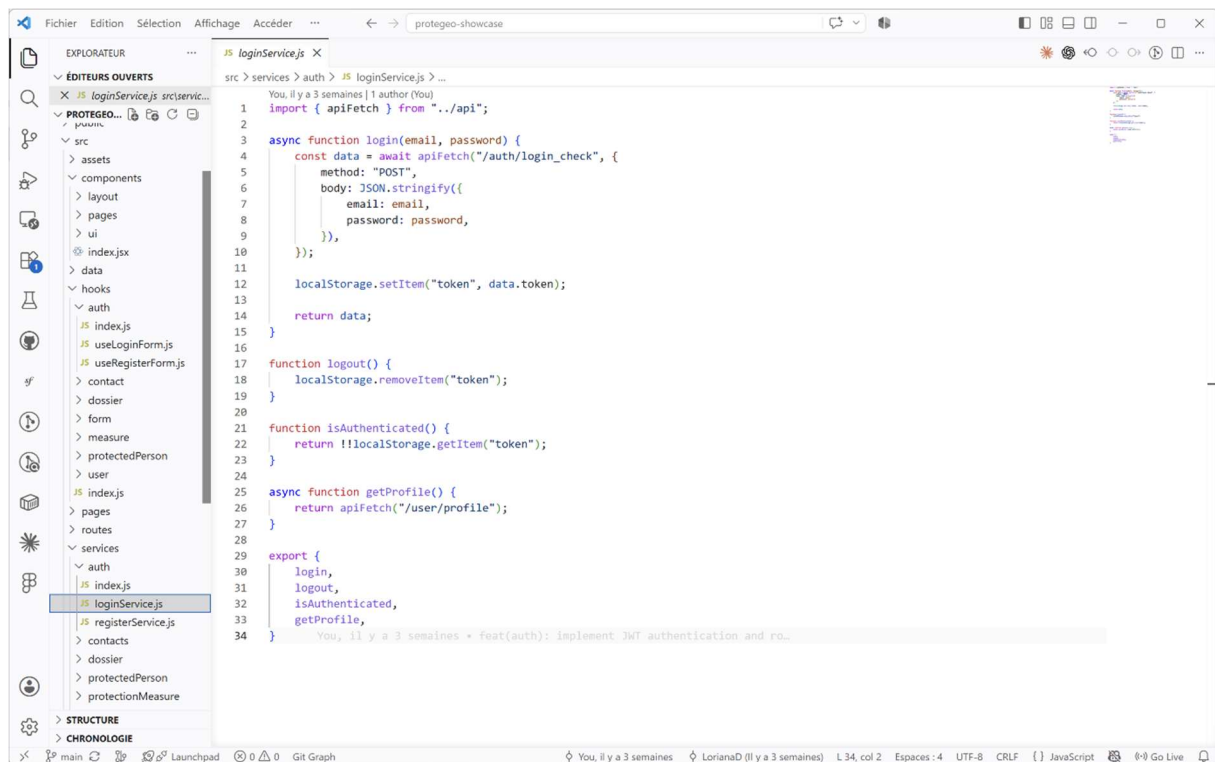


Figure 23 - Service d'authentification
Le service envoie les identifiants à l'API et conserve le jeton JWT reçu après la connexion.

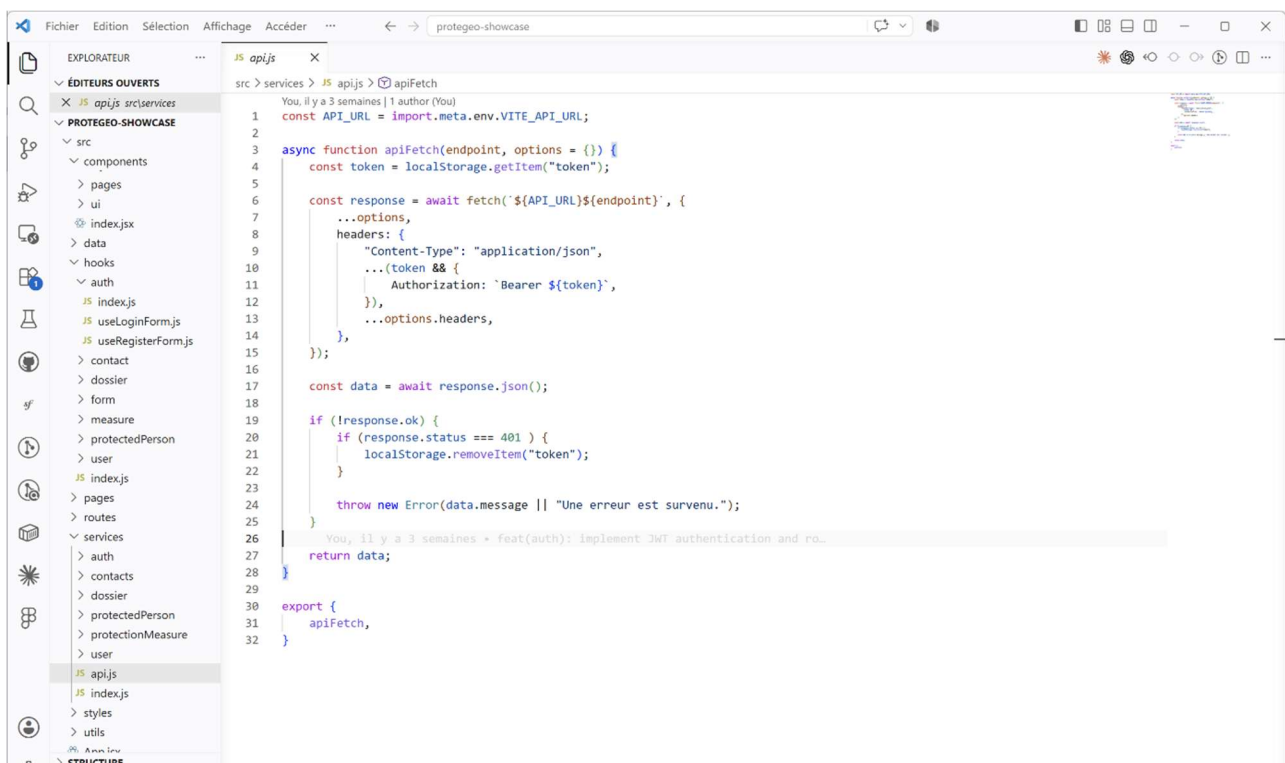


Figure 24 - Fonction générique de communication avec l'API
La fonction ajoute l'adresse de l'API, le format JSON et le jeton d'authentification aux requêtes.

Annexes techniques

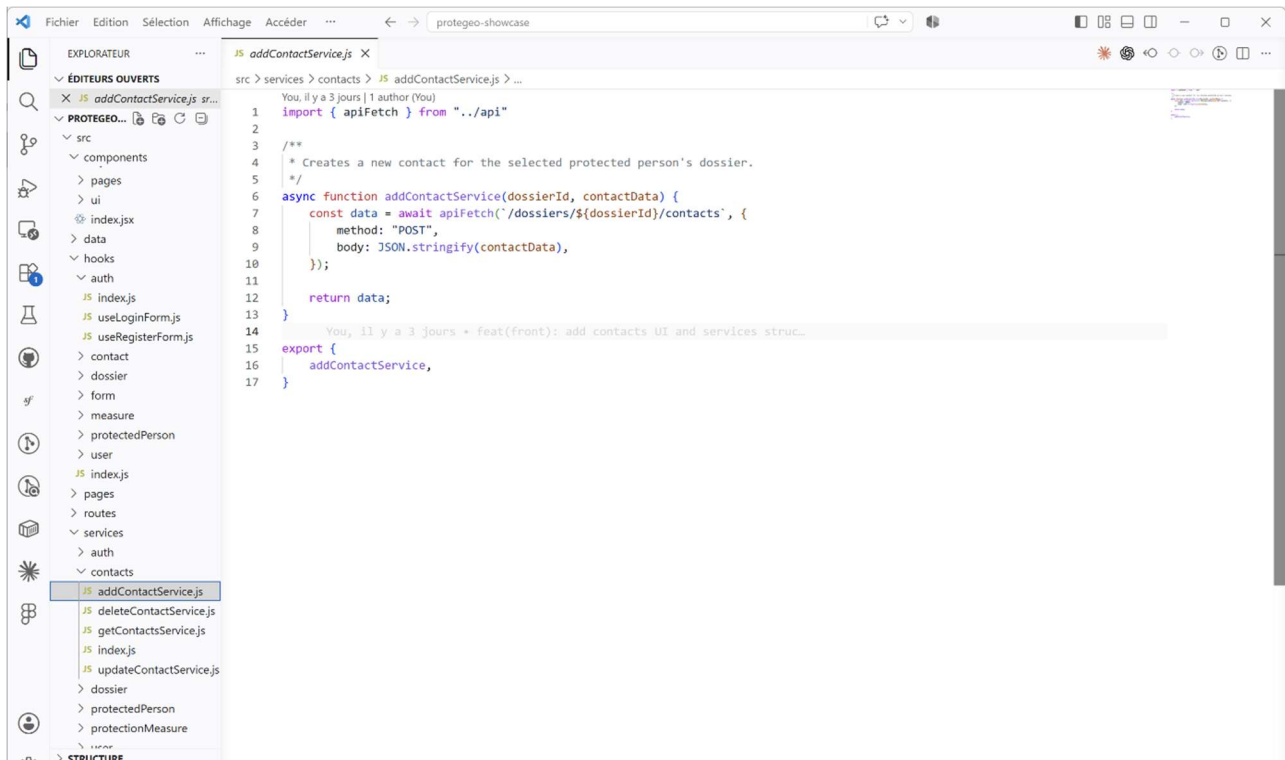


Figure 25 - Service d'ajout d'un contact

```
import { getProtectedPersonService } from "@services";
import { useEffect, useState } from "react"

function useProtectedPerson(dossierId) {
  const [protectedPerson, setProtectedPerson] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  async function fetchProtectedPerson() {
    if (!dossierId) {
      setProtectedPerson(null);
      setLoading(false);
      return;
    }

    setLoading(true);
    setError("");

    try {
      const personData = await getProtectedPersonService(dossierId);
      setProtectedPerson(personData);
    } catch (error) {
      setProtectedPerson(null);
      setError(
        error.message || "Impossible de récupérer les informations."
      );
    } finally {
      setLoading(false);
    }
  }

  useEffect(() => {
    fetchProtectedPerson();
  }, [dossierId]);

  return {
    protectedPerson,
    loading,
    error,
    refreshProtectedPerson: fetchProtectedPerson,
  };
}

export {
  useProtectedPerson,
}
```

Figure 26 - Hook de récupération de la personne protégée
Le hook `useProtectedPerson` charge les informations de la personne protégée lorsque le dossier sélectionné change.

Annexes techniques

```
import { createFormData } from "@utils";
import { useEffect, useState } from "react";

function useEditableForm(fieldList, sourceData) {
  const [editing, setEditing] = useState(false);

  const [formData, setFormData] = useState(
    createFormData(fieldList)
  );

  useEffect(() => {
    if (sourceData) {
      setFormData(
        createFormData(fieldList, sourceData)
      );
    }
  }, [sourceData]);

  function handleChange(event) {
    const { name, value } = event.target;

    setFormData((currentData) => ({
      ...currentData,
      [name]: value,
    }));
  }

  function handleEdit() {
    setEditing(true);
  }

  function handleCancel() {
    setEditing(false);

    setFormData(
      createFormData(fieldList, sourceData)
    );
  }

  function closeEditing() {
    setEditing(false);
  }

  return {
    editing,
    formData,
    setFormData,
    handleChange,
    handleEdit,
    handleCancel,
    closeEditing,
  };
}

export {
  useEditableForm,
}
```

Figure 27 - Hook générique de formulaire modifiable
Il centralise le mode d'édition, les valeurs du formulaire, la modification des champs et l'annulation.

```
import { ContactFormModal, DashboardSection, DashboardSectionLoading, InfoField } from "@components/ui";
import { useContacts, useFamilyContactForm } from "@hooks";
import { formatContactCards, formatContactTableRows } from "@utils";
import { useOutletContext } from "react-router";
import { ContactCardList, ContactObservationNote, ContactTable } from "../contacts";

function FamilyDashboardProtectedProfile() {
  const { page, dossierId } = useOutletContext();

  const section = page.family;

  const { contacts, isLoading, error, refreshContacts } = useContacts(dossierId, "family");

  const partnerCards = formatContactCards(
    contacts,
    section.partners
  );

  const parentCards = formatContactCards(
    contacts,
    section.parents
  );

  const siblingTable = formatContactTableRows(
    contacts,
    "sibling"
  );

  const trustedPeopleRows = formatContactTableRows(
    contacts,
    "trusted_person"
  );

  const subrogatedRows = formatContactTableRows(
    contacts,
    "subrogated"
  );

  const { isContactModalOpen, contactFormData, contactFormFields, isAdding, addError, handleOpenContactModal,
    handleCloseContactModal, handleContactChange, handleContactSubmit } = useFamilyContactForm(dossierId, section.form, refreshContacts);

  if (isLoading) {
    return (
      <DashboardSectionLoading section={section} page={page}/>
    );
  }

  if (error) {
    return (
      <DashboardSection title={section.header.title}>
        <p>{error}</p>
      </DashboardSection>
    );
  }

  return (
    <
      <DashboardSection title={section.header.title} actionLabel={section.header.btn_label} addLabel=
        {section.header.btn_label_add} onClick={handleOpenContactModal}>
        <p>{contacts.length === 0 ? "Aucun membre trouvé." : ""}</p>
      </div>
      <div className="family-info">
        <ContactCardList contactCards={partnerCards}/>
        <ContactCardList contactCards={parentCards}/>
        <ContactTable contactType={section.sibling} rows={siblingTable}/>
        <ContactTable contactType={section.trusted_people} rows={trustedPeopleRows}/>
        <ContactTable contactType={section.subrogated} rows={subrogatedRows}/>
        <ContactObservationNote section={section}/>
      </div>
    </DashboardSection>
    <ContactFormModal
      form={section.form}
      fields={contactFormFields}
      values={contactFormData}
      onChange={handleContactChange}
      onClose={handleCloseContactModal}
      onSubmit={handleContactSubmit}
      category="family"
      cancelLabel={page.footer_form.btn_cancel_label}
      submitLabel={page.footer_form.btn_recorded_label}
      loading={isAdding}
      error={addError}
    />
  );
}

export default FamilyDashboardProtectedProfile;
```

Figure 28 - Composition de la section des contacts familiaux
Le composant principal assemble les hooks, les données formatées, les cartes, les tableaux et la modale d'ajout.

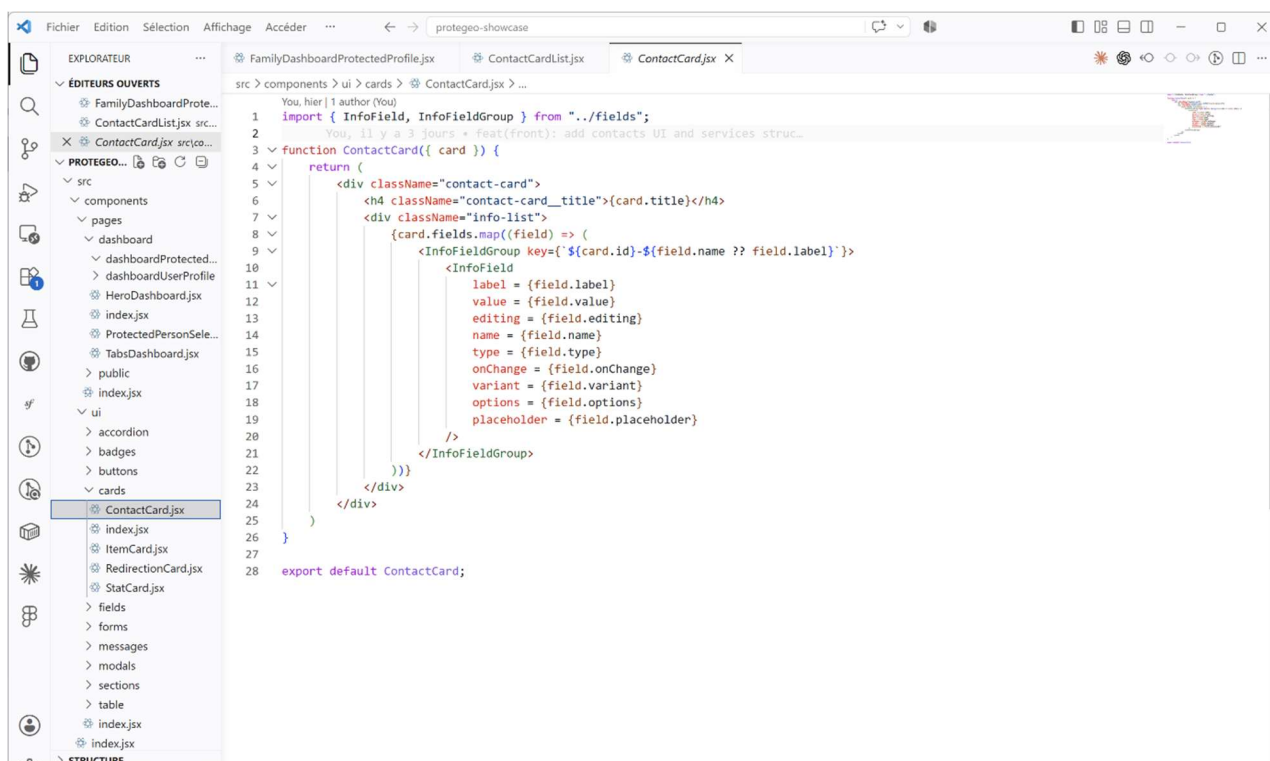


Figure 29 - Composant réutilisable de carte de contact

E. Modélisation et base de données

57

MLD

MLD
(Modèle Logique des Données)

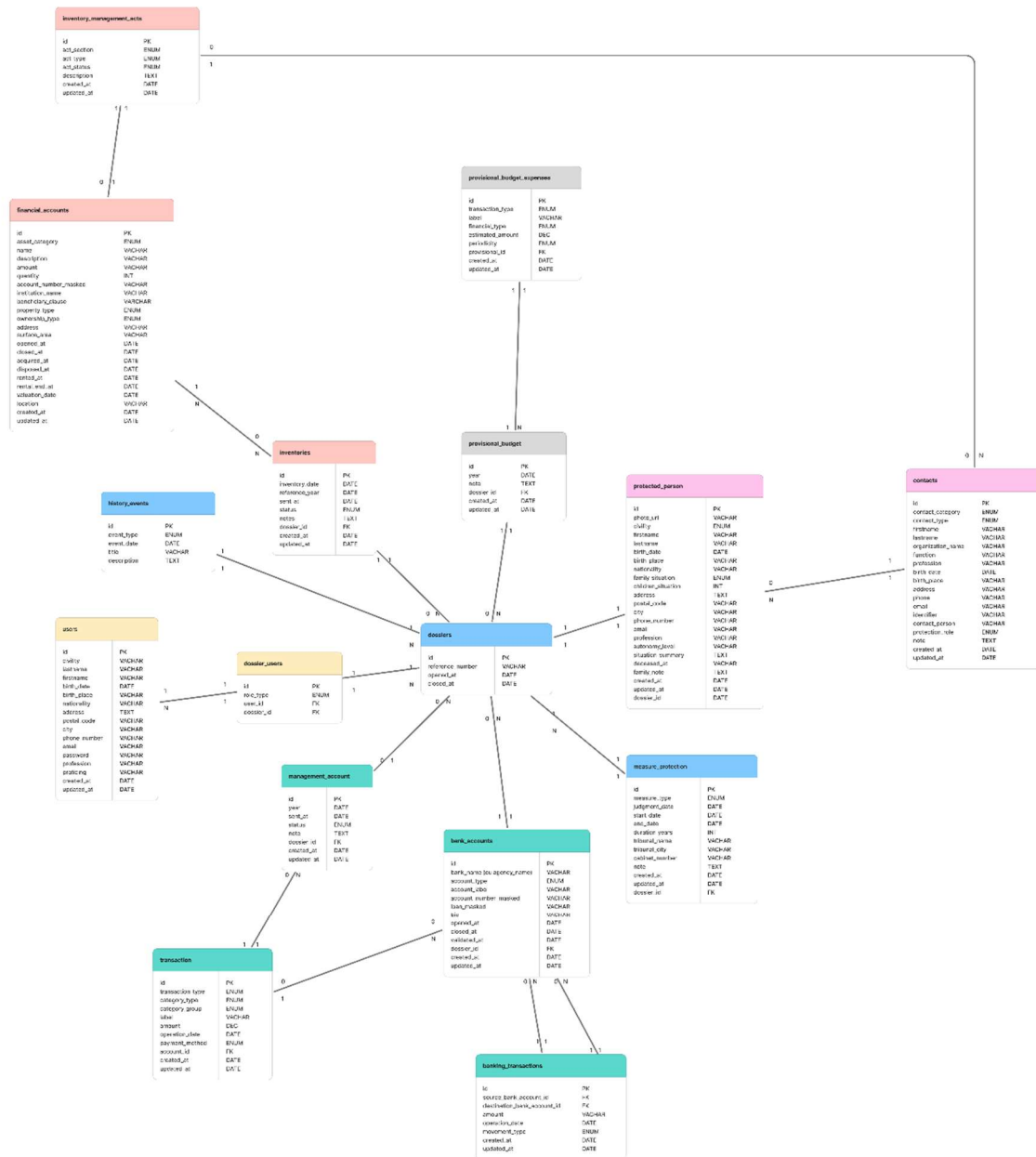


Figure 31 - Modèle Logique de Données (MLD) de Protégéo

Annexes techniques

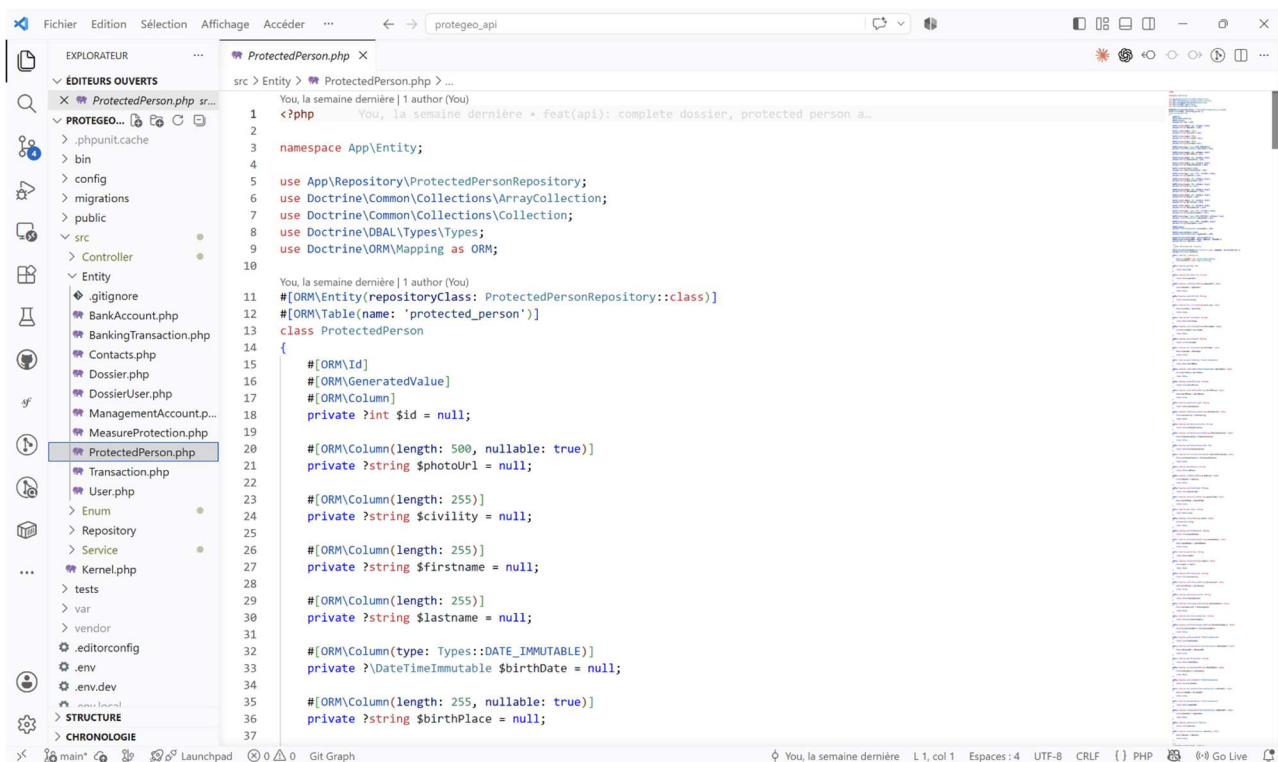


Figure 32 - Entité Symfony représentant une table de la base de données

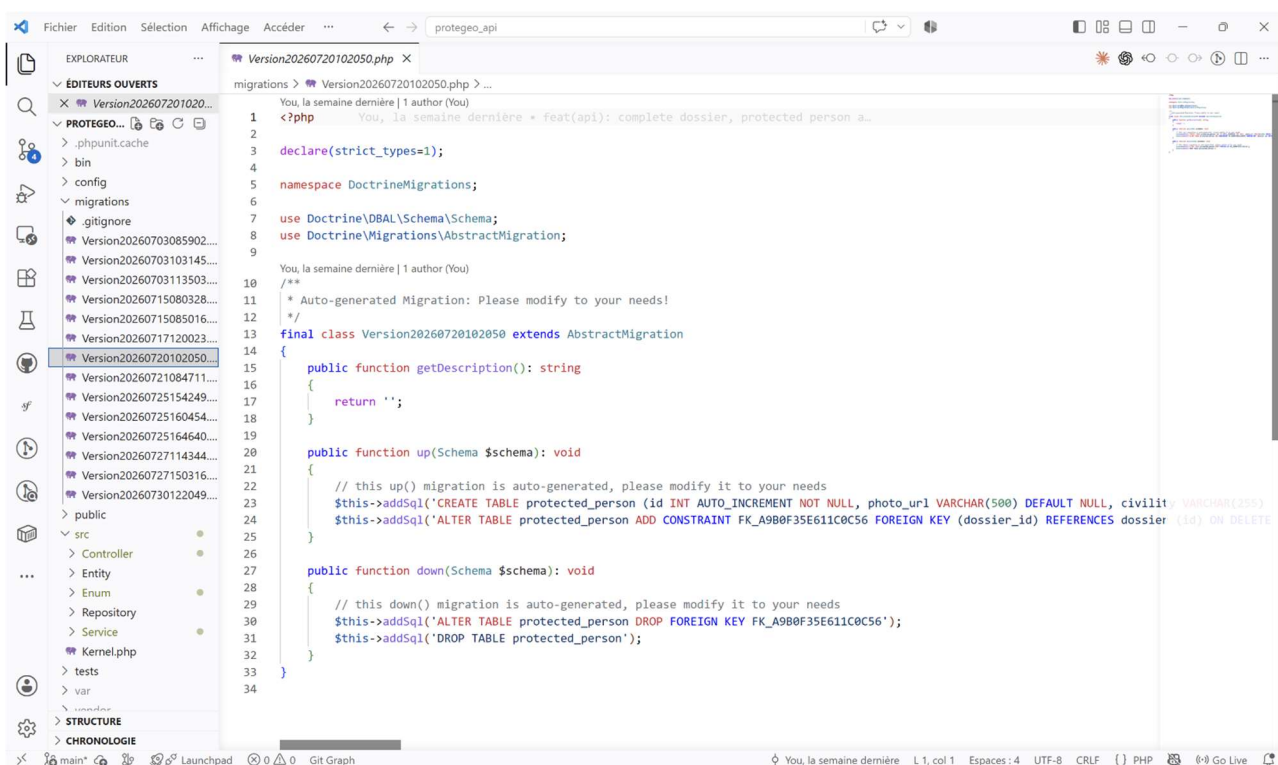


Figure 33 - Migration Doctrine générée automatiquement

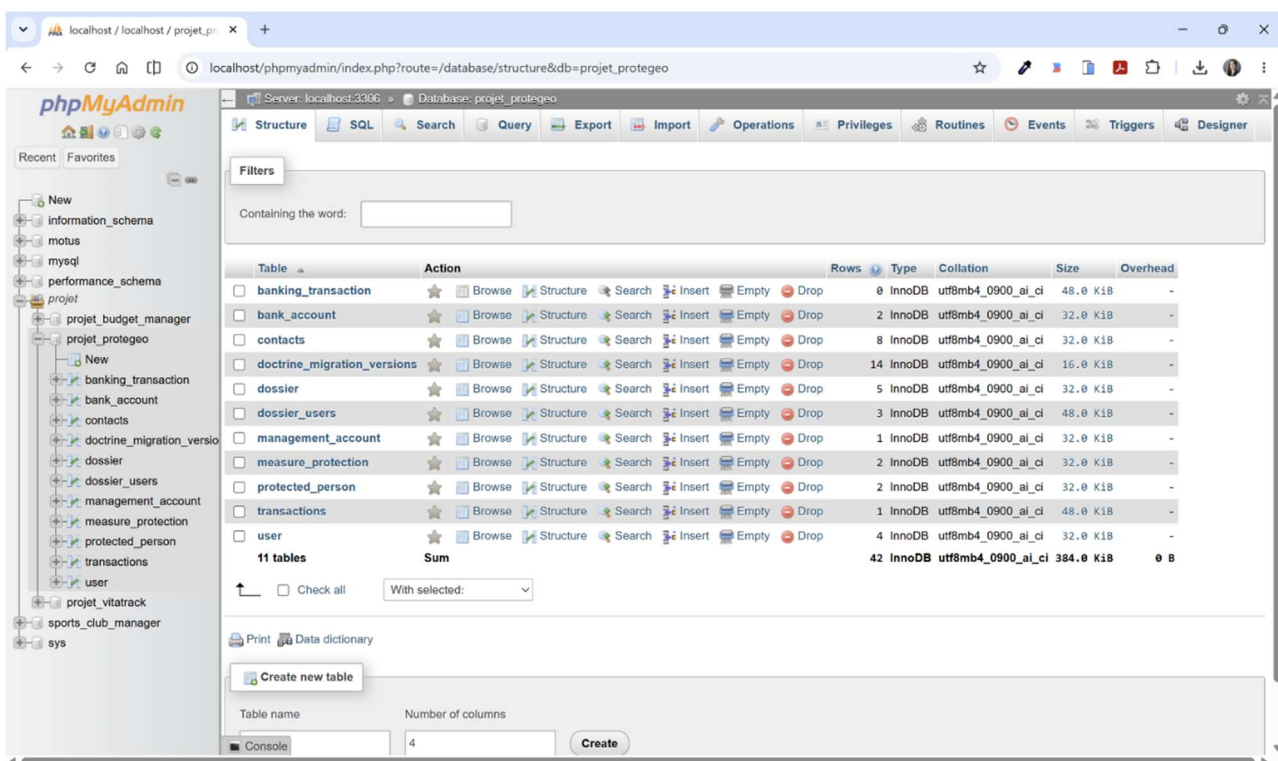


Figure 34 - Structure de la base de données dans phpMyAdmin

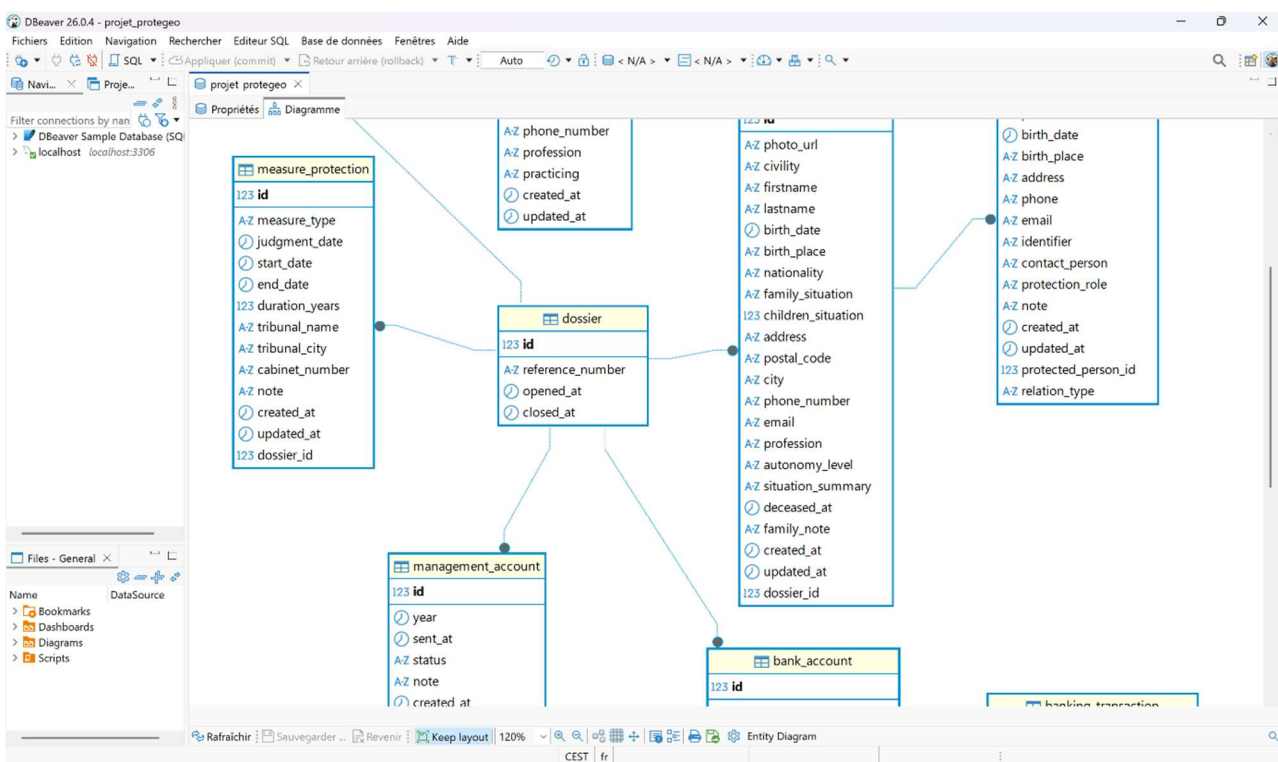


Figure 35 - Relations principales entre les entités de la base de données

F. Architecture et logique métier Back-end

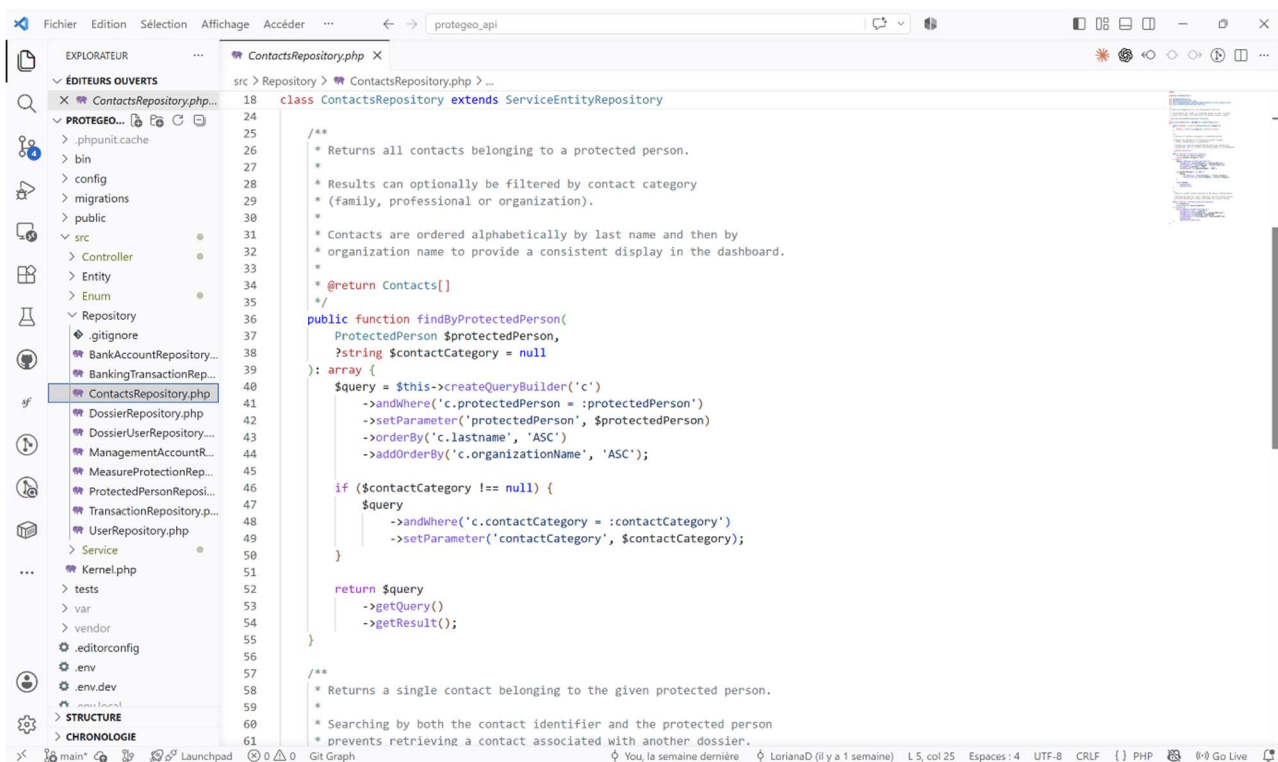


Figure 36 - Exemple d'une méthode personnalisée dans un repository Doctrine
Le repository centralise les requêtes d'accès aux données. Les méthodes personnalisées permettent de récupérer uniquement les informations correspondant au dossier concerné.

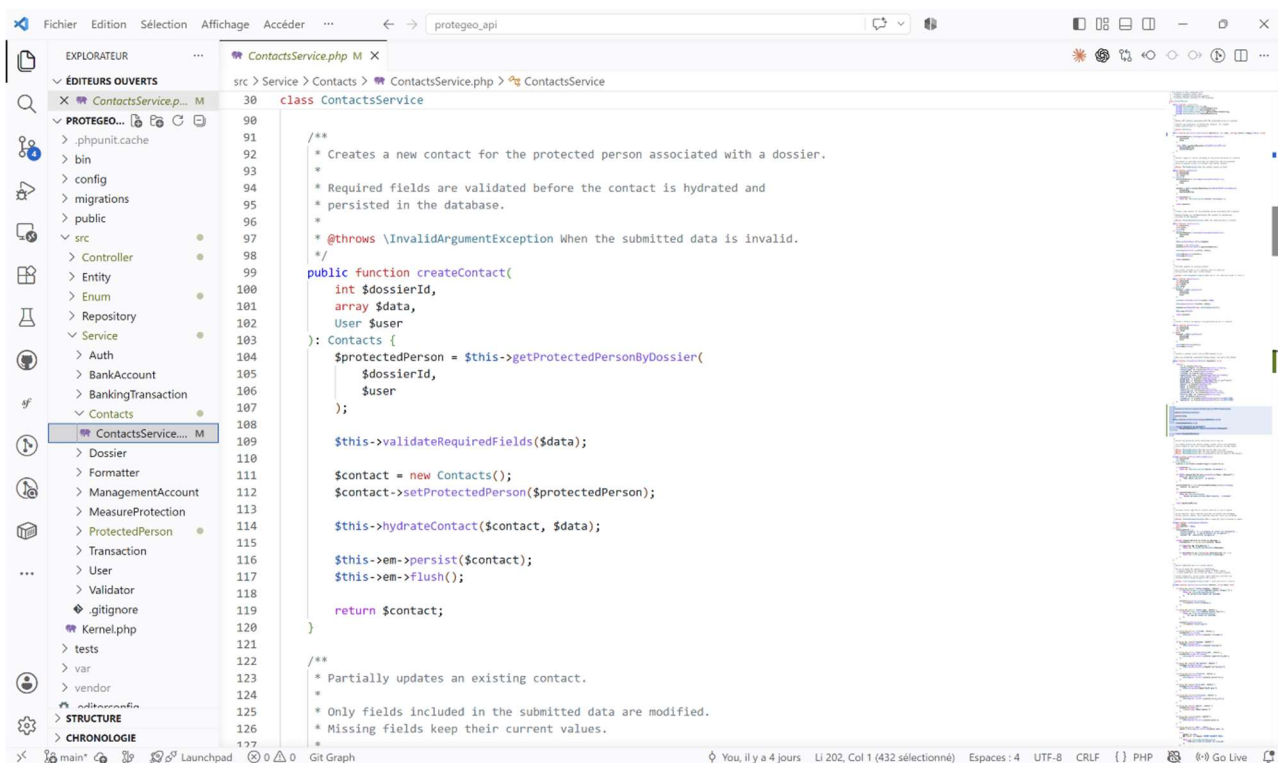


Figure 37 - Exemple d'opérations CRUD dans un service métier
Les méthodes du service permettent de créer, consulter, modifier et supprimer les données de manière centralisée.

Annexes techniques

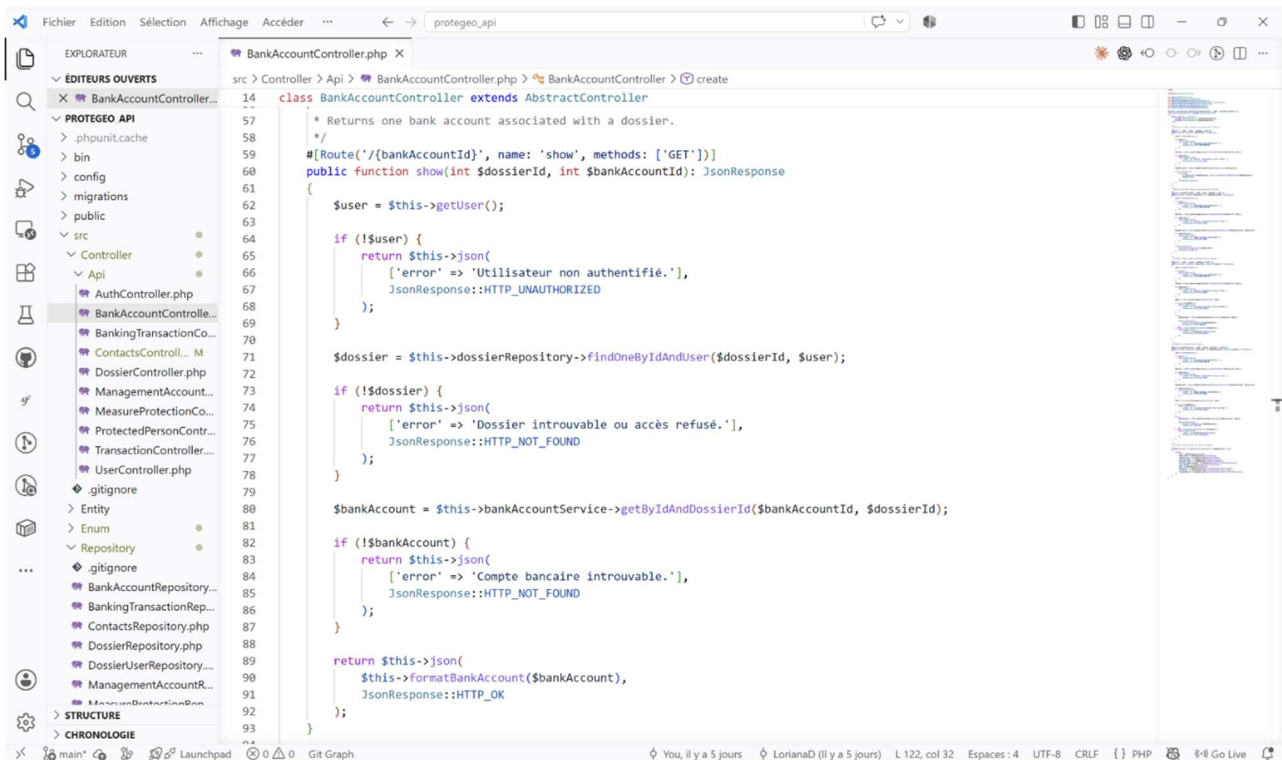


Figure 38 - Vérification des droits d'accès avant la consultation d'un compte bancaire
Le contrôleur vérifie l'authentification de l'utilisateur, son accès au dossier concerné, puis récupère uniquement les données associées à ce dossier.

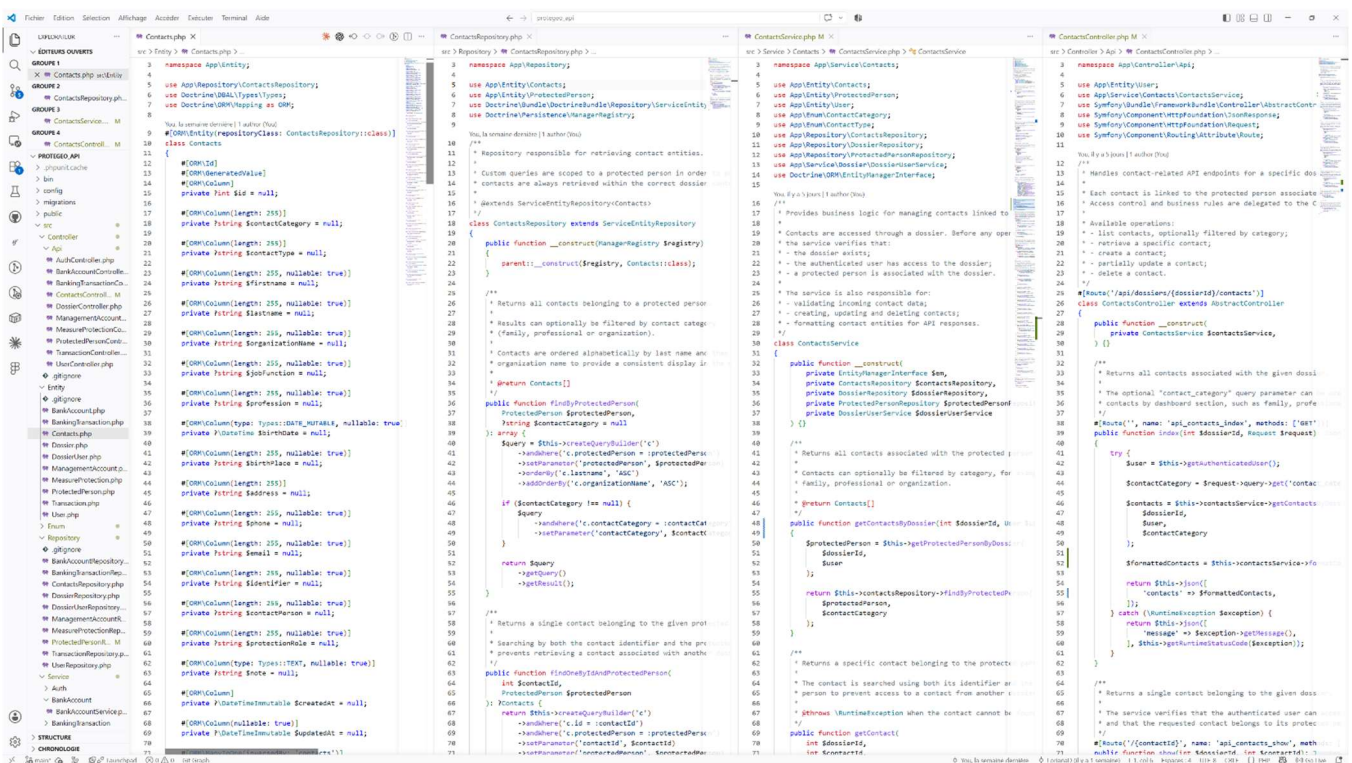


Figure 39 - Organisation de la logique métier liée aux contacts
La gestion des contacts illustre l'organisation du Back-end en plusieurs couches. L'entité représente les données, le repository centralise les requêtes vers la base de données, le service regroupe les traitements métier et le contrôleur prend en charge les requêtes et les réponses HTTP.

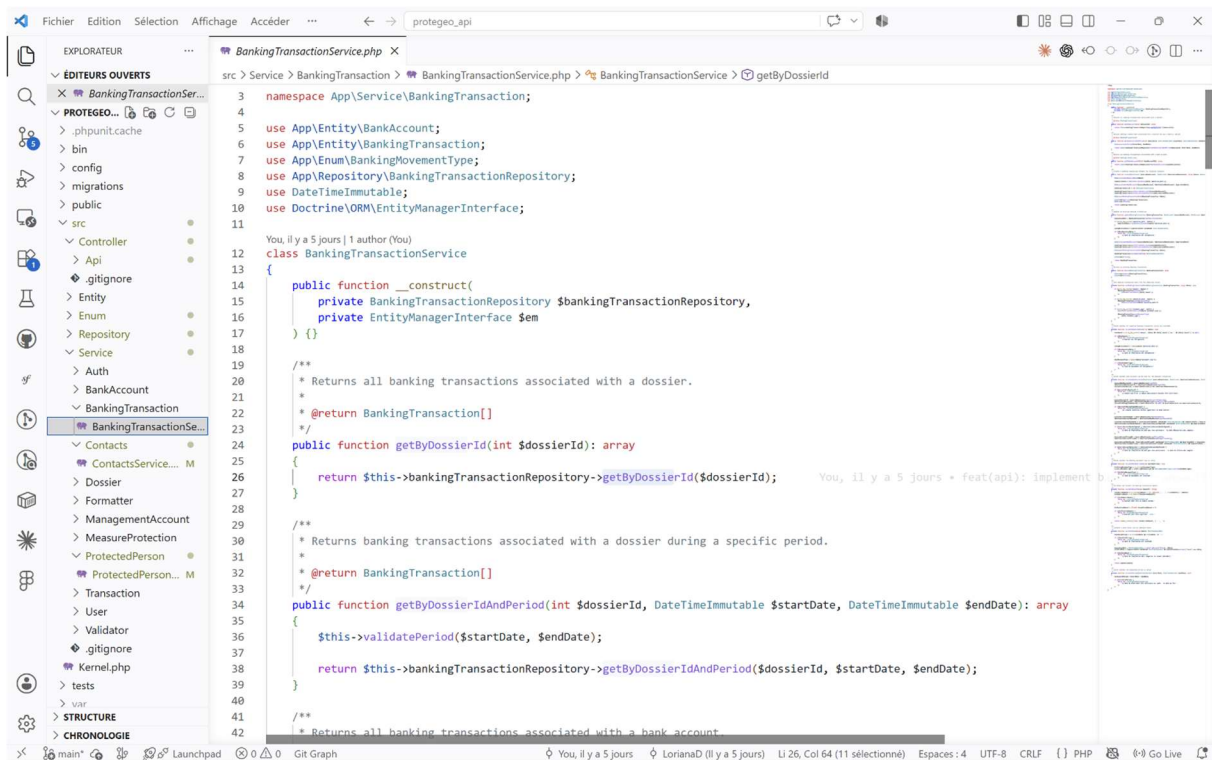


Figure 40 - Méthodes de consultation des mouvements bancaires

Le service délègue les recherches au repository en transmettant l'identifiant du dossier. Un mouvement précis est recherché à partir de son identifiant et du dossier auquel il doit appartenir, afin de limiter l'accès aux données autorisées.

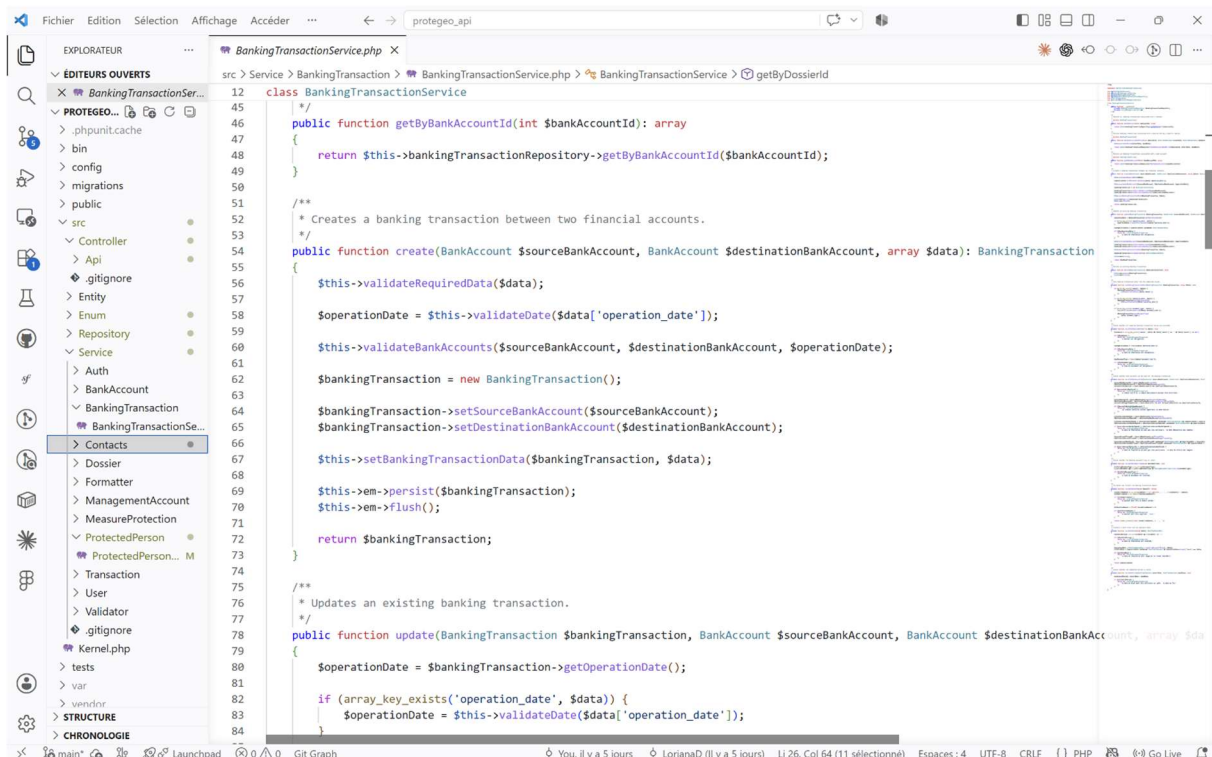


Figure 41 - Création d'un mouvement entre deux comptes bancaires

Lors de la création, le service récupère les comptes source et destinataire associés au dossier, renseigne les données du mouvement puis enregistre la nouvelle entité. Cette organisation garantit que le traitement est effectué de manière identique, quelle que soit la route depuis laquelle le service est appelé.

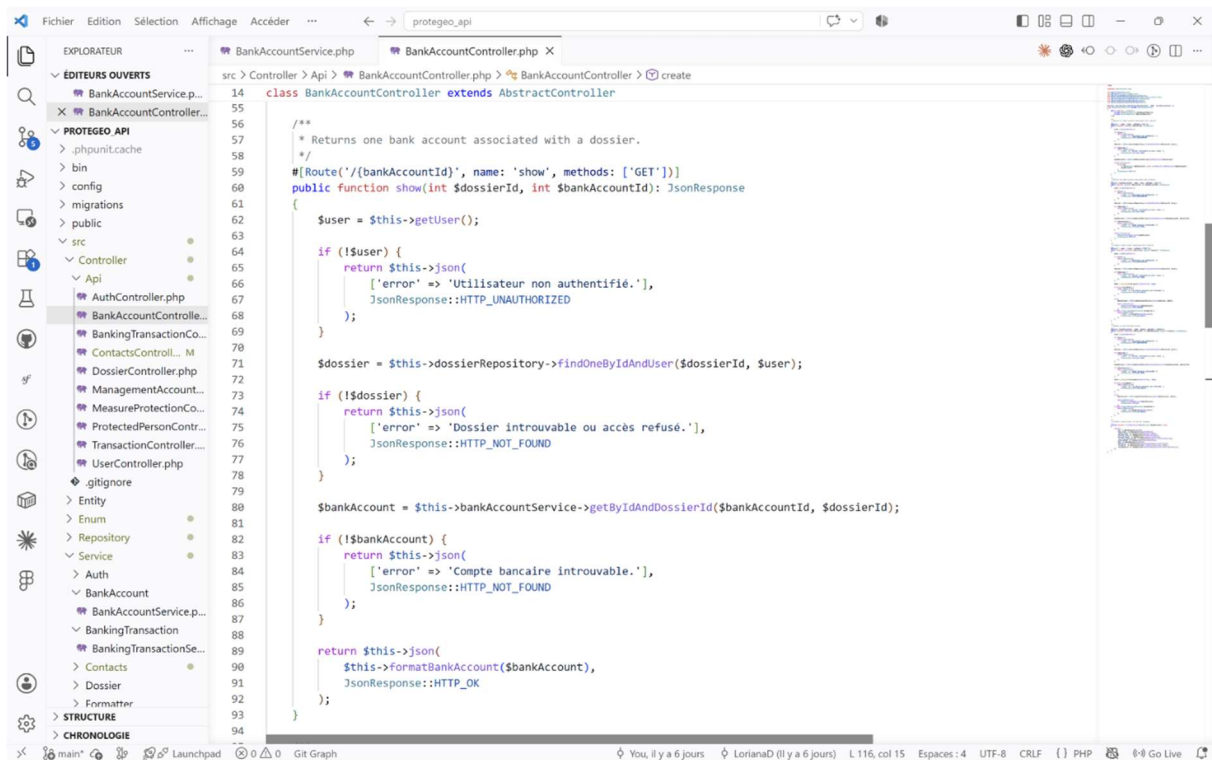


Figure 42 - Vérification de l'utilisateur et de son accès au dossier

Le contrôleur vérifie d'abord que l'utilisateur est authentifié. Le dossier est ensuite recherché à partir de son identifiant et de l'utilisateur connecté. Si aucune correspondance n'est trouvée, le traitement est interrompu avant l'accès au compte bancaire.

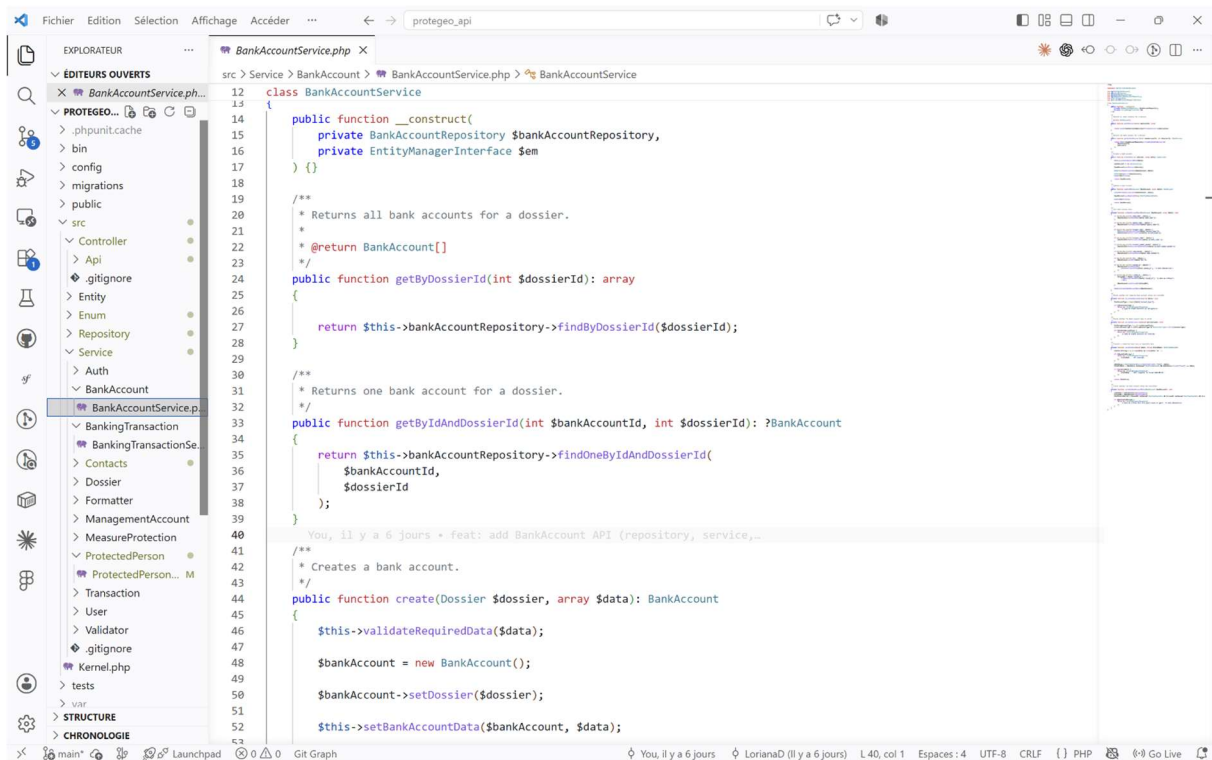


Figure 43 - Vérification de l'appartenance du compte bancaire au dossier

La méthode getByIdAndDossierId() transmet au repository l'identifiant du compte bancaire et celui du dossier. Le compte n'est retourné que lorsqu'il appartient bien au dossier consulté.

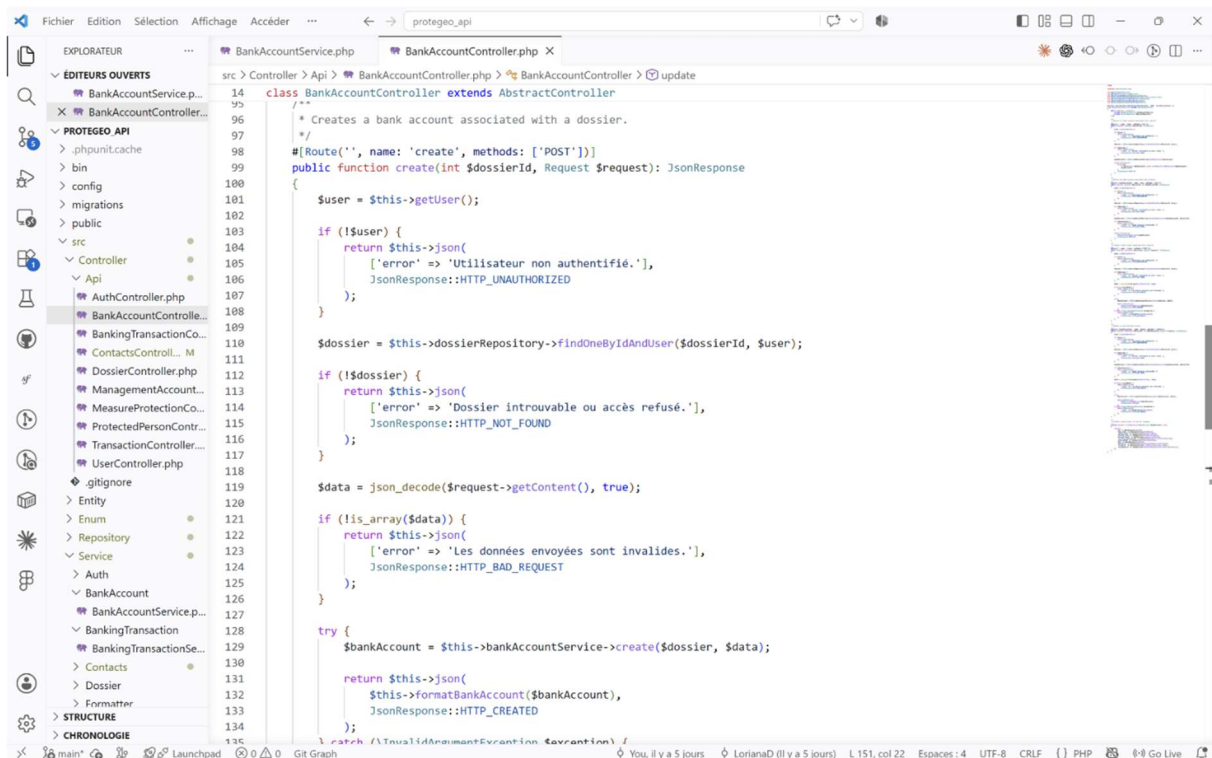


Figure 44 - Traitement d'une requête de création par le contrôleur
 La méthode `create()` reçoit une requête HTTP POST destinée à créer un compte bancaire. Le contrôleur vérifie l'utilisateur et son accès au dossier, récupère les données JSON envoyées, puis les transmet au service métier. Lorsque la création est terminée, le compte bancaire est formaté et retourné avec le code HTTP 201.

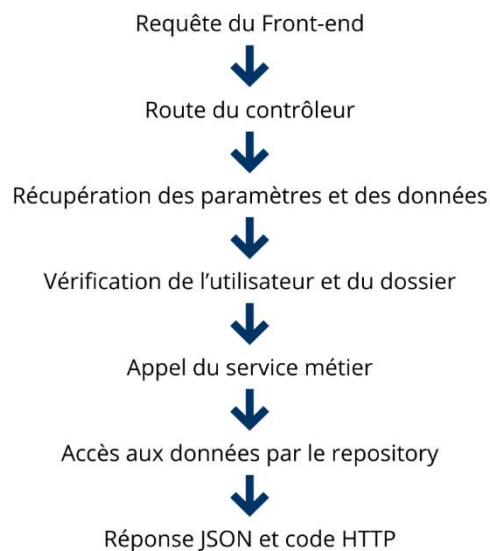


Figure 45
 Le traitement d'une requête suit un parcours organisé entre les différentes couches du Back-end. Le contrôleur coordonne les opérations, le service applique les règles métier et le repository communique avec la base de données.

G. Tests de l'API et gestion des erreurs

Annexes techniques

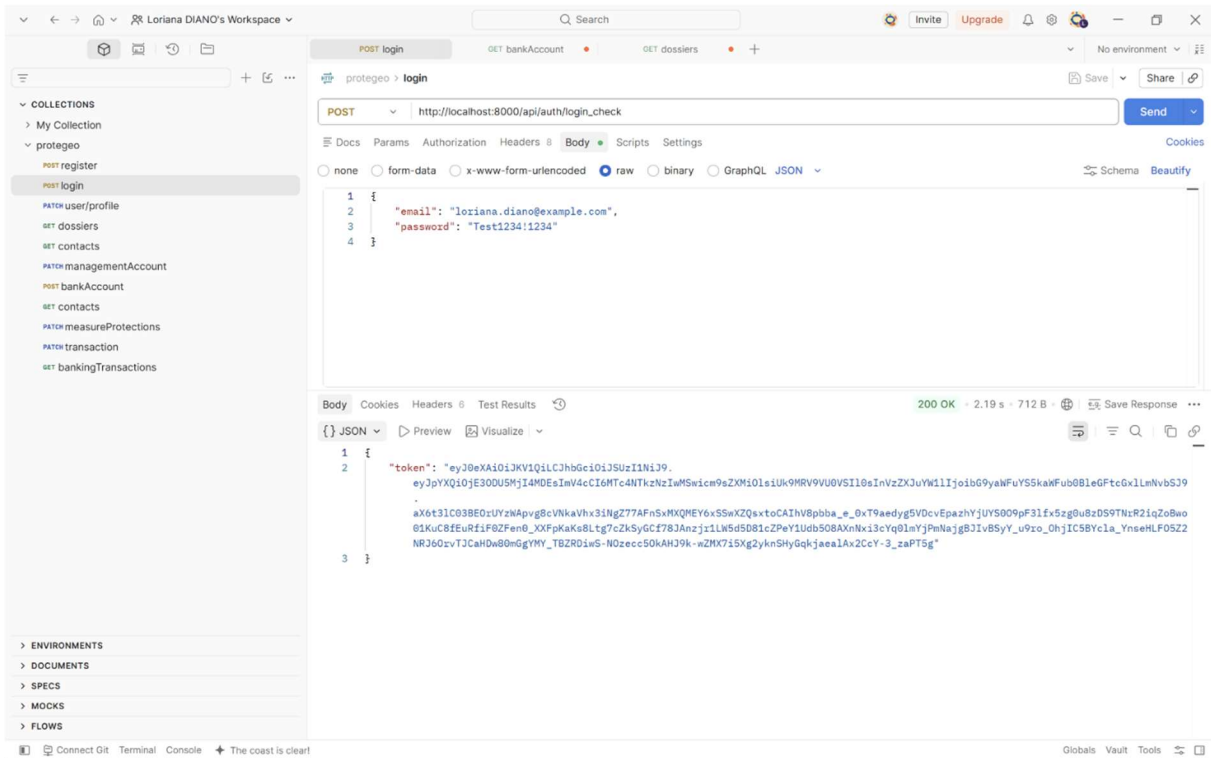


Figure 46 - Authentification de l'utilisateur avec Postman

La requête de connexion vérifie les identifiants transmis et retourne un jeton JWT. Ce jeton permet ensuite d'authentifier les requêtes envoyées vers les routes protégées de l'API.

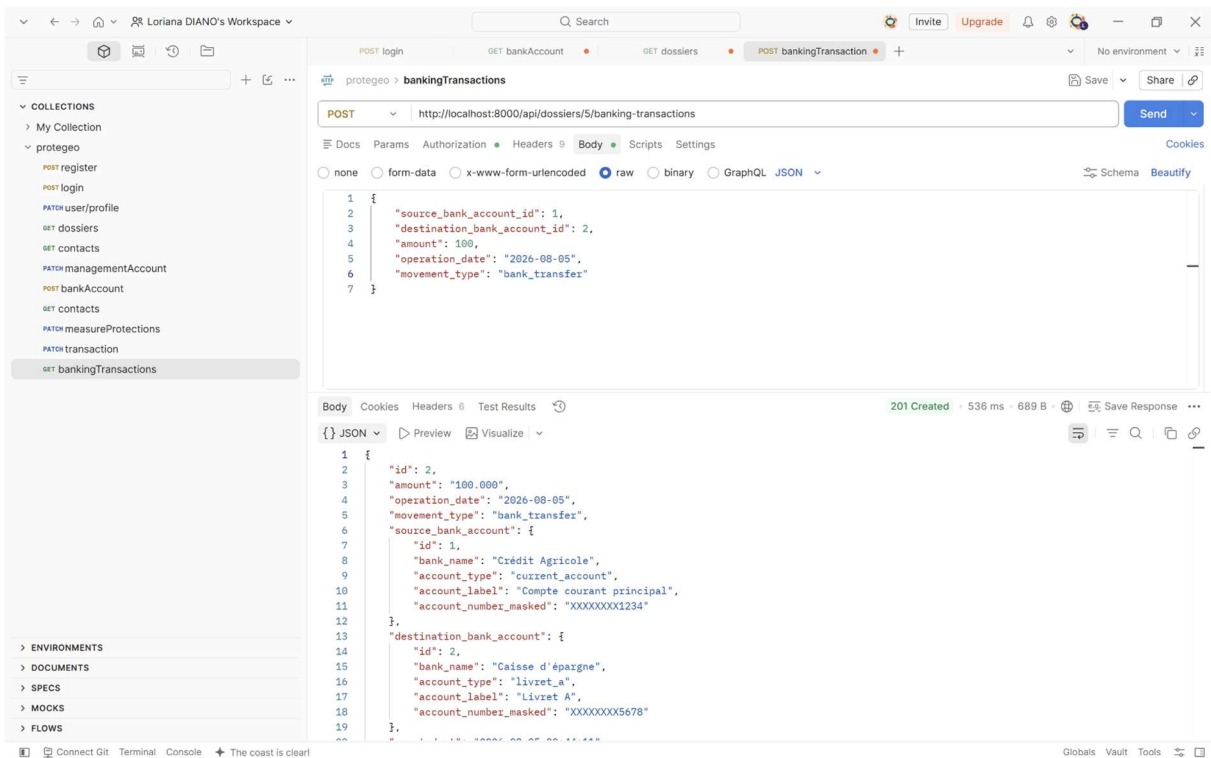


Figure 47 - Test réussi de création d'un mouvement bancaire

La requête authentifiée transmet les comptes source et destinataire ainsi que les informations du mouvement. Après validation par le contrôleur et le service métier, l'API crée la ressource et retourne une réponse JSON avec le code HTTP 201.

Annexes techniques

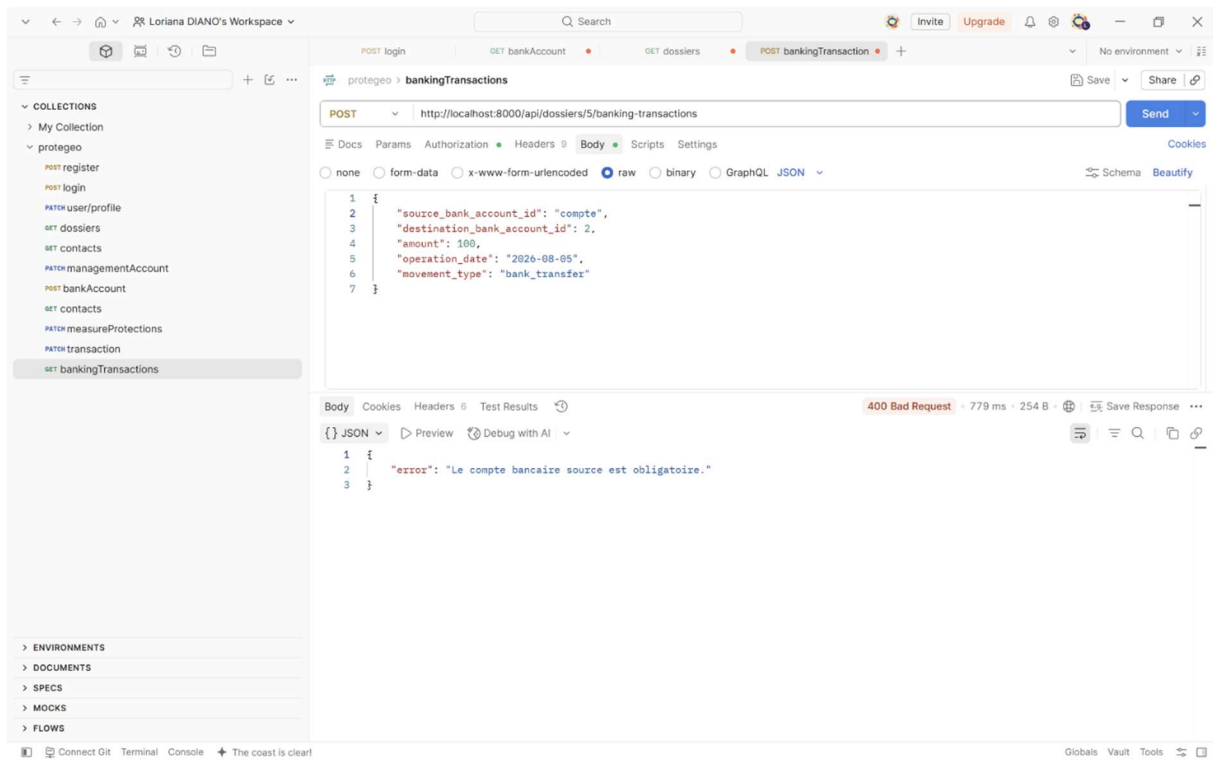


Figure 48 - Rejet d'une requête contenant des données invalides
Lorsque l'identifiant du compte source est absent ou invalide, le contrôleur interrompt le traitement. Aucune entité n'est enregistrée et l'API retourne un message d'erreur accompagné du code HTTP 400.

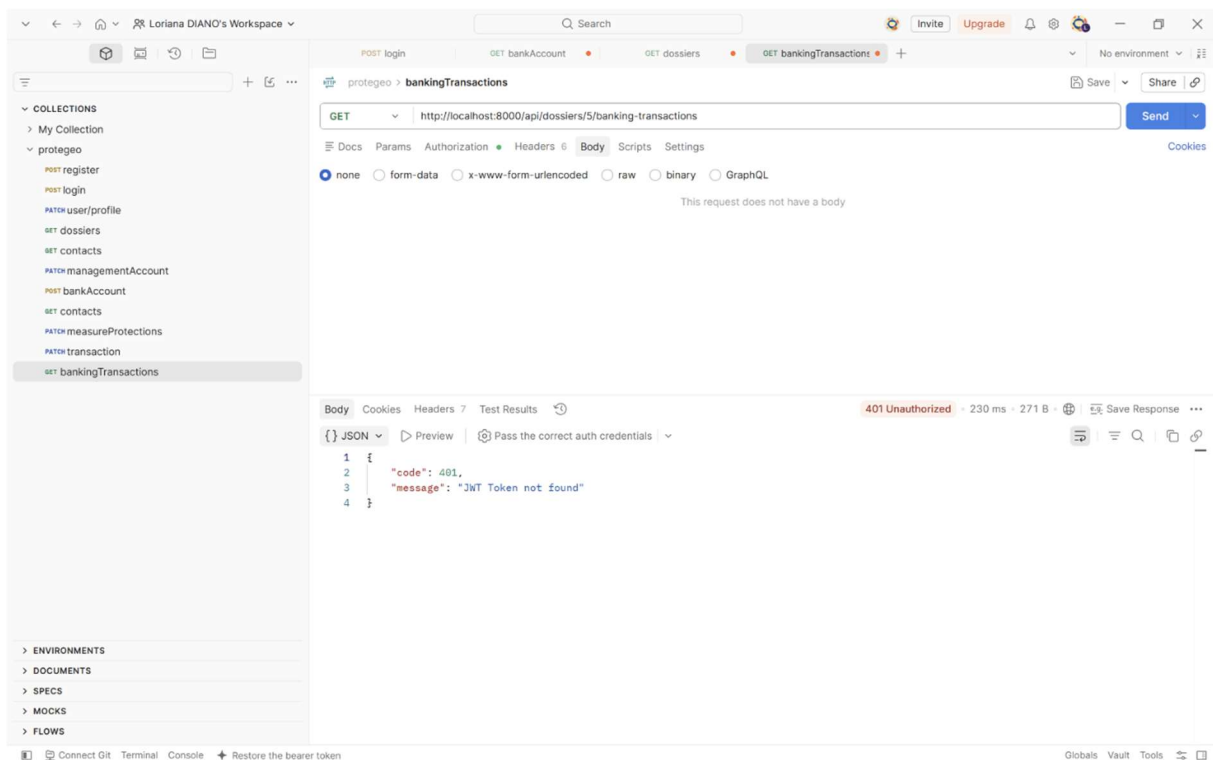


Figure 49 - Refus d'une requête non authentifiée
Une requête envoyée sans authentification ne peut pas accéder aux données protégées. L'API interrompt le traitement et retourne le code HTTP 401.

H.Déploiement et mise en production

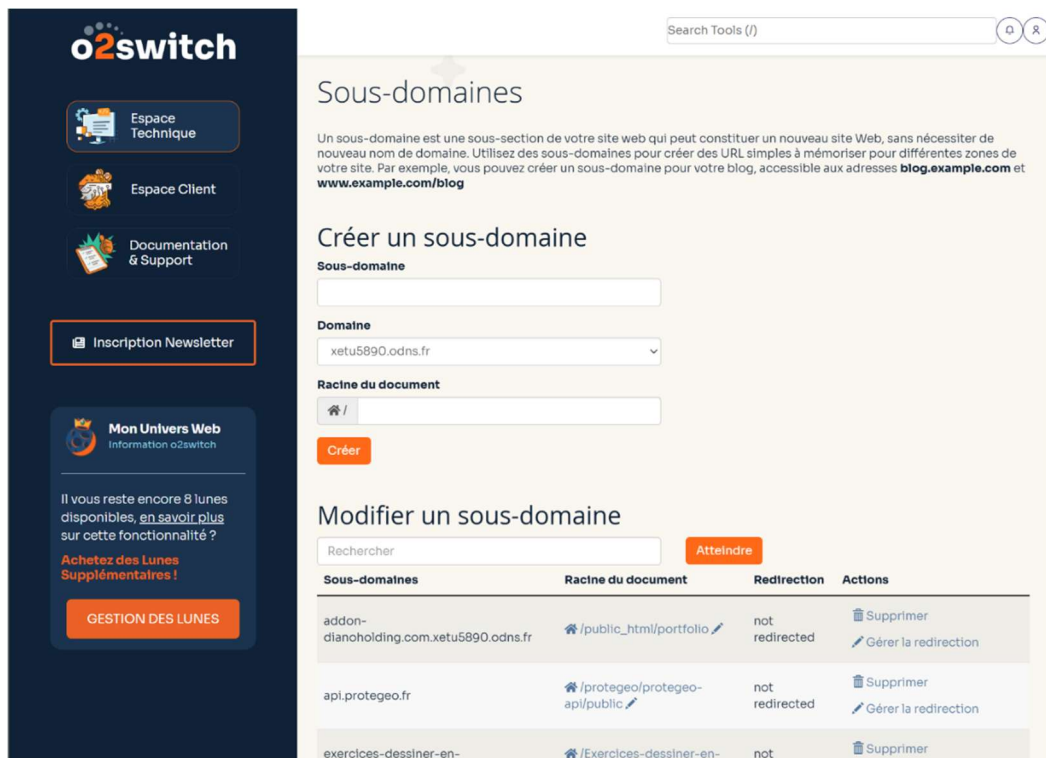


Figure 50 - Configuration du sous-domaine de l'API

Le sous-domaine **api.protegeo.fr** utilise comme racine le dossier **protegeo/protegeo-api/public**. Seul le point d'entrée **public** de Symfony est ainsi exposé sur Internet.

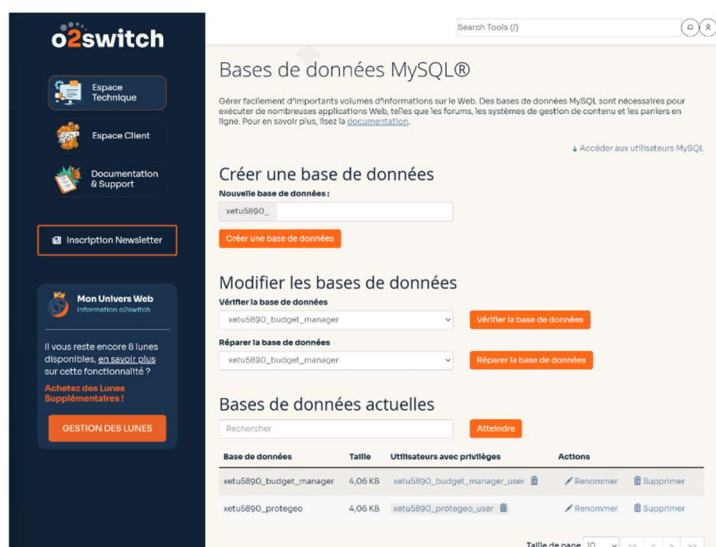


Figure 51 - Création de la base de données de production

Une base MySQL et un utilisateur dédié ont été créés pour l'environnement de production. L'utilisateur possède les privilèges nécessaires pour permettre à Doctrine d'exécuter les migrations et de manipuler les données.

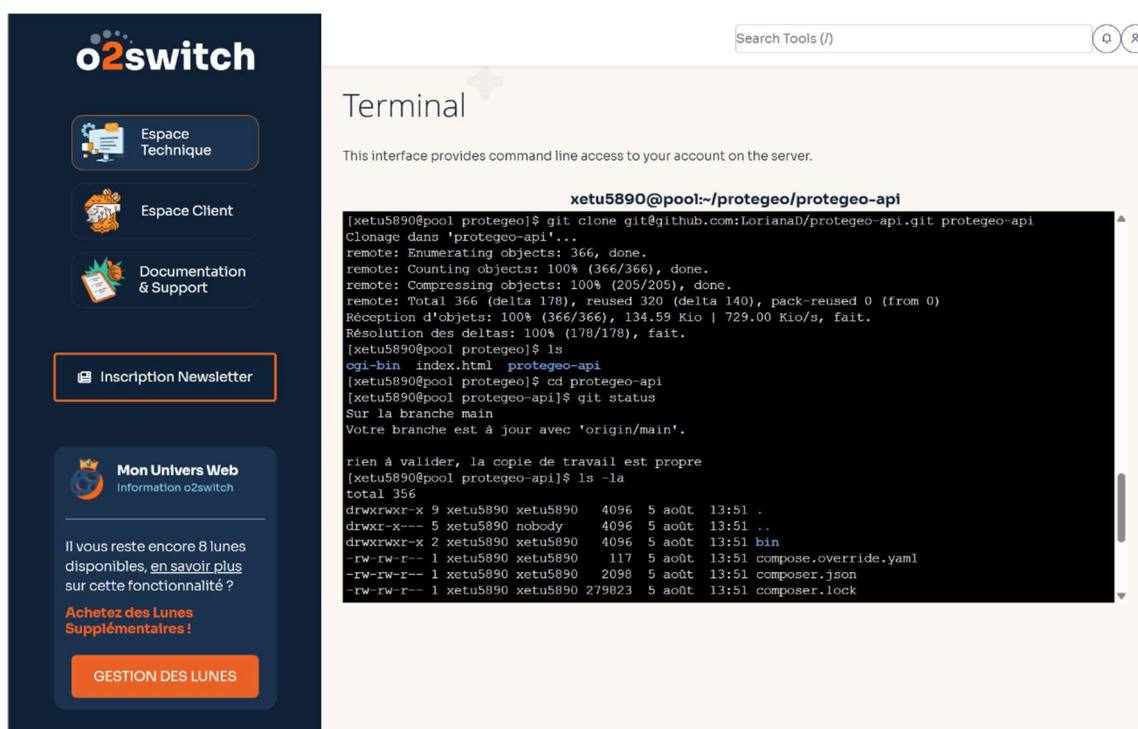


Figure 52 - Déploiement du projet Symfony avec Git
Le dépôt GitHub de l'API est cloné dans le répertoire de production. La commande `git status` permet ensuite de vérifier la branche utilisée et l'état de la copie de travail.

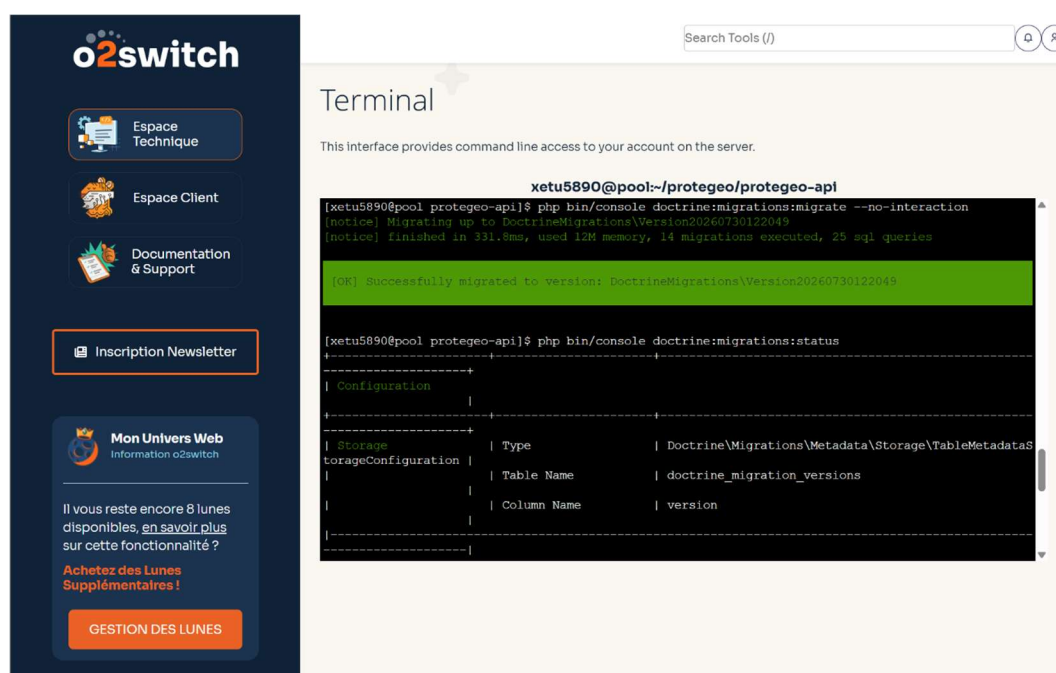


Figure 53 - Exécution des migrations Doctrine
Les migrations Doctrine ont été exécutées sur la base de production. Le message de réussite confirme la création de la structure correspondant aux entités de l'application.

Annexes techniques

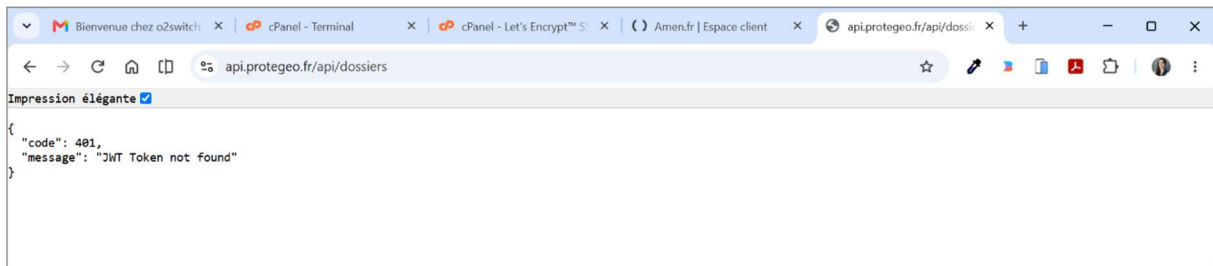


Figure 54 - Vérification de l'API déployée

La route protégée est accessible depuis le sous-domaine de production. En l'absence de jeton JWT, l'API renvoie une réponse HTTP 401, confirmant que la sécurité est correctement appliquée.

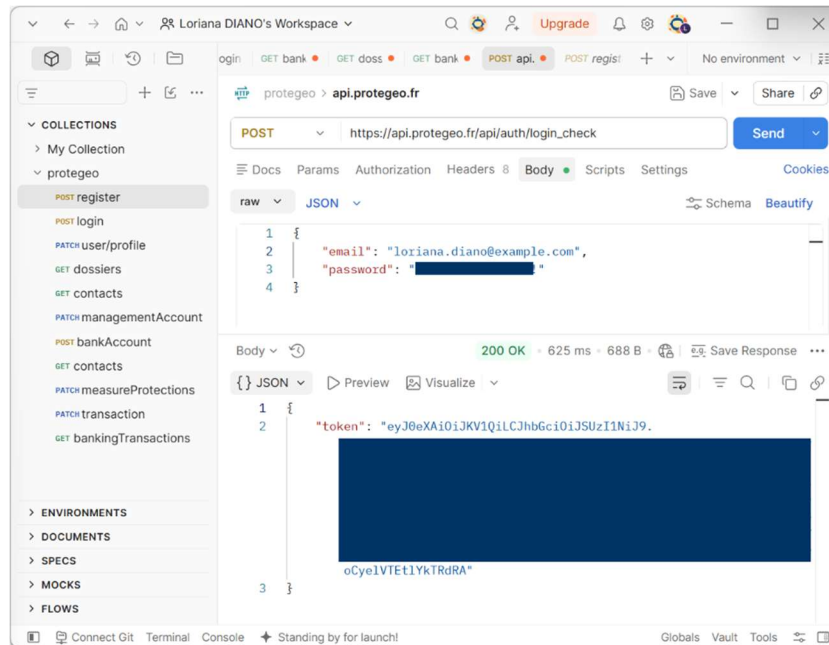


Figure 55 - Test de l'authentification sur l'API de production
Une requête POST est envoyée vers la route d'authentification de l'API de production. Lorsque les identifiants sont valides, l'API retourne un jeton JWT avec un code HTTP 200. Ce jeton permet ensuite d'authentifier les requêtes vers les routes protégées.

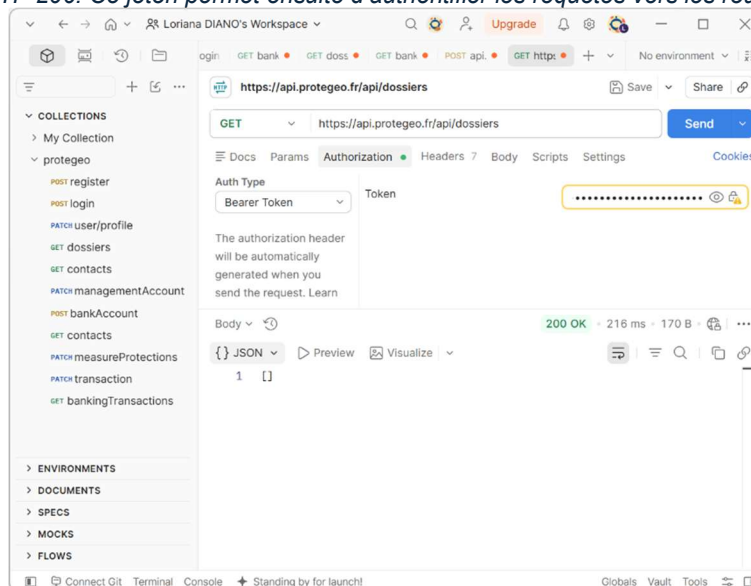


Figure 56 - Test d'une route protégée avec un jeton JWT

Annexes techniques

Le jeton JWT obtenu lors de la connexion est transmis dans l'en-tête d'autorisation de la requête. L'API accepte alors l'accès à la route protégée et retourne un code HTTP 200. Le tableau vide indique qu'aucun dossier n'est encore enregistré dans la base de données de production.

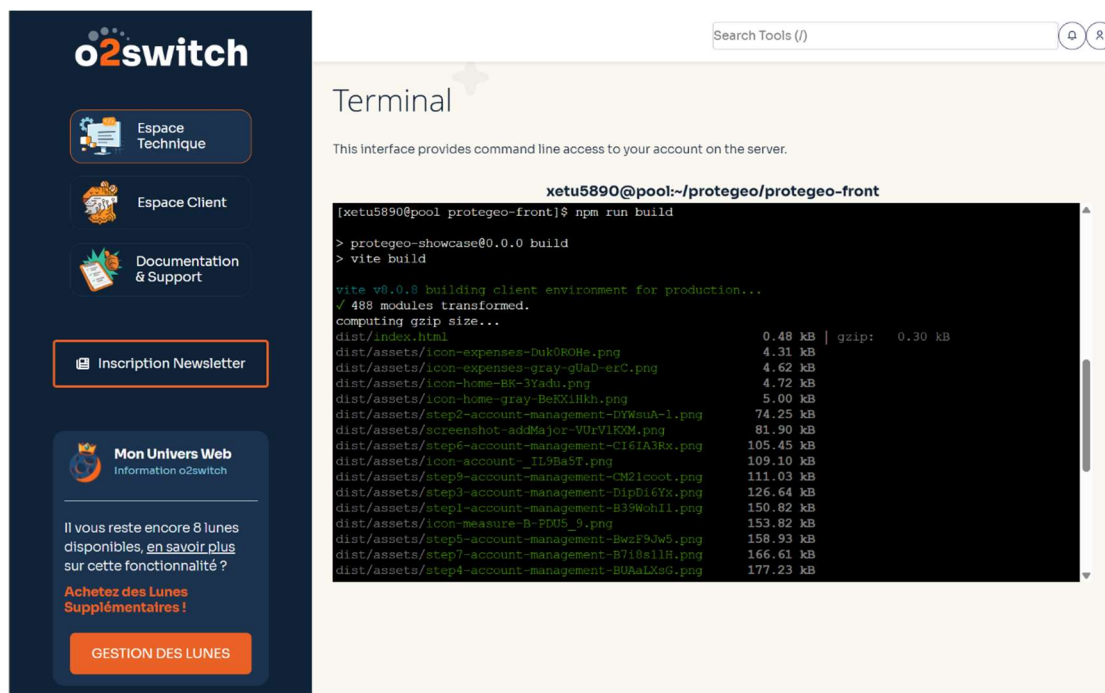


Figure 57 - Compilation du Front-end React avec Vite

La commande `npm run build` génère dans le dossier `dist` une version optimisée du Front-end. Vite compile les composants React, les styles et les ressources nécessaires à l'affichage de l'application en production.

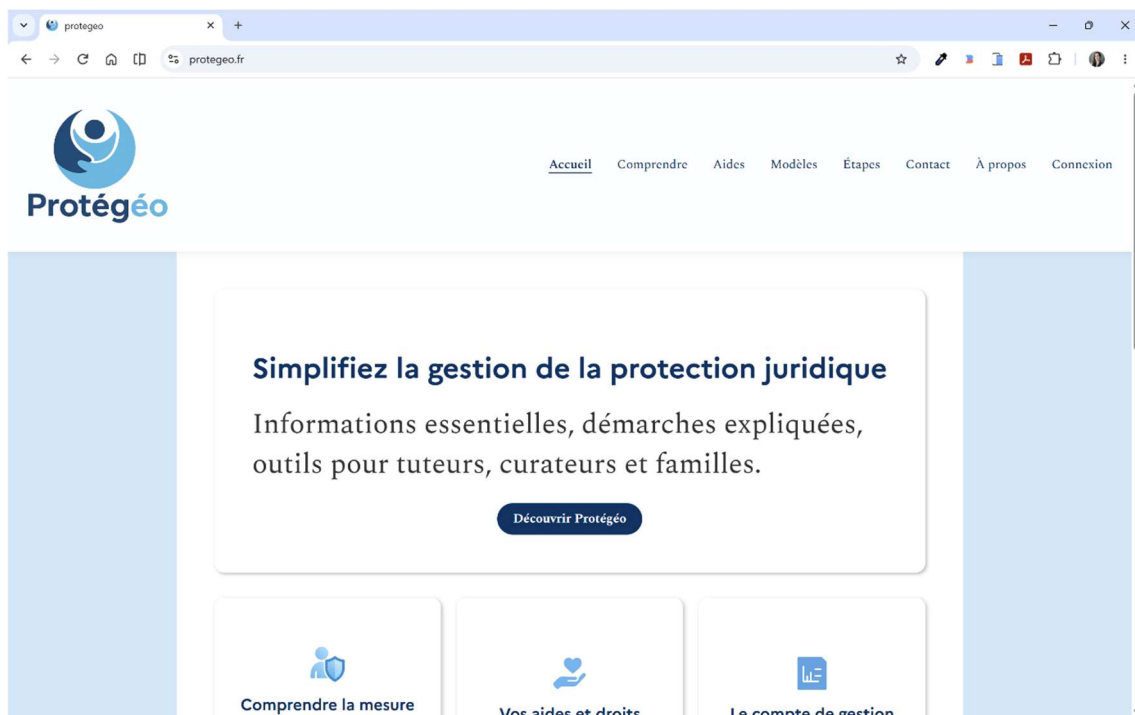


Figure 58 - Front-end de Protégéo accessible en production

La page d'accueil est accessible depuis le domaine `protegeo.fr`. Son affichage confirme que les fichiers compilés ont correctement été publiés sur l'hébergement et sont servis par le domaine principal.

o2switch

Espace Technique
Espace Client
Documentation & Support
Inscription Newsletter

Mon Univers Web
Information o2switch

Il vous reste encore 8 lunes disponibles, [en savoir plus](#) sur cette fonctionnalité ?
Achetez des Lunes Supplémentaires !
GESTION DES LUNES

Let's Encrypt™ SSL

Let's Encrypt est une autorité de certification gratuite, libre, vous permettant de créer un certificat SSL sans aucun frais. Cette page permet aux hébergés d'activer en quelques clics un certificat pour un nom de domaine configuré.

A noter : Les certificats Let's Encrypt sont mis à disposition sans aucune garantie : pour un site commercial nous vous recommandons un certificat payant.

Les certificats Let's Encrypt disposent d'une durée limitée de validité. Néanmoins, nous réalisons une tentative de renouvellement automatique. Vous recevrez alors un courriel informant du déroulement de cette procédure.
Etat du service Let's Encrypt (fetching ...)
[Voir les paramètres](#)

Besoin de forcer l'utilisation du HTTPS sur un site internet ?

Générer et d'installer un certificat SSL avec cet outil Let's Encrypt **ne suffira pas à forcer l'utilisation du HTTPS** sur un site internet. Le site sera **accessible en HTTPS** mais le **HTTPS ne sera pas forcé, par défaut**, il n'y aura pas de redirection de HTTP vers HTTPS.

Si vous souhaitez **forcer l'utilisation du HTTPS**, il faut faire des **réglages supplémentaires** sur le site. Ces réglages dépendent de la technologie que vous avez utilisée pour créer votre site (CMS).

- [Comment forcer le HTTPS sur un site WordPress ?](#)
- [Comment forcer le HTTPS sur un site générique ?](#)

Vos domaines avec des certificats Let's Encrypt

Show 10 entrées

Nom de domaine	Hôtes alternatifs	État	Validation	Expiration	Actions
apl.protegeo.fr	www.apl.protegeo.fr	Installé	http-01	03 Nov 2026	Supprimer Réinstaller Voir
exercices-dessiner-en-	www.exercices-dessiner-en-	Installé	http-01	13 Oct 2026	Supprimer Réinstaller

Figure 59 - Installation du certificat SSL pour l'API
Un certificat SSL Let's Encrypt est installé pour le sous-domaine `api.protegeo.fr`. Il permet de sécuriser les échanges entre le Front-end React et l'API Symfony avec le protocole HTTPS.